Grado en Ingeniería del Software / Informática

Fundamentos de la Web

Manual de Referencia Teórico-Práctico para Ingenieros de Software

Autor:
David Sebastián Sticea Covaciu

Basado en la docencia de Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto e Iván Chicano

Universidad Rey Juan Carlos (URJC) | Curso Académico 2024–2025

Versión del documento: 1.0
Compilado: 8 de noviembre de 2026

Estado: Completo y Revisado
Licencia: CC BY-SA 4.0

Prefacio y Guía del Estudiante

El presente documento constituye un manual exhaustivo y riguroso de la asignatura **Fundamentos de la Web**. Ha sido diseñado meticulosamente con una premisa formativa fundamental: *permitir que cualquier persona que comience con cero conocimientos previos adquiera una comprensión total y profunda de la ingeniería web, comprendiendo no solo la sintaxis de las tecnologías actuales, sino los principios de diseño, protocolos de red, decisiones arquitectónicas y compromisos técnicos (trade-offs) que gobiernan la Web moderna.*

Estructura de la Obra

El temario se organiza en capítulos temáticos consolidados que integran la visión teórica y la aplicación práctica:

- **Capítulo 1: Fundamentos de la Web, Arquitecturas y Formatos de Datos.** Aborda la diferencia entre Internet y la WWW, los protocolos TCP/IP, el funcionamiento del protocolo HTTP, la evolución de los servidores web de ficheros a aplicaciones dinámicas, la taxonomía de las seis arquitecturas web contemporáneas (desde páginas estáticas hasta Single Page Applications con hidratación) y los formatos fundamentales de intercambio de datos: XML y JSON, junto con esquemas de validación y ejercicios resueltos de modelado.
- **Capítulo 2: Tecnologías de Cliente: HTML5, CSS3 y Bootstrap 5.** Trata la tríada del cliente web comenzando por HTML5 estructural y semántico, accesibilidad web (a11y), tablas de datos y formularios interactivos seguros; continúa con CSS3 avanzado, el algoritmo de la cascada y especificidad, el modelo de caja, Flexbox y CSS Grid Layout en profundidad; y concluye con el framework Bootstrap 5, su sistema de cuadrícula responsiva de 12 columnas, catálogo de componentes y estrategias de personalización, incluyendo soluciones exhaustivas a los ejercicios propuestos en el programa académico.

«El software es una combinación entre arte e ingeniería; la Web es el lienzo interactivo más universal que la humanidad ha creado jamás.»

Índice general

Prefacio y Guía del Estudiante **1**

1. Fundamentos de la Web, Arquitecturas y Formatos de Datos **5**
1.1. Historia de la Web y Principios de Red 5
1.1.1. Diferencia Fundamental: Internet vs. World Wide Web 5
1.1.2. Nacimiento de la Web: El CERN y los Primeros Estándares 6
1.1.3. La Popularización y las Guerras de Navegadores 6
1.1.4. Los Organismos de Estandarización y el «Living Standard» 6
1.1.5. La Trinidad del Front-End y su Cronología 7
1.2. Arquitectura de Red y el Modelo Cliente-Servidor 7
1.2.1. Características del Cliente y del Servidor 8
1.2.2. Direccionamiento IP, Nombres de Dominio y NAT 8
1.2.3. Puertos de Red y la Regla de Exclusión de Procesos 9
1.3. El Protocolo HTTP en Profundidad 10
1.3.1. Estructura Canónica de una URL 10
1.3.2. Anatomía de los Mensajes HTTP: Peticiones y Respuestas 10
1.3.3. Métodos HTTP Semánticos e Idempotencia 11
1.3.4. Familias de Códigos de Estado HTTP 12
1.3.5. El Dilema del Estado: Cookies, Sesiones y Tokens JWT 13
1.4. Evolución del Servidor Web y Tecnologías Backend 13
1.4.1. Servidores Web Estáticos (1990–1993) 13
1.4.2. El Nacimiento de la Web Dinámica: CGI (Common Gateway Interface) 13
1.4.3. El Ecosistema Backend Moderno y las Bases de Datos 14
1.5. Taxonomía y Arquitecturas de las Aplicaciones Web 15
1.5.1. La Matriz Arquitectónica 2×2 15
1.5.2. Análisis Detallado de las 6 Arquitecturas 15
1.6. Formatos de Intercambio de Datos: XML y JSON 18
1.6.1. XML (eXtensible Markup Language) 19
1.6.2. JSON (JavaScript Object Notation) 20
1.6.3. Comparativa Técnica Exhaustiva: XML vs. JSON 21
1.7. Ejercicios Prácticos Resueltos del Tema 1 21

2. Tecnologías de Cliente Web: HTML5, CSS3 y Bootstrap 5 **26**
2.1. HTML5: Estructura, Semántica y Formularios 26
2.1.1. Definición y Rol en el Ecosistema Web 26
2.1.2. Diferencias Cruciales: HTML vs. XML vs. XHTML 26
2.1.3. Entorno de Trabajo, Editores y Documentación Oficial 27
2.1.4. Anatomía de un Documento HTML5 Estándar 27

2.1.5. Elementos de Texto y Jerarquía Semántica . . . 28
2.1.6. Hiperenlaces (`<a>` - Anchor Tags) . . . 29
2.1.7. Imágenes y Contenido Multimedia . . . 30
2.1.8. Listas de Elementos . . . 31
2.1.9. Tablas de Datos Semánticas . . . 31
2.1.10. HTML5 Semántico: Maquetación Estructural Moderna . . . 32
2.1.11. Formularios Web Interactivos (`<form>`) . . . 33
2.1.12. Elementos Genéricos: `<div>` y `<span>` . . . 35
2.2. CSS3: Cascada, Box Model, Flexbox y Grid Layout . . . 35
2.2.1. Definición y Principios de CSS . . . 35
2.2.2. Métodos de Inclusión de CSS . . . 35
2.2.3. Sintaxis de una Regla CSS . . . 36
2.2.4. El Algoritmo de la Cascada, Especificidad y Herencia . . . 36
2.2.5. Colores y Variables CSS (Custom Properties) . . . 37
2.2.6. Tipografía, Fuentes y Unidades de Medida . . . 38
2.2.7. El Modelo de Caja (*Box Model*) . . . 39
2.2.8. Catálogo Avanzado de Selectores CSS . . . 39
2.2.9. Comportamiento de Visualización: La Propiedad `display` . . . 40
2.2.10. Posicionamiento Tradicional (`position`) . . . 41
2.2.11. Maquetación Moderna 1: Flexbox (Flexible Box Layout) . . . 41
2.2.12. Maquetación Moderna 2: CSS Grid Layout . . . 43
2.3. Bootstrap 5: Framework Front-End para Diseño Rápido . . . 44
2.3.1. Definición, Filosofía y Evolución . . . 44
2.3.2. Inclusión de Bootstrap en Proyectos . . . 44
2.3.3. El Sistema de Maquetación: Contenedores y Grid de 12 Columnas . . . 45
2.3.4. Paleta Semántica de Colores y Tipografía . . . 46
2.3.5. Catálogo de Componentes UI Esenciales . . . 46
2.4. Ejercicios Prácticos Resueltos del Tema 2 . . . 47
2.4.1. Ejercicio 1 de HTML: Creación de Página Semántica sin Estilos . . . 47
2.4.2. Ejercicios Resueltos de CSS . . . 51
2.4.3. Ejercicio 14 de Bootstrap Resuelto: Réplica de Portal Web . . . 55

3. Tecnologías del Servidor Web: JavaScript, Node.js, Express y MongoDB 61
3.1. Introducción a JavaScript y Runtimes (3.1.1) . . . 61
3.1.1. Naturaleza y Evolución de JavaScript . . . 61
3.1.2. JavaScript en el Navegador vs Fuera del Navegador . . . 62
3.1.3. Arquitectura, Rendimiento y Concurrencia en Node.js . . . 63
3.1.4. Versiones de Node.js y Entorno de Desarrollo . . . 64
3.2. El Lenguaje JavaScript (3.1.2) . . . 65
3.2.1. Comparativa de Paradigmas: Java vs JavaScript . . . 65
3.2.2. Sintaxis Básica, Declaración de Variables y Tipos . . . 65
3.2.3. Operadores, Coerción de Tipos e Igualdad Estricta . . . 67
3.2.4. Arrays en JavaScript . . . 67
3.2.5. Sentencias de Control de Flujo y Evaluación Booleana . . . 68
3.2.6. Funciones, Ámbitos, Clausuras y Funciones Flecha . . . 69
3.2.7. Programación Orientada a Objetos: Literales y Clases ES6 . . . 71
3.2.8. Manejo de Excepciones y Clases de Error Personalizadas . . . 72
3.2.9. Ejercicios Resueltos del Bloque 3.1.2 . . . 73
3.3. Estructuras de Datos en JavaScript (3.1.3) . . . 78
3.3.1. Almacenamiento en Memoria y Complejidad Algorítmica . . . 78
3.3.2. Conjuntos (`Set`) . . . 78

3.3.3. Mapas (`Map`) . . . 79
3.3.4. Patrones de Recorrido e Iteración . . . 80
3.3.5. Ordenación y Búsqueda . . . 80
3.3.6. Ejercicios Resueltos del Bloque 3.1.3 . . . 81
3.4. Modularidad en JavaScript: ESM y Node.js (3.1.4 y 3.1.5) . . . 83
3.4.1. Principios de Modularidad y ECMAScript Modules (ESM) . . . 83
3.4.2. Módulos Nativos de Node.js y el Prefijo `node:` . . . 84
3.4.3. Gestión de Paquetes con npm y Coexistencia con CommonJS . . . 85
3.4.4. Ejercicios Resueltos del Bloque 3.1.4 y 3.1.5 . . . 86
3.5. Aplicaciones Web con Node.js y Express (3.2.1) . . . 87
3.5.1. Arquitectura de Express y Pipeline de Middlewares . . . 87
3.5.2. Estructura Profesional de un Proyecto Express . . . 87
3.5.3. Renderizado en Servidor (SSR) con Plantillas Mustache . . . 89
3.5.4. Procesamiento de Formularios, Parámetros y el Patrón PRG . . . 89
3.5.5. Ejercicios Resueltos del Bloque 3.2.1: Tablón de Anuncios . . . 90
3.6. Gestión y Subida de Archivos con Multer (3.2.2) . . . 93
3.6.1. El Protocolo de Subida de Ficheros: `multipart/form-data` . . . 93
3.6.2. Almacenamiento Seguro: ¿Por qué NUNCA en la carpeta `public/`? . . . 93
3.6.3. Configuración de Multer y Servicio Seguro de Imágenes . . . 94
3.6.4. Ejercicio Resuelto 5: Tablón de Anuncios con Imágenes . . . 95
3.7. Introducción a las Bases de Datos (3.3.1) . . . 97
3.7.1. El Problema de la Persistencia en Memoria Volátil . . . 97
3.7.2. Bases de Datos Relacionales (SQL) vs NoSQL . . . 97
3.8. MongoDB: Modelo de Documentos y Shell (3.3.2) . . . 98
3.8.1. Modelo Documental, BSON y el Identificador `_id` . . . 98
3.8.2. Ediciones y Despliegue con Docker . . . 99
3.8.3. Operaciones CRUD en la Consola `mongosh` . . . 99
3.9. MongoDB con Node.js (3.3.3) . . . 100
3.9.1. El Driver Oficial `mongodb` y Conexión Asíncrona . . . 100
3.9.2. Operaciones de Inserción y Carga Masiva . . . 101
3.9.3. Consultas Avanzadas, Cursores, Paginación y Proyección . . . 101
3.9.4. Actualización y Eliminación de Documentos . . . 102
3.9.5. Gestión Avanzada de Arrays de Objetos Embebidos . . . 103
3.9.6. Ejercicio Resuelto 1: Consultas Complejas sobre Catálogo de Películas . . . 104
3.10. Aplicaciones Web con MongoDB y Express (3.3.4) . . . 105
3.10.1. Integración de la Capa de Persistencia en Express . . . 105
3.10.2. Ejercicio Resuelto Completo: Tablón de Anuncios Persistente . . . 106

CAPÍTULO 1

Fundamentos de la Web, Arquitecturas y Formatos de Datos

1.1 Historia de la Web y Principios de Red

Para entender el desarrollo web moderno como ingenieros de software, es imprescindible desmitificar su infraestructura subyacente. La Web no existe en el vacío: es una aplicación distribuida a escala planetaria construida sobre la red de redes, **Internet**.

1.1.1 Diferencia Fundamental: Internet vs. World Wide Web

Existe una confusión habitual en el público general que confunde ambos términos, pero en ingeniería informática representan capas conceptuales y temporales completamente diferenciadas:

> ▶ **Concepto Clave: Internet**
>
> **Internet** es la infraestructura física y lógica de telecomunicaciones descentralizada y global. Consiste en una gigantesca malla de redes interconectadas que utilizan la familia de protocolos **TCP/IP** para permitir que dispositivos de cualquier fabricante o arquitectura física intercambien paquetes binarios de datos. Nació a finales de los años 60 a partir del proyecto militar y académico estadounidense **ARPANET** (1969).

> ▶ **Concepto Clave: World Wide Web (WWW)**
>
> La **World Wide Web** (comúnmente abreviada como la *Web*) es un **sistema de información y distribución de documentos de hipertexto y multimedia interconectados** que funciona en la capa de aplicación sobre la infraestructura de Internet. Fue concebida más de dos décadas después de Internet, en 1989–1991.

En términos sencillos: Internet son las autopistas, raíles y túneles mundiales de comunicación; la Web es el servicio de trenes que transporta información hipermedia a través de esas vías. Otros servicios que viajan por las mismas autopistas de Internet de manera totalmente independiente a la Web son el correo electrónico (SMTP/IMAP), la transferencia de ficheros directa (FTP/SFTP), el acceso remoto seguro por terminal (SSH) o las llamadas de voz sobre IP (RTP/SIP).

1.1.2 Nacimiento de la Web: El CERN y los Primeros Estándares

En marzo de 1989, el físico británico **Tim Berners-Lee**, trabajando en el Centro Europeo para la Investigación Nuclear (**CERN**) en Suiza, redactó la propuesta titulada *«Information Management: A Proposal»*. El CERN albergaba a miles de científicos de todo el mundo que utilizaban ordenadores heterogéneos y sistemas operativos incompatibles entre sí, lo que provocaba una enorme pérdida de tiempo y documentación al intentar compartir resultados.

◦ Contexto Histórico & Evolución: La Invención de los Tres Pilares de la Web (1990–1992)

Junto al ingeniero belga **Robert Cailliau**, Tim Berners-Lee presentó formalmente en 1990 la arquitectura de la Web, basada en tres invenciones revolucionarias que siguen vigentes en la actualidad:

1. **URI/URL (Uniform Resource Identifier / Locator):** Un sistema unificado y universal de direccionamiento para localizar inequívocamente cualquier recurso de información en la red.
2. **HTTP (HyperText Transfer Protocol):** Un protocolo de red ligero y sin estado para que los clientes soliciten documentos y los servidores los entreguen.
3. **HTML (HyperText Markup Language):** Un lenguaje de marcado para estructurar textos y vincularlos entre sí mediante enlaces de hipertexto.

En una estación de trabajo *NeXTcube* desarrollada por Steve Jobs, Berners-Lee programó a finales de 1990 el primer servidor web de la historia (`httpd`) y el primer navegador-editor gráfico interactivo (*WorldWideWeb*). En agosto de 1991, el proyecto se publicó a través de Internet para toda la comunidad científica mundial.

Un hito trascendental para la historia de la humanidad ocurrió el 30 de abril de 1993: **el CERN liberó el código fuente de la Web y renunció a todos los derechos de propiedad intelectual, poniéndolo bajo dominio público**. Esta decisión desinteresada impidió la fragmentación propietaria de la tecnología y desató la mayor revolución informativa desde la imprenta de Gutenberg.

1.1.3 La Popularización y las Guerras de Navegadores

En 1993, Marc Andreessen y Eric Bina desarrollaron en el NCSA (Universidad de Illinois) el navegador **Mosaic**. Mosaic fue el primer navegador en ejecutarse en sistemas operativos masivos como Windows y Macintosh, e introdujo la capacidad histórica de incrustar imágenes directamente dentro del flujo de texto (etiqueta `<img>`), convirtiendo la Web en un medio visual y accesible para el público masivo.

Poco después, Andreessen fundó **Netscape Communications**, lanzando *Netscape Navigator* (1994), que dominó el 90 % del mercado en sus inicios. Viendo amenazada su hegemonía en el escritorio, Microsoft lanzó *Internet Explorer* en 1995 integrado en Windows 95, desatando la llamada **Primera Guerra de Navegadores**. Durante este conflicto comercial, ambas compañías introdujeron extensiones propietarias al lenguaje HTML sin coordinarse, provocando que las páginas web se rompieran si no eran visitadas con el navegador específico para el que fueron diseñadas (el famoso cartel de *«Optimizado para Internet Explorer a 800x600»*).

1.1.4 Los Organismos de Estandarización y el «Living Standard»

Para salvar a la Web del caos de la fragmentación y salvaguardar su interoperabilidad, Tim Berners-Lee fundó en octubre de 1994 el **World Wide Web Consortium (W3C)**. Desde entonces, la gobernanza de la Web se sustenta sobre varios organismos internacionales:

- **W3C (World Wide Web Consortium):** Organismo neutral que define las recomendaciones oficiales de tecnologías web fundamentales, como HTML (históricamente), XML, SVG y CSS.
- **IETF (Internet Engineering Task Force):** Comunidad técnica internacional abierta que estandariza los protocolos base de Internet mediante documentos **RFC** (*Request for Comments*), incluyendo IP, TCP, UDP y HTTP.
- **ECMA International:** Organismo encargado de la estandarización del lenguaje JavaScript a través de la especificación **ECMAScript** (comité técnico TC39).
- **WHATWG (Web Hypertext Application Technology Working Group):** Formado en 2004 por desarrolladores de Apple, Mozilla y Opera debido a la frustración con la lentitud del W3C y su intento de sustituir HTML por XHTML. El WHATWG impulsó **HTML5** y hoy mantiene la especificación como un *Living Standard* (estándar vivo en continua evolución, sin números de versión fijos). En 2019, el W3C y el WHATWG firmaron un acuerdo histórico reconociendo al WHATWG como la autoridad única del estándar HTML y del DOM.

1.1.5 La Trinidad del Front-End y su Cronología

La Web como plataforma interactiva se consolidó a través de tres tecnologías independientes que evolucionaron con una cadencia temporal precisa:

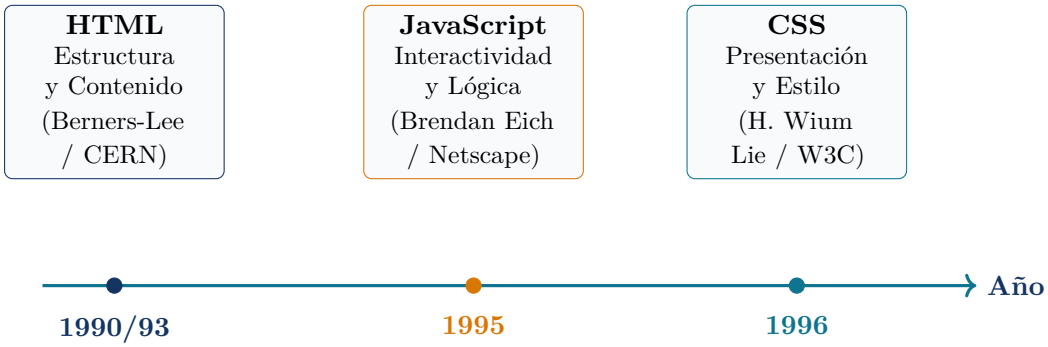


Figura 1.1: Cronología del nacimiento de los tres pilares del desarrollo Front-End.

- **HTML (1990/1993):** Nació para marcar documentos científicos con hiperenlaces. No disponía de capacidades estéticas avanzadas ni de lógica de programación.
- **JavaScript (1995):** Creado por Brendan Eich en Netscape en tan solo diez días (bajo el nombre clave *Mocha*, luego renombrado a *LiveScript* y finalmente a *JavaScript* por razones comerciales aprovechando la popularidad de Java). Permitía dotar a las páginas de validaciones locales y pequeñas animaciones sin consultar al servidor.
- **CSS (1996):** Creado por Håkon Wium Lie y Bert Bos bajo el auspicio del W3C para resolver un problema crítico: los documentos HTML se habían llenado de etiquetas visuales horribles (`<font>`, `<center>`, tablas anidadas) que mezclaban la presentación con el significado. CSS permitió la **separación radical entre contenido y diseño**.

1.2 Arquitectura de Red y el Modelo Cliente-Servidor

El funcionamiento de toda aplicación web se sustenta sobre el paradigma de arquitectura distribuida denominado **Cliente-Servidor**.

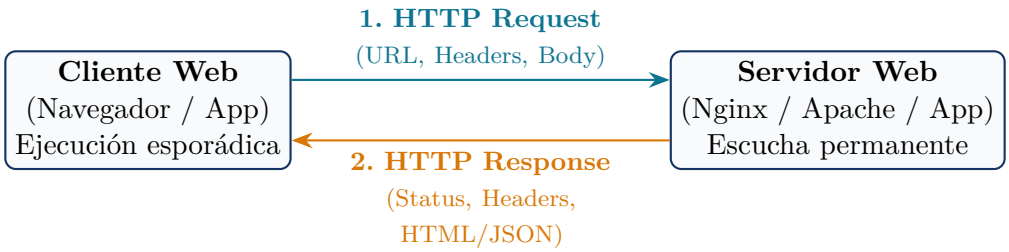


Figura 1.2: Ciclo fundamental de Petición/Respuesta en el modelo Cliente-Servidor.

1.2.1 Características del Cliente y del Servidor

- **El Dispositivo Servidor:** Es una máquina (física o virtual) conectada a la red de forma ininterrumpida que ejecuta un **software servidor**. Este software permanece en estado de escucha activa en un puerto de red esperando la llegada de peticiones externas.
- **El Dispositivo Cliente:** Es el dispositivo final (ordenador personal, teléfono móvil, tableta, consola de videojuegos, dispositivo IoT) que ejecuta un **software cliente** (el navegador web como Google Chrome, Mozilla Firefox, Apple Safari, o una aplicación móvil). El cliente solo inicia la comunicación cuando requiere un recurso del servidor, emitiendo una petición puntual y cerrando o suspendiendo el canal tras recibir la respuesta.

1.2.2 Direccionamiento IP, Nombres de Dominio y NAT

Para que dos máquinas puedan encontrarse en Internet, cada interfaz de red dispone de un identificador numérico único denominado **Dirección IP**:

- **IPv4:** Direcciones de 32 bits representadas en cuatro octetos decimales (ejemplo: `185.199.108.153`). El espacio teórico de aproximadamente $4{,}3 \times 10^9$ direcciones se encuentra agotado a nivel mundial.
- **IPv6:** Direcciones de 128 bits expresadas en notación hexadecimal (ejemplo: `2001:0db8:85a3::8a2e:037` con capacidad prácticamente inagotable ($3{,}4 \times 10^{38}$ direcciones).

Dado que los seres humanos memorizan palabras mejor que secuencias numéricas, la infraestructura de Internet incorpora el **DNS (Domain Name System)**: una base de datos jerárquica y distribuida a nivel mundial que traduce nombres de dominio legibles (como `www.google.es`) a su correspondiente dirección IP.

▲ ¡Atención / Ingeniería!: La Realidad de las Direcciones IP: NAT (Network Address Translation)

En una visión teórica simplificada, se afirma que «cada dispositivo conectado a Internet tiene una dirección IP pública propia». Sin embargo, en la práctica de la ingeniería de redes, esto casi nunca es cierto para los clientes debido al mecanismo **NAT** (*Network Address Translation*).
Los routers domésticos y corporativos asignan a los ordenadores y móviles direcciones IP privadas (rangos reservados como `192.168.x.x` o `10.x.x.x`) no enrutables en Internet. Cuando el cliente hace una petición web, el router traduce la IP privada a la única dirección IP pública compartida por toda la red local. Esto protege a los clientes de ataques entrantes directos, pero refuerza el principio de que los clientes inician las conexiones y los servidores son quienes deben tener IPs públicas accesibles o nombres de

dominio mapeados.

1.2.3 Puertos de Red y la Regla de Exclusión de Procesos

Un único servidor físico con una única dirección IP puede ejecutar simultáneamente múltiples servicios (por ejemplo, una página web, un servidor de correo y un servidor de bases de datos). Para dirigir el tráfico al proceso correcto en el sistema operativo, la capa de transporte utiliza el concepto de **Puerto de Red** (números enteros de 16 bits que abarcan de 0 a 65535):

- **Puertos Conocidos (Well-Known Ports, 0–1023):** Reservados para protocolos estándar de Internet. Requieren privilegios de administrador en sistemas tipo UNIX para vincularse a ellos.
- **Puertos Registrados (1024–49151):** Empleados por aplicaciones y servicios de usuario (por ejemplo, servidores de bases de datos o entornos de desarrollo).
- **Puertos Dinámicos / Efímeros (49152–65535):** Asignados temporalmente por el sistema operativo a los clientes para recibir la respuesta a sus peticiones salientes.

Tabla 1.1: Protocolos de aplicación habituales sobre TCP/IP y sus puertos por defecto.

Protocolo	Nombre Completo	Puerto	Propósito en la Ingeniería
HTTP	Hypertext Transfer Protocol	80 / TCP	Comunicación web sin cifrar por defecto.
HTTPS	HTTP Secure	443 / TCP	Comunicación web cifrada con TLS/SSL.
SSH	Secure Shell	22 / TCP	Administración remota segura de servidores por terminal.
FTP	File Transfer Protocol	20, 21 / TCP	Transferencia clásica de ficheros (insegura en texto plano).
SMTP	Simple Mail Transfer	25, 587 / TCP	Envío y enrutamiento de correos electrónicos.
DNS	Domain Name System	53 / UDP, TCP	Resolución de nombres de dominio a direcciones IP.
RTP	Real-time Transport Protocol	Dinámico / UDP	Transmisión en tiempo real de audio y vídeo en streaming.

▲ ¡Atención / Ingeniería!: Principio de Exclusión: El Error «Address Already in Use»

Un puerto de red en una misma interfaz e IP solo puede ser abierto y utilizado en modo escucha por **un único proceso servidor al mismo tiempo**. Si dos procesos intentan vincularse (`bind`) al mismo puerto e interfaz simultáneamente, el sistema operativo deniega la operación arrojando el clásico error de desarrollo:

```
Error: listen EADDRINUSE: address already in use :::8080
```

Si un servidor web decide no escuchar en el puerto por defecto (80 para HTTP o 443 para HTTPS), el cliente debe especificar explícitamente el puerto en la URL para poder comunicarse; por ejemplo: `http://localhost:8080/index.html`.

1.3 El Protocolo HTTP en Profundidad

El protocolo **HTTP** es el corazón de la Web. Se trata de un protocolo de la capa de aplicación cliente-servidor, sin estado (*stateless*), basado en mensajes de texto plano (en HTTP/1.x) o tramas binarias multiplexadas (en HTTP/2 y HTTP/3).

1.3.1 Estructura Canónica de una URL

La **URL** (*Uniform Resource Locator*) es la cadena de caracteres normalizada que identifica la ubicación exacta de un recurso en la red y el mecanismo para acceder a él. Su anatomía completa responde a la siguiente sintaxis:

```
https://usuario:password@www.ejemplo.es:8080/articulos/tecnologia?categoria=web&orden=desc#
```

Tabla 1.2: Desglose de los componentes de una URL.

Componente	Fragmento del ejemplo	Función y Consideraciones
Esquema (Protocolo)	`https`	Protocolo de comunicación a emplear (http, https, ftp).
Credenciales (Opcional)	`usuario:password@`	Autenticación básica HTTP (desaconsejada en URL por seguridad).
Nombre de Host / Dominio	`www.ejemplo.es`	Nombre FQDN o dirección IP del servidor destino.
Puerto (Opcional)	`:8080`	Puerto TCP. Si se omite, se asume el puerto estándar (80 u 443).
Ruta (Path)	`/articulos/tecnologia`	Jerarquía del recurso dentro del espacio lógico del servidor.
Query String (Consulta)	`?categoria=web...`	Pares clave-valor precedidos por `?` y separados por `&`.
Fragmento (Hash)	`#seccion3`	Identificador procesado solo en el cliente (no se envía al servidor).

1.3.2 Anatomía de los Mensajes HTTP: Peticiones y Respuestas

Toda interacción web consiste en un mensaje de petición emitido por el cliente y un mensaje de respuesta devuelto por el servidor.

Formato de una Petición HTTP (Request)

Consta de tres partes obligatorias:

1. **Línea de Petición (Request Line):** Método HTTP, la ruta del recurso solicitado y la versión del protocolo.
2. **Cabeceras de Petición (Request Headers):** Pares clave-valor que proporcionan metadatos sobre el cliente, formatos aceptados, cookies de sesión, etc.

3. **Línea en blanco obligatoria (CRLF):** Separa las cabeceras del cuerpo.
4. **Cuerpo del mensaje (Body - Opcional):** Presente habitualmente en métodos como `POST`, `PUT` o `PATCH` para enviar datos (formularios, JSON, ficheros).

```
POST /api/v1/login HTTP/1.1
Host: api.empresa.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
Content-Type: application/json
Content-Length: 52
Accept: application/json

{"username": "david_s", "password": "SuperSecretPassword123"}
```

Listing 1.1: Ejemplo real de una petición HTTP POST enviando credenciales.

Formato de una Respuesta HTTP (Response)

1. **Línea de Estado (Status Line):** Versión del protocolo, el código numérico de estado y un texto descriptivo (*reason phrase*).
2. **Cabeceras de Respuesta (Response Headers):** Metadatos del servidor, tipo MIME del contenido devuelto, longitud, directivas de caché, etc.
3. **Línea en blanco (CRLF).**
4. **Cuerpo del mensaje (Body):** Los datos solicitados (HTML, imagen, JSON, etc.).

```
HTTP/1.1 200 OK
Date: Sun, 08 Nov 2026 18:00:00 GMT
Server: nginx/1.24.0
Content-Type: application/json; charset=UTF-8
Content-Length: 78
Set-Cookie: sessionId=abc987xyz; Path=/; HttpOnly; Secure

{"status": "authenticated", "userId": 4402, "token": "eyJhbGciOiJIUzI1NiIs...
    "}
```

Listing 1.2: Ejemplo real de respuesta HTTP exitosa (200 OK) con payload JSON.

1.3.3 Métodos HTTP Semánticos e Idempotencia

En el estándar HTTP, los métodos representan las intenciones semánticas del cliente sobre el recurso. En ingeniería de software, dos propiedades formales rigen estos métodos:

▶ Concepto Clave: Método Seguro (Safe Method)

Un método es **seguro** si su invocación no provoca ninguna modificación en el estado del servidor (es de solo lectura). No tiene efectos colaterales observables.

▶ Concepto Clave: Método Idempotente (Idempotent Method)

Un método es **idempotente** si ejecutar la misma petición múltiples veces consecutivas produce exactamente el mismo resultado y deja al sistema en el mismo estado que ejecutarla una sola vez ($f(f(x)) = f(x)$). Si una petición de red se interrumpe por fallo de conexión, el cliente puede reenviarla con seguridad si el método es idempotente.

Tabla 1.3: Clasificación formal de los métodos HTTP principales.

Método	Propósito Semántico en la Web	¿Seguro?	¿Idempotente?
GET	Recuperar la representación de un recurso existente.	**Sí**	**Sí**
HEAD	Igual que GET, pero devuelve únicamente las cabeceras sin cuerpo.	**Sí**	**Sí**
POST	Procesar datos o crear un nuevo recurso subordinado.	No	No
PUT	Reemplazar por completo el recurso indicado (o crearlo si no existe).	No	**Sí**
PATCH	Modificar parcialmente un recurso existente.	No	No
DELETE	Eliminar permanentemente el recurso especificado.	No	**Sí**
OPTIONS	Consultar los métodos de comunicación permitidos por el servidor.	**Sí**	**Sí**

1.3.4 Familias de Códigos de Estado HTTP

Los códigos numéricos de respuesta de tres dígitos se agrupan en cinco familias según su primer dígito:

- **1xx – Informativos:** Petición recibida y en proceso (ej. `101 Switching Protocols` para WebSockets).
- **2xx – Respuestas Exitosas:**
  - `200 OK`: Petición procesada con éxito y recurso devuelto en el cuerpo.
  - `201 Created`: Recurso creado exitosamente (habitual tras un `POST`). La cabecera `Location` suele indicar su nueva URL.
  - `204 No Content`: Petición procesada con éxito pero la respuesta intencionadamente no incluye cuerpo (común en peticiones `DELETE`).
- **3xx – Redirecciones:** Se requiere una acción adicional del cliente:
  - `301 Moved Permanently`: El recurso cambió de URL para siempre (actualiza los índices de buscadores SEO).
  - `302 Found` / `307 Temporary Redirect`: Redirección temporal.
  - `304 Not Modified`: El recurso no ha cambiado desde la última visita en caché del cliente; no se envía cuerpo para ahorrar ancho de banda.
- **4xx – Errores de Cliente:** La petición contiene sintaxis errónea o no puede procesarse:
  - `400 Bad Request`: Petición mal formada o datos JSON inválidos.
  - `401 Unauthorized`: Autenticación requerida; faltan credenciales o el token ha expirado.
  - `403 Forbidden`: El servidor reconoce al usuario pero este no posee permisos suficientes para acceder al recurso.
  - `404 Not Found`: El recurso solicitado no existe en la ruta especificada.

- `405 Method Not Allowed`: El recurso existe pero no admite el método utilizado (ej. intentar un `POST` en una ruta de solo lectura).

- **5xx – Errores de Servidor:** El servidor falló al intentar atender una petición válida:
  - `500 Internal Server Error`: Excepción no controlada o fallo genérico en el código backend.
  - `502 Bad Gateway`: Un proxy inverso o balanceador recibió una respuesta inválida del servidor interno.
  - `503 Service Unavailable`: Servidor temporalmente sobrecargado o en mantenimiento.
  - `504 Gateway Timeout`: El servidor interno de aplicación tardó demasiado tiempo en responder al proxy inverso.

1.3.5 El Dilema del Estado: Cookies, Sesiones y Tokens JWT

HTTP fue concebido como un protocolo **sin estado (stateless)**: cada petición se procesa de forma completamente aislada e independiente, sin que el servidor recuerde ninguna interacción previa del mismo cliente. Sin embargo, aplicaciones como carritos de la compra o portales bancarios requieren recordar la identidad del usuario a lo largo de múltiples páginas.

Para solucionar esta limitación surgieron dos estrategias arquitectónicas esenciales:

1. **Sesiones Basadas en Cookies (Stateful):** Al autenticarse, el servidor genera un identificador aleatorio de sesión (*Session ID*) guardado en la memoria o base de datos del servidor (ej. Redis), y lo envía al cliente mediante la cabecera `Set-Cookie: session_id=xyz; HttpOnly; Secure`. El navegador almacena la cookie y la adjunta automáticamente en todas las peticiones posteriores. El servidor busca el ID y recupera los datos del usuario.
2. **Tokens Autocontenidos / JWT (Stateless Authentication):** El servidor firma criptográficamente un token (**JSON Web Token**) que contiene la identidad y los roles del usuario. El cliente almacena el token y lo envía explícitamente en la cabecera `Authorization: Bearer <token>`. El servidor no necesita almacenar sesiones en memoria: valida la firma criptográfica con su clave secreta y confía en la información contenida en el token.

1.4 Evolución del Servidor Web y Tecnologías Backend

1.4.1 Servidores Web Estáticos (1990–1993)

En los primeros años de la Web, los servidores (como **NCSA HTTPd**, el posterior **Apache HTTP Server** o **Microsoft IIS**) eran extremadamente simples: actuaban como despachadores de archivos físicos del disco duro. Cuando llegaba una petición para `/documentos/guia.html`, el servidor web buscaba en su directorio raíz (*DocumentRoot*) la ruta local `/var/www/html/documentos/guia.html` leía el fichero byte a byte y lo enviaba en el cuerpo de la respuesta HTTP con el `Content-Type: text/html`.

1.4.2 El Nacimiento de la Web Dinámica: CGI (Common Gateway Interface)

En 1993 nació la necesidad de presentar información variable (como el tiempo meteorológico, cotizaciones bursátiles o un simple contador de visitas). Para ello se creó la especificación **CGI (Common Gateway Interface)**: un estándar que permitía al servidor web delegar la

generación del HTML a un programa externo ejecutable escrito en lenguajes como C, Perl o scripts de Shell.

> **▲ ¡Atención / Ingeniería!: La Ineficiencia de CGI y el Desafío C10K**
>
> En el modelo CGI tradicional, **por cada petición HTTP entrante, el servidor web debía crear un proceso completamente nuevo en el sistema operativo** (*fork-and-exec*) para ejecutar el script, pasarle los parámetros por variables de entorno y leer su salida estándar (`stdout`).
> Bajo cargas de trabajo reales con cientos o miles de usuarios simultáneos, el coste computacional y de memoria de instanciar tantos procesos colapsaba los servidores (problema de escalabilidad). Esto motivó la creación de módulos persistentes integrados en el propio servidor web (como `mod_perl`, `mod_php` o servidores de aplicaciones Java con hilos de ejecución persistentes).

1.4.3 El Ecosistema Backend Moderno y las Bases de Datos

Hoy en día, las aplicaciones web no sirven páginas estáticas de disco, sino que ejecutan servidores de aplicación altamente optimizados que reciben la URL, ejecutan lógica de negocio, consultan bases de datos y generan las respuestas al vuelo:



Figura 1.3: Flujo de datos en una aplicación web dinámica con base de datos.

Entre los principales lenguajes y plataformas backend destacan:

- **Java / Kotlin:** Dominante en entornos bancarios y corporativos gracias a frameworks como **Spring Boot** y Jakarta EE. Gran robustez tipada, multihilo y rendimiento con la JVM.
- **JavaScript / TypeScript sobre Node.js o Deno:** Permite utilizar el mismo lenguaje tanto en cliente como en servidor. Se basa en el motor V8 de Google y una arquitectura asíncrona no bloqueante basada en un bucle de eventos (*Event Loop*). Frameworks: Express, NestJS, Fastify.
- **Python:** Muy popular por su sintaxis limpia y rapidez de desarrollo. Frameworks: **Django** (arquitectura completa con ORM y panel de administración), **Flask** y **FastAPI** (asíncrono de alto rendimiento para APIs).
- **PHP:** El lenguaje que impulsó la web dinámica clásica (usado históricamente por Facebook, Wikipedia y WordPress). Hoy en día cuenta con frameworks de nivel empresarial como **Laravel** y **Symfony**.
- **C# (.NET Core):** Plataforma moderna de alto rendimiento y código abierto desarrollada por Microsoft.
- **Go (Golang):** Desarrollado por Google, destaca por su velocidad de compilación nativa a binario único y su modelo de concurrencia ligera mediante *goroutines*.

En la capa de persistencia, los ingenieros seleccionan entre dos grandes familias:

- **Bases de Datos Relacionales (SQL):** PostgreSQL, MySQL, MariaDB, Oracle, Microsoft SQL Server. Estructuran la información en tablas con esquemas estrictos, relaciones mediante claves foráneas y garantías formales **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad).
- **Bases de Datos NoSQL:** Diseñadas para alta escalabilidad horizontal y esquemas flexibles:
  - *Documentales:* MongoDB, CouchDB (almacenan documentos JSON/BSON).
  - *Clave-Valor:* Redis, Memcached (cachés en memoria de latencia sub-milisegundo).
  - *Columnar / Familias de Columnas:* Apache Cassandra, ScyllaDB (análisis masivo distribuido).
  - *Grafos:* Neo4j (relaciones complejas como redes sociales o detección de fraudes).

1.5 Taxonomía y Arquitecturas de las Aplicaciones Web

Uno de los aportes teóricos y prácticos más importantes de la materia es la clasificación de los sitios y aplicaciones web en función de cómo combinan las capacidades del cliente y del servidor.

1.5.1 La Matriz Arquitectónica 2×2

Todo desarrollo web se sitúa en la intersección de dos ejes:

1. **El Servidor:** ¿Es **Estático** (solo despacha archivos preexistentes del disco/CDN) o **Dinámico** (ejecuta código que genera respuestas en tiempo real mediante lógica y bases de datos)?
2. **El Cliente:** ¿Es **Estático** (solo renderiza HTML y CSS, o navega mediante recargas completas sin JS) o **Dinámico** (ejecuta JavaScript en el navegador del usuario para manipular la interfaz o solicitar datos en segundo plano)?

De esta matriz surgen las **seis arquitecturas web fundamentales**:

1.5.2 Análisis Detallado de las 6 Arquitecturas

1. Página Web Estática

- **Mecanismo Técnico:** El cliente solicita una URL por HTTP. El servidor busca el archivo HTML/CSS en disco y lo devuelve intacto. El navegador visualiza la página sin ejecutar código JavaScript. Al pulsar cualquier enlace, el navegador descarga un nuevo archivo HTML y repinta la pantalla completa desde cero.
- **Tecnologías:** HTML y CSS puro (o maquetado mediante frameworks CSS como Bootstrap).
- **Ventajas:** Consumo de recursos ínfimo en servidor, rendimiento extremo de entrega a través de redes CDN (*Content Delivery Networks*), coste económico prácticamente nulo (servicios gratuitos como GitHub Pages, Netlify o Vercel) y superficie de ataque de seguridad casi inexistente (no hay inyección SQL ni código vulnerable en servidor).
- **Desventajas:** Contenido idéntico para todos los usuarios. No permite gestión de sesiones, usuarios ni persistencia de datos. Su actualización requiere editar ficheros y desplegar por Git o SSH.

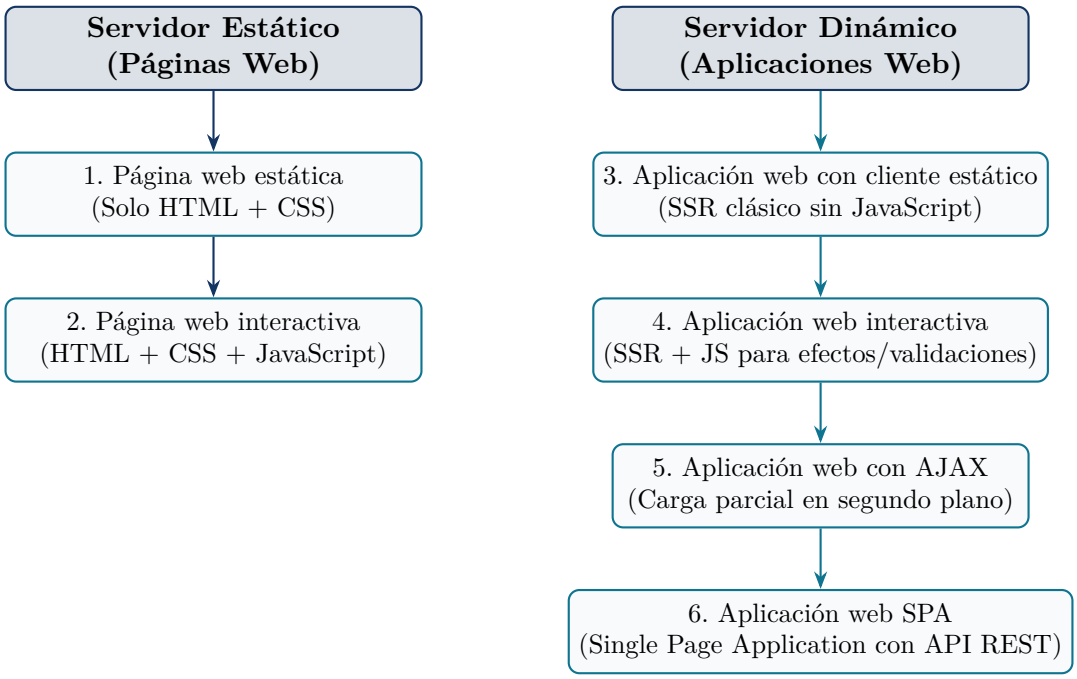


Figura 1.4: Taxonomía de las seis arquitecturas web del programa de la asignatura.

2. Página Web Interactiva

- **Mecanismo Técnico:** El contenido continúa alojado estáticamente en el disco duro del servidor web, pero los archivos HTML enlazan scripts de JavaScript que se ejecutan en el navegador del cliente.
- **Objetivo del JavaScript:** Proporcionar interactividad visual y utilidades en el cliente: animaciones complejas, galerías de imágenes (*carousels*), menús desplegables, acordeones que muestran u ocultan texto para facilitar la lectura de documentos extensos, o calculadoras que operan puramente en local.
- **Ventajas e Inconvenientes:** Mantiene la simplicidad y el coste cero de alojamiento en servidor estático, pero enriquece notablemente la experiencia de usuario. Su límite radica en que no puede almacenar datos persistentes en un servidor central.

3. Aplicación Web con Cliente Estático (SSR Clásico Puro)

- **Mecanismo Técnico:** El servidor es dinámico (ejecuta PHP, Java, Python, Node.js) y se conecta a una base de datos para generar el documento HTML en tiempo real ante cada petición (*Server-Side Rendering*). Sin embargo, **el cliente no ejecuta ningún código JavaScript**. Las interacciones del usuario se limitan a hiperenlaces ordinarios y formularios que emiten peticiones `GET` o `POST`. Cada clic provoca la recarga absoluta de la página en el navegador.
- **Ejemplo Emblemático:** La versión básica HTML de **Gmail** (mantenida por Google para dispositivos con conexiones ultra-lentas hasta su descontinuación a principios de 2024).
- **Ventajas:** Funciona de forma garantizada en navegadores antiguos o con JavaScript deshabilitado por seguridad; accesibilidad absoluta y consumo mínimo de CPU y batería en el dispositivo cliente.

- **Desventajas:** Pésima experiencia de usuario (navegación tosca con parpadeo blanco constante en pantalla al recargar la página completa, incluso cuando el 95 % del contenido no ha variado).

4. Aplicación Web Interactiva (SSR con JavaScript Progresivo)

- **Mecanismo Técnico:** Es la arquitectura predominante de la Web tradicional empresarial. El servidor genera las páginas dinámicamente mediante plantillas en servidor (JSP, Thymeleaf, Blade, Django Templates), pero la página resultante incluye JavaScript para:
  - Validar formularios en el cliente antes de enviarlos (evitando esperar al servidor para avisar de contraseñas cortas o formatos de email erróneos).
  - Añadir efectos visuales avanzados, modales emergentes y componentes dinámicos.
- **Ventajas:** Excelente equilibrio entre SEO sólido (los motores de búsqueda indexan HTML completo devuelto por el servidor) y buena experiencia de usuario interactiva.

5. Aplicación Web con AJAX (Carga Asíncrona Parcial)

- **¿Qué es AJAX?:** Acrónimo de **Asynchronous JavaScript And XML** (término acuñado por Jesse James Garrett en 2005). Es una técnica de ingeniería que permite al código JavaScript del navegador realizar peticiones HTTP al servidor web en segundo plano, sin interferir con la página visible ni recargarla.
- **Mecanismo Técnico:** Cuando llega la respuesta del servidor en segundo plano, el código JavaScript captura los datos y manipula quirúrgicamente el DOM (*Document Object Model*) para actualizar únicamente la sección de la pantalla que ha cambiado.
- **¿Qué devuelve el servidor ante una petición AJAX?:**
  - *Fragmentos de HTML:* El servidor devuelve solo el bloque HTML que cambió (ejemplo: tres nuevos comentarios) y JavaScript los inserta directamente mediante `innerHTML` o `append()`.
  - *Información Estructurada (JSON):* El servidor devuelve datos puros en formato JSON a través de una API REST. JavaScript interpreta el objeto y construye dinámicamente los elementos visuales.
- **Casos de Uso Revolucionarios:**
  - **Autocompletado / Google Suggest:** Al teclear una letra, se pide en segundo plano una lista de sugerencias sin recargar la página.
  - **Scroll Infinito:** En plataformas como Twitter/X o Google Imágenes, al aproximarse el usuario al final de la pantalla, JavaScript pide los siguientes 20 elementos y los concatena fluidamente, eliminando los botones de paginación tradicionales.
  - **Validación de Disponibilidad:** Comprobación en vivo de si un nombre de usuario ya está registrado en el momento en que se escribe en el campo del formulario.

6. Aplicación Web SPA (Single Page Application)

- **Filosofía Arquitectónica:** La técnica AJAX llevada a su máxima expresión. El servidor web entrega al inicio un único archivo `index.html` casi vacío junto a una hoja de estilos y un paquete pesado de JavaScript (*bundle*). A partir de ese instante, **la página nunca más se vuelve a recargar**.

- **Client-Side Rendering (CSR):** Cuando el usuario hace clic para cambiar de sección (por ejemplo, de "Perfil.ª Çonfiguración"), el navegador no pide un nuevo HTML: el enrutador de JavaScript (*Client-Side Router*) intercepta la URL mediante la *History API* del navegador, desmonta los componentes visuales antiguos de la pantalla, solicita datos en formato JSON a una **API REST** del servidor y monta los nuevos componentes en milisegundos.
- **Ejemplos Reales:** Google Maps, WhatsApp Web, Gmail moderno, GitHub, Spotify Web.
- **Frameworks Líderes:** React (Meta), Angular (Google), Vue.js (Evan You) y Svelte.
- **Ventajas:** Experiencia de usuario ultra-fluida idéntica a una aplicación nativa de escritorio; total desacoplamiento entre el equipo de Front-End (interfaz) y Back-End (lógica de negocio y API REST reusable para aplicaciones móviles).
- **Desventajas e Ingeniería Avanzada:** Mayor complejidad de desarrollo, tiempo de carga inicial lento (*Initial Page Load* debido al tamaño del bundle JS) y dificultades de posicionamiento en motores de búsqueda (SEO), ya que los rastreadores web tradicionales leen el HTML inicial vacío. Para solucionar esto, la industria moderna recurre a arquitecturas híbridas con **SSR e Hidratación** (como Next.js para React o Nuxt para Vue).

Tabla 1.4: Matriz comparativa formal de las seis arquitecturas web.

Arquitectura	Servidor	Cliente (JS)	Experiencia (UX)	Caso de Uso Recomendado
1. Web Estática	Estático (Disco)	No ejecuta JS	Básica, recarga completa.	Documentación técnica, blogs personales, landing pages.
2. Web Interactiva	Estático (Disco)	Efectos visuales y utilidades locales	Fluida en cliente, sin persistencia.	Páginas promocionales con animaciones, catálogos estáticos.
3. App Cliente Estático	Dinámico (BD)	No ejecuta JS	Tosca, recarga continua de página.	Sistemas con conectividad crítica, entornos sin soporte de JS.
4. App Interactiva	Dinámico (SSR)	Validaciones y efectos progresivos	Buena y robusta, SEO excelente.	Portales de comercio electrónico clásicos, webs corporativas.
5. App con AJAX	Dinámico (SSR + API)	Peticiones en segundo plano	Muy fluida, actualización parcial.	Redes sociales con feeds continuos, paneles con datos en vivo.
6. App SPA	Dinámico (API REST)	Control total del renderizado (CSR)	Óptima (sensación de app nativa).	Paneles de administración (dashboards), suites SaaS, webapps complejas.

1.6 Formatos de Intercambio de Datos: XML y JSON

En el modelo cliente-servidor desacoplado, los sistemas informáticos desarrollados en distintos lenguajes (un frontend en JavaScript y un backend en Java o Python) necesitan intercambiar información estructurada de manera universal, estandarizada e independiente de la plataforma. Los dos estándares dominantes en la historia de la Web son **XML** y **JSON**.

1.6.1 XML (eXtensible Markup Language)

Estandarizado por el W3C en 1998, XML es un metalenguaje de marcado derivado de SGML pensado para almacenar y transportar datos estructurados legibles tanto por humanos como por máquinas.

Sintaxis y Reglas de un Documento Bien Formado

Un documento XML se organiza en una estructura de árbol estricta compuesta por **elementos** y **atributos**:

```
<?xml version="1.0" encoding="UTF-8"?>
<character id="01">
    <name>Galadriel</name>
    <race>Elve</race>
    <ring>Nenya</ring>
</character>
```

Listing 1.3: Estructura canónica de un archivo XML (tolkien.xml).

Para que un archivo XML se considere **bien formado** (*Well-Formed*), debe cumplir rigurosamente las siguientes restricciones sintácticas:

1. **Prólogo / Declaración XML obligatoria:** Especifica la versión y la codificación (habitualmente UTF-8): `<?xml version="1.0.ᵉⁿcoding=ÜTF-8¿>`.
2. **Elemento Raíz Único:** Debe existir un único elemento padre que envuelva a todos los demás elementos del documento.
3. **Cierre Obligatorio de Etiquetas:** Toda etiqueta de apertura (`<etiqueta>`) debe tener su correspondiente etiqueta de cierre (`</etiqueta>`), o ser autoclusiva (`<etiqueta/>`).
4. **Distinción de Mayúsculas y Minúsculas (*Case Sensitivity*):** `<Nombre>` y `<nombre>` son dos elementos completamente diferentes.
5. **Anidamiento Estricto:** Las etiquetas no pueden solaparse. Lo que se abre primero dentro de un elemento, debe cerrarse antes de cerrar el contenedor (`<a><b></b></a>` es correcto; `<a><b></a></b>` es un error fatal de análisis sintáctico).
6. **Reglas de Nomenclatura:**
   - Los nombres deben comenzar por una letra o por un guión bajo (`_`).
   - No pueden comenzar por números, signos de puntuación ni por la cadena reservada `xml` (en cualquier combinación de mayúsculas).
   - No están permitidos espacios en blanco ni retornos de carro antes o después del nombre de la etiqueta.
   - Se recomienda evitar guiones medios (`-`) y puntos (`.`) para no generar conflictos con analizadores sintácticos.
7. **Valores de Atributos Siempre Entrecomillados:** En XML, todo valor de atributo debe figurar obligatoriamente entre comillas dobles o simples (`id="01"`). Un elemento no puede tener atributos duplicados con el mismo nombre.

El Modelo Arbóreo del DOM en XML

Al procesar un archivo XML, el analizador construye en memoria el **DOM (Document Object Model)**: una jerarquía de nodos en la que el elemento raíz es el tronco y los elementos hijos, atributos y textos son ramas y hojas:

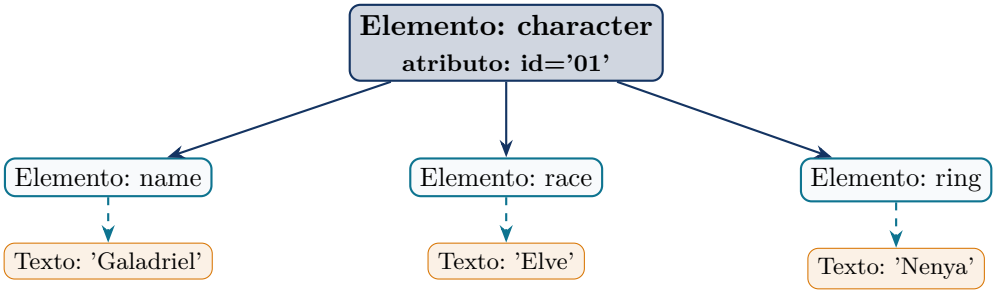


Figura 1.5: Representación en árbol DOM del documento XML.

Validación en XML: DTD y XML Schema (XSD)

Un documento bien formado puede no ser semánticamente correcto según las reglas de negocio de una empresa. Para verificar que un XML cumple con la estructura esperada se utilizan mecanismos de validación:

- **DTD (Document Type Definition):** El mecanismo clásico de SGML. Define qué elementos están permitidos y cuántas veces pueden aparecer. Sus limitaciones son notables: no utiliza sintaxis XML y carece de tipado de datos (trata casi todo como cadenas de texto).
- **XML Schema (XSD - XML Schema Definition):** Estandarizado por el W3C. Utiliza sintaxis nativa XML y proporciona un sistema de tipado fuerte y riguroso (enteros, booleanos, fechas, expresiones regulares, rangos numéricos), convirtiéndose en el estándar de oro en banca, telecomunicaciones y sistemas empresariales.

1.6.2 JSON (JavaScript Object Notation)

Estandarizado por primera vez por Douglas Crockford a comienzos de los 2000 y formalizado bajo la norma internacional **ECMA-404** y el **RFC 8259**, JSON es un formato ultraligero de texto plano basado en un subconjunto de la notación literal de objetos de JavaScript.

Tipos de Datos y Estructura en JSON

Un documento JSON válido solo puede contener los siguientes tipos de datos fundamentales:

1. **String (Cadena de texto):** Secuencia de caracteres Unicode delimitada **obligatoriamente por comillas dobles** (`"texto"`). Las comillas simples están terminantemente prohibidas en la especificación JSON.
2. **Number (Número):** Entero o de coma flotante en base 10 (ej. `42`, `-12.5`, `1.5e3`). No distingue entre enteros y flotantes.
3. **Boolean (Booleano):** Valores literales en minúscula: `true` o `false`.
4. **Null:** El valor nulo literal: `null`.
5. **Array (Arreglo / Lista ordenada):** Colección indexada de cero o más valores delimitada por corchetes: `[valor1, valor2, ...]`.

6. **Object (Objeto / Diccionario asociativo):** Colección no ordenada de pares clave-valor delimitada por llaves: `{clave": valor}`. Las claves deben ser cadenas de texto entre comillas dobles.

```
{
  "city": "Móstoles",
  "population": 214000,
  "postal": [28931, 28933],
  "active": true,
  "points_of_interest": [
    {
      "name": "Plaza del Pradillo",
      "category": "Plaza pública",
      "rating": 4.6
    },
    {
      "name": "Parque Natural El Soto",
      "category": "Parque",
      "rating": 4.8
    }
  ],
  "mayor": null
}
```

Listing 1.4: Ejemplo completo de objeto JSON estructurado con arrays y tipos mixtos.

▲ ¡Atención / Ingeniería!: Restricciones Críticas en la Sintaxis de JSON

1. **Ausencia de Comentarios:** El estándar oficial JSON prohíbe taxativamente los comentarios (`//...` o `/*...*/`). Douglas Crockford eliminó deliberadamente los comentarios para impedir que la gente utilizara directivas de compilación propietarias que destruyeran la interoperabilidad entre analizadores. Para archivos de configuración que requieran comentarios se utiliza una variante denominada **JSONC** (soportada en VS Code).
2. **Prohibición de la Coma Final (*Trailing Comma*):** Poner una coma tras el último atributo de un objeto o array (ejemplo: `{.ª": 1, "b": 2,}`) provoca un error de sintaxis fatal en cualquier analizador JSON estricto.
3. **Codificación:** La codificación obligatoria por defecto para transmisión en red es **UTF-8**.

Validación en JSON: JSON Schema

Al igual que XSD en XML, en el ecosistema JSON existe la especificación **JSON Schema**. Permite declarar formalmente la estructura esperada de una API REST: tipos de datos obligatorios, expresiones regulares para emails o códigos postales, y validación automática de payloads en frameworks como Express o FastAPI.

1.6.3 Comparativa Técnica Exhaustiva: XML vs. JSON

1.7 Ejercicios Prácticos Resueltos del Tema 1

A continuación se resuelven paso a paso los ejercicios planteados en el temario oficial del curso.

★ Ejercicio Resuelto: Ejercicio 1: Modelado de un Currículum Vitae en XML

Tabla 1.5: Comparativa técnica de ingeniería entre XML y JSON.

Criterio de Evaluación	XML (Extensible Markup Language)	JSON (JavaScript Object Notation)
Verbosidad y Tamaño	Muy alta. Requiere etiquetas de apertura y cierre explícitas repetidas (`<nombre>...</nombre>`), consumiendo más ancho de banda.	Muy baja. Sintaxis compacta y directa basada en llaves y corchetes.
Coste de Análisis (Parsing)	Elevado. Requiere analizadores DOM complejos que consumen abundante memoria y ciclos de CPU.	Muy bajo y ultra-rápido. En JavaScript la conversión es instantánea y nativa (`JSON.parse()`).
Tipado Nativo	No nativo. En el documento todo es texto; el tipado requiere esquemas XSD externos.	Nativo (distingue números, booleanos, cadenas, listas, objetos y null de forma inherente).
Soporte de Metadatos	Excelente mediante el uso de atributos en las etiquetas (`<fecha format=ÏSO»`).	Debe modelarse explícitamente mediante subobjetos.
Seguridad y Vulnerabilidades	Susceptible a ataques complejos como **XXE** (*XML External Entity*) y bombas XML (*Billion Laughs*).	Más seguro por diseño, aunque vulnerable a inyecciones si se procesa erróneamente con `eval()`.
Dominio Actual	Documentos ofimáticos (DOCX, XLSX), SVG, feeds RSS, integración bancaria/SOAP.	**El estándar absoluto** de las APIs REST modernas, microservicios y la Web actual.

Enunciado Oficial: *Crea un archivo XML que represente la información de un Currículum Vitae (CV) con las siguientes secciones: Información Personal, Experiencia Laboral, Educación, Habilidades e Idiomas con sus respectivos niveles.*
Solución de Ingeniería: Se diseña un documento XML bien formado, respetando las buenas prácticas de nomenclatura, utilizando atributos para metadatos identificativos e incorporando la cabecera correspondiente.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<curriculum_vitae id="CV-2026-0042" lang="es">
    <!-- Seccion 1: Informacion Personal -->
    <informacion_personal>
        <nombre_completo>David Sebastian Sticea Covaciu</
    nombre_completo>
        <titulo_profesional>Ingeniero de Software</titulo_profesional>
        <contacto>
            <email>david.sticea@alumnos.urjc.es</email>
            <telefono prefijo="+34">600123456</telefono>
            <direccion>
                <calle>Calle Tulipán s/n</calle>
                <ciudad>Móstoles</ciudad>
                <codigo_postal>28933</codigo_postal>
                <pais>España</pais>
            </direccion>
        </contacto>
    </informacion_personal>

    <!-- Seccion 2: Experiencia Laboral -->
    <experiencia_laboral>
        <puesto id="exp_01">
            <cargo>Desarrollador Web Full-Stack</cargo>
            <empresa>Tech Solutions S.L.</empresa>
            <periodo>
```

```
                <fecha_inicio>2024-02-01</fecha_inicio>
                <fecha_fin>2025-08-31</fecha_fin>
            </periodo>
            <responsabilidades>
                <item>Desarrollo de APIs RESTful utilizando Node.js y
    Express.</item>
                <item>Maquetación de interfaces responsivas con HTML5 y
     Bootstrap 5.</item>
            </responsabilidades>
        </puesto>
    </experiencia_laboral>

    <!-- Seccion 3: Educacion -->
    <educacion>
        <estudio id="edu_01">
            <titulacion>Grado en Ingeniería del Software</titulacion>
            <institucion>Universidad Rey Juan Carlos</institucion>
            <anio_inicio>2023</anio_inicio>
            <anio_graduacion>2027</anio_graduacion>
        </estudio>
    </educacion>

    <!-- Seccion 4: Habilidades -->
    <habilidades>
        <habilidad categoria="Frontend">HTML5 / CSS3 / JavaScript</
    habilidad>
        <habilidad categoria="Frontend">Bootstrap 5</habilidad>
        <habilidad categoria="Backend">Java / Spring Boot</habilidad>
        <habilidad categoria="Bases de Datos">PostgreSQL / MongoDB</
    habilidad>
        <habilidad categoria="Herramientas">Git / Docker / Linux</
    habilidad>
    </habilidades>

    <!-- Seccion 5: Idiomas -->
    <idiomas>
        <idioma codigo="es" nivel="Nativo">Español</idioma>
        <idioma codigo="en" nivel="C1">Inglés (Certificado Cambridge)</
    idioma>
        <idioma codigo="ro" nivel="Bilingüe">Rumano</idioma>
    </idiomas>
</curriculum_vitae>
```

Listing 1.5: currículum_vitae.xml: Solución completa al Ejercicio 1.

★ Ejercicio Resuelto: Ejercicio 2: Traducción Canónica del CV a JSON

Enunciado Oficial: *Traduce el Ejercicio 1 de un Currículum Vitae con XML a formato JSON.*

Solución de Ingeniería: En la traducción a JSON aprovechamos las estructuras nativas que no existían de forma limpia en XML: los arrays de objetos indexados (`[]`), valores booleanos y números enteros reales sin necesidad de comillas. Nótese cómo la verbosidad disminuye drásticamente y la legibilidad aumenta.

```json
{
  "curriculum_vitae": {
    "id": "CV-2026-0042",
    "idioma_documento": "es",
    "informacion_personal": {
      "nombre_completo": "David Sebastián Sticea Covaciu",
      "titulo_profesional": "Ingeniero de Software",
      "contacto": {
        "email": "david.sticea@alumnos.urjc.es",
        "telefono": {
          "prefijo": "+34",
          "numero": 600123456
        },
        "direccion": {
          "calle": "Calle Tulipán s/n",
          "ciudad": "Móstoles",
          "codigo_postal": 28933,
          "pais": "España"
        }
      }
    },
    "experiencia_laboral": [
      {
        "id": "exp_01",
        "cargo": "Desarrollador Web Full-Stack",
        "empresa": "Tech Solutions S.L.",
        "periodo": {
          "fecha_inicio": "2024-02-01",
          "fecha_fin": "2025-08-31"
        },
        "responsabilidades": [
          "Desarrollo de APIs RESTful utilizando Node.js y Express.",
          "Maquetación de interfaces responsivas con HTML5 y Bootstrap 5."
        ]
      }
    ],
    "educacion": [
      {
        "id": "edu_01",
        "titulacion": "Grado en Ingeniería del Software",
        "institucion": "Universidad Rey Juan Carlos",
        "anio_inicio": 2023,
        "anio_graduacion": 2027
      }
    ],
    "habilidades": [
      { "nombre": "HTML5 / CSS3 / JavaScript", "categoria": "Frontend" },
      { "nombre": "Bootstrap 5", "categoria": "Frontend" },
      { "nombre": "Java / Spring Boot", "categoria": "Backend" },
```

```
      { "nombre": "PostgreSQL / MongoDB", "categoria": "Bases de Datos" },
      { "nombre": "Git / Docker / Linux", "categoria": "Herramientas" }
    ],
    "idiomas": [
      { "idioma": "Español", "codigo": "es", "nivel": "Nativo" },
      { "idioma": "Inglés", "codigo": "en", "nivel": "C1", "certificado": "Cambridge" },
      { "idioma": "Rumano", "codigo": "ro", "nivel": "Bilingüe" }
    ]
  }
}
```

Listing 1.6: curriculum_vitae.json: Solución canónica al Ejercicio 2.

CAPÍTULO 2

Tecnologías de Cliente Web: HTML5, CSS3 y Bootstrap 5

En este capítulo nos adentramos de lleno en el desarrollo del lado del cliente (*Front-End*). Una interfaz web profesional moderna es el resultado de la integración armónica de tres capas de abstracción bien diferenciadas:

1. **Estructura y Semántica:** Implementada mediante **HTML5**.
2. **Presentación, Diseño y Maquetación Espacial:** Implementada con **CSS3** (utilizando modelos modernos como Flexbox y CSS Grid).
3. **Frameworks y Aceleración de Prototipado:** Implementado mediante **Bootstrap 5**, que estandariza componentes UI y un sistema de cuadrícula responsiva de 12 columnas.

—

2.1 HTML5: Estructura, Semántica y Formularios

2.1.1 Definición y Rol en el Ecosistema Web

▶ **Concepto Clave: HTML (HyperText Markup Language)**

HTML es el lenguaje de marcado estándar universal utilizado para estructurar y presentar contenido en la World Wide Web. Los navegadores web no son más que motores de renderizado especializados cuyo cometido principal es interpretar el código HTML, descargar los recursos asociados (hojas de estilo, imágenes, fuentes, scripts) y representarlos visualmente en la pantalla del usuario.

El lenguaje no es un lenguaje de programación: carece de estructuras de control de flujo (bucles, condicionales) o evaluación de expresiones aritméticas. Su función es puramente **estructural y semántica**: indica qué partes del texto son títulos, cuáles son párrafos, dónde hay enlaces, imágenes o tablas, y qué significado tienen dichos bloques dentro de la jerarquía de la información.

2.1.2 Diferencias Cruciales: HTML vs. XML vs. XHTML

Es vital que un ingeniero distinga con claridad estos tres estándares afines:

- **HTML:** Diseñado para la presentación y estructuración de información humana en navegadores web. Tolera cierta flexibilidad sintáctica en elementos vacíos y atributos.
- **XML:** Diseñado para almacenar e intercambiar datos estructurados entre aplicaciones de forma estricta. Todo elemento debe cerrarse sin excepción y la sintaxis no admite ambigüedades.
- **XHTML (eXtensible HTML):** Fue un intento del W3C a principios de los años 2000 por reformular HTML bajo las reglas rígidas de XML 1.0 (obligando a cerrar todas las etiquetas con sintaxis como `<br />` y escribir todo en minúsculas estrictas). Si un documento XHTML contenía un mínimo error de sintaxis, los navegadores debían mostrar una pantalla de error fatal en lugar de renderizar la página. Este enfoque fracasó en la práctica real de la industria.
  En respuesta, el consorcio **WHATWG** impulsó **HTML5**, que recuperó un analizador pragmático y tolerante a fallos, adoptando una filosofía orientada a la web viva (*Living Standard*) que se utiliza unánimemente en la actualidad.

2.1.3 Entorno de Trabajo, Editores y Documentación Oficial

Para editar documentos HTML, los ingenieros utilizan herramientas modernas:

- **Editores de Código e IDEs: Visual Studio Code** (el editor libre estándar en la industria, complementado con extensiones como *Live Server* u *Open in Default Browser*) y entornos especializados como **WebStorm** de JetBrains.
- **Herramientas de Desarrollador (DevTools):** Integradas en navegadores como Google Chrome o Mozilla Firefox (accesibles mediante la tecla `F12`). Permiten inspeccionar el árbol DOM en vivo, editar estilos CSS en tiempo real, auditar la red (*Network tab*) y evaluar la accesibilidad.
- **Playgrounds Online:** Entornos en la nube para experimentar y compartir prototipos sin instalar software local: `codepen.io`, `jsfiddle.net` y `jsbin.com`.
- **Documentación Canónica:** La documentación de referencia absoluta y de máxima calidad mantenida por la comunidad es **MDN Web Docs** (`https://developer.mozilla.org/es/docs/Web/HTML`), complementada por agregadores rápidos como `https://devdocs.io/`.

2.1.4 Anatomía de un Documento HTML5 Estándar

Todo documento HTML5 bien estructurado debe cumplir con la siguiente plantilla canónica:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Manual de Fundamentos de la Web para
    Ingenieros">
    <title>Título Profesional de la Página</title>
    <link rel="stylesheet" href="css/estilos.css">
    <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
    <h1>Bienvenido a la Web Semántica</h1>
    <p>Este es el cuerpo principal del documento.</p>
</body>
</html>
```

Listing 2.1: Plantilla canónica de un documento HTML5 profesional.

Desglose de los Componentes Fundamentales

1. **`<!DOCTYPE html>`:** En versiones antiguas de HTML (HTML 4.01), el DOCTYPE requería una larga e incomprensible referencia a un DTD. En HTML5 se simplificó al máximo: solo indica al navegador que interprete el documento siguiendo los estándares web modernos, evitando activar el **Quirks Mode** (modo de compatibilidad hacia atrás que emula los fallos de renderizado de navegadores de los años 90).
2. **`<html lang=.es»`:** La etiqueta raíz que envuelve todo el documento. El atributo `lang` es de suma importancia: ayuda a los lectores de pantalla a pronunciar el texto con el acento e idioma correctos y a los motores de búsqueda a indexar el contenido en la región idiomática adecuada.
3. **`<head>`:** Contenedor de **metadatos**. La información contenida en el `<head>` no se dibuja directamente en el área visible de la página web, pero es fundamental para su funcionamiento:
   - **`<meta charset=ÜTF-8»`:** Especifica la tabla de codificación de caracteres. UTF-8 incluye prácticamente todos los caracteres de todos los alfabetos humanos (incluyendo tildes, la letra eñe y emojis), evitando que aparezcan caracteres corruptos (*mojibake*). Debe colocarse en las primeras líneas del `<head>`.
   - **`<meta name="viewportçontent="width=device-width, initial-scale=1.0»`:** La directiva indispensable del diseño web responsivo (*Responsive Web Design*). Indica al navegador móvil que fije el ancho del área de visualización (*viewport*) al ancho físico real de la pantalla del dispositivo en lugar de asumir por defecto un escritorio virtual de 980 píxeles.
   - **`<title>`:** El título que aparece en la pestaña del navegador y en los resultados de búsqueda de Google.
   - **`<link rel="stylesheet"href="...»`:** Vincula hojas de estilo CSS externas.
4. **`<body>`:** Contiene todo el contenido visual y multimedia accesible por el usuario final (textos, imágenes, vídeos, formularios).

2.1.5 Elementos de Texto y Jerarquía Semántica

El texto en HTML se organiza de forma piramidal y estructurada:

- **Encabezados (`<h1>` a `<h6>`):** Indican el nivel jerárquico de importancia de las secciones.

> **▲ ¡Atención / Ingeniería!: Regla de Oro en Ingeniería: Jerarquía de Títulos**
>
> En una página web profesional debe existir **un único elemento `<h1>`** que represente el tema principal del documento. Los títulos subsiguientes deben anidarse respetando los niveles de forma continua (`h1` → `h2` → `h3`) sin saltarse escalones jerárquicos por razones puramente visuales (el tamaño visual se modifica con CSS, nunca cambiando la etiqueta semántica).

- **Párrafos (`<p>`):** Bloques de texto continuo. Los navegadores aplican por defecto márgenes verticales para separarlos.
- **Saltos de Línea forzados (`<br>`) y Separadores (`<hr>`):** Elementos autoclusivos. `<br>` genera un salto de línea dentro de un párrafo sin iniciar uno nuevo; `<hr>` representa una ruptura temática (*Horizontal Rule*).
- **Formato Inline Semántico vs. Visual:**
  - **`<strong>`:** Confiere **gran importancia o urgencia semántica** (los lectores de pantalla lo leen con énfasis marcado; visualmente se renderiza en negrita). Se prefiere a la etiqueta clásica obsoleta `<b>` (que solo oscurecía el texto sin transmitir significado).
  - **`<em>`:** Confiere **énfasis de entonación o estrés léxico** (los lectores de pantalla cambian el tono de voz; visualmente se muestra en cursiva). Se prefiere a la etiqueta puramente estética `<i>`.
  - **`<mark>`:** Resalta texto como si se le pasara un marcador fluorescente (útil para términos coincidentes en un buscador).
  - **`<code>`:** Representa fragmentos de código fuente en tipografía monoespaciada.
  - **`<pre>`:** Bloque preformateado que conserva exactamente todos los espacios en blanco, tabulaciones y saltos de línea del código original.

2.1.6 Hiperenlaces (<a> - Anchor Tags)

El hipertexto es el elemento distintivo de la Web: permite conectar documentos entre sí mediante enlaces interactivos:

```
<!-- 1. Enlace absoluto a un servidor externo -->
<a href="https://www.urjc.es" target="_blank" rel="noopener noreferrer">
    Universidad Rey Juan Carlos</a>

<!-- 2. Enlace relativo a otra pagina en el mismo servidor -->
<a href="contacto.html">Ir a Contacto</a>

<!-- 3. Enlace interno a una seccion dentro de la misma pagina (Ancla) -->
<a href="#requisitos">Saltar a la seccion de Requisitos</a>

<!-- Destino del ancla en la misma pagina -->
<h2 id="requisitos">Requisitos del Sistema</h2>

<!-- 4. Protocolos especiales: correo electronico y llamada telefonica -->
<a href="mailto:soporte@empresa.com">Enviar Correo</a>
<a href="tel:+34912345678">Llamar a Soporte</a>
```

Listing 2.2: Variantes de hiperenlaces en HTML5.

▲ ¡Atención / Ingeniería!: Seguridad en Hiperenlaces: La Vulnerabilidad «Reverse Tabnabbing»

Cuando se utiliza el atributo `target="_blank"` para abrir un enlace en una pestaña nueva, la página de destino obtiene acceso al objeto JavaScript `window.opener` de la página original. Un sitio web malicioso podría cambiar la ubicación de la pestaña original (`window.opener.location = 'sitio-phishing.html'`) sin que el usuario se dé cuenta. Para neutralizar esta vulnerabilidad de seguridad, los ingenieros deben incluir siempre el

atributo:

`rel="noopener noreferrer"`

Los navegadores modernos aplican `rel="noopener"` automáticamente por defecto al usar `target="_blank"`, pero es una buena práctica de ingeniería declararlo explícitamente.

2.1.7 Imágenes y Contenido Multimedia

La inclusión de recursos gráficos se realiza mediante la etiqueta autoclusiva `<img>`:

```
<figure>
    <img src="img/arquitectura.png"
         alt="Diagrama arquitectonico de la red cliente-servidor mostrando la peticion HTTP"
         width="800"
         height="450"
         loading="lazy">
    <figcaption>Figura 1: Flujo de peticiones a traves de la pasarela de servicios.</figcaption>
</figure>
```

Listing 2.3: Inclusión semántica y accesible de imágenes.

- **El Atributo Obligatorio `alt` (Texto Alternativo):**
  1. **Accesibilidad Universal:** Los usuarios invidentes o con baja visión utilizan lectores de pantalla que leen en voz alta la descripción del atributo `alt`.
  2. **Resiliencia de Red:** Si la imagen no carga por fallo de conexión o ruta rota, el navegador muestra el texto descriptivo.
  3. **SEO:** Los robots de Google y otros motores de búsqueda no «ven» píxeles; entienden el contenido de las imágenes leyendo el atributo `alt`.
- **Dimensiones (`width` y `height`) y Rendimiento:** Indicar siempre las dimensiones nativas en la etiqueta permite al navegador reservar el espacio exacto en pantalla antes de que la imagen termine de descargarse, previniendo el molesto salto de maquetación conocido en ingeniería web como **CLS (Cumulative Layout Shift)**.
- **Formatos Gráficos en la Web:**
  - **PNG:** Compresión sin pérdida, soporte de canal alfa (transparencia limpia). Ideal para logotipos, esquemas, capturas de pantalla y gráficos con pocos colores sólidos.
  - **JPEG / JPG:** Compresión con pérdida. Ideal para fotografías complejas y texturas con millones de gradientes de color.
  - **GIF:** Limitado a 256 colores. Útil históricamente para animaciones simples de bajo peso.
  - **SVG (Scalable Vector Graphics):** Formato vectorial basado en XML. No contiene píxeles, sino ecuaciones geométricas (líneas, curvas, polígonos). Es infinitamente escalable a cualquier resolución de pantalla (pantallas Retina o 4K) sin perder nitidez y su peso es ridículamente pequeño. Es el estándar indiscutible para iconos y logotipos modernos.
  - **WebP y AVIF:** Formatos modernos de alta compresión desarrollados para optimizar la velocidad de la web, reduciendo el peso de las imágenes entre un 30 % y un 50 % respecto a JPEG sin pérdida perceptible de calidad.

2.1.8 Listas de Elementos

HTML proporciona tres tipos de listas bien diferenciadas:

1. **Listas No Ordenadas (`<ul>`):** Colección de elementos (`<li>`) donde el orden de aparición no altera el significado. Visualmente se representan con viñetas (*bullets*).
2. **Listas Ordenadas (`<ol>`):** Colección indexada numéricamente donde la secuencia de los pasos es relevante (recetas, instrucciones de instalación). Admite atributos como `start="5"`, `reversed` o `type=".A|a|I|i|1"`.
3. **Listas de Definición (`<dl>`):** Pares de términos (`<dt>` - *definition term*) y sus correspondientes descripciones (`<dd>` - *definition description*). Ideales para glosarios, diccionarios o listas de especificaciones técnicas.

```
<!-- Lista ordenada con sublista anidada -->
<ol>
    <li>Fase de Disenio
        <ul>
            <li>Wireframing en Figma</li>
            <li>Definicion de paleta cromatica</li>
        </ul>
    </li>
    <li>Fase de Implementacion</li>
</ol>

<!-- Lista de definicion -->
<dl>
    <dt>HTTP</dt>
    <dd>Protocolo de transferencia de hipertexto.</dd>
    <dt>CSS</dt>
    <dd>Hojas de estilo en cascada para la capa visual.</dd>
</dl>
```

Listing 2.4: Listas anidadas y de definición en HTML5.

2.1.9 Tablas de Datos Semánticas

Las tablas en HTML deben reservarse **exclusivamente para datos tabulares** (series numéricas, horarios, inventarios, matrices comparativas). Su estructura semántica completa consta de:

```
<table>
    <caption>Listado de Alumnos y Calificaciones de la Asignatura</caption>
    <thead>
        <tr>
            <th scope="col">ID</th>
            <th scope="col">Nombre</th>
            <th scope="col">Apellidos</th>
            <th scope="col">Calificacion</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>01</td>
            <td>David</td>
            <td>Sticea</td>
            <td>Sobresaliente (10)</td>
        </tr>
        <tr>
            <td>02</td>
```

```
            <td>Elena</td>
            <td>Gomez</td>
            <td>Notable (8.5)</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <th scope="row" colspan="3">Nota Media del Grupo</th>
            <td>9.25</td>
        </tr>
    </tfoot>
</table>
```

Listing 2.5: Estructura completa de una tabla semántica profesional.

- **`<caption>`:** Título descriptivo accesible de la tabla.
- **`<thead>`, `<tbody>`, `<tfoot>`:** Agrupaciones lógicas que permiten a los navegadores y a la impresión en papel repetir las cabeceras en cada página si la tabla es muy larga.
- **`<th>` vs `<td>`:** `<th>` define una celda de encabezado (en negrita y centrada por defecto) e incluye el atributo `scope=çol|row"` para orientar a los lectores de pantalla; `<td>` es una celda ordinaria de datos.
- **Fusión de Celdas:**
  - `colspan="n"`: Expande la celda a lo largo de n columnas horizontales.
  - `rowspan="n"`: Expande la celda verticalmente a lo largo de n filas consecutivas.

2.1.10 HTML5 Semántico: Maquetación Estructural Moderna

Antes de la llegada de HTML5, los desarrolladores maquetaban la totalidad de los sitios web anidando infinitos elementos genéricos `<div>` con identificadores o clases inventadas (`<div id="header»`, `<div class="menu»`, `<div id="sidebar»`), un fenómeno conocido en la profesión como **«divitis»**. Esto impedía que los motores de búsqueda y los lectores de accesibilidad comprendieran la estructura real de la página.

HTML5 introdujo un conjunto canónico de **etiquetas semánticas estructurales** que describen con precisión el propósito de cada región:

- **`<header>`:** Cabecera introductoria de la página o de una sección individual. Suele contener el logo, título principal y barras de búsqueda.
- **`<nav>`:** Bloque que contiene los enlaces de navegación fundamentales del sitio web.
- **`<main>`:** Representa el contenido central y sustancial único de la página. **Solo puede existir un elemento `<main>` visible por documento** y no debe contener bloques repetitivos como cabeceras o pies de página globales.
- **`<article>`:** Bloque de contenido independiente y autocontenido que tendría pleno sentido si se extrajera de la página y se leyera por separado (por ejemplo: un artículo de blog, una noticia, una tarjeta de producto o un comentario de un usuario).
- **`<section>`:** Agrupación temática genérica de contenido. Cada `<section>` suele comenzar con su propio encabezado (`<h2>`–`<h6>`).
- **`<aside>`:** Contenido tangencial o secundario respecto al flujo principal (barras laterales, enlaces a temas relacionados, publicidad).

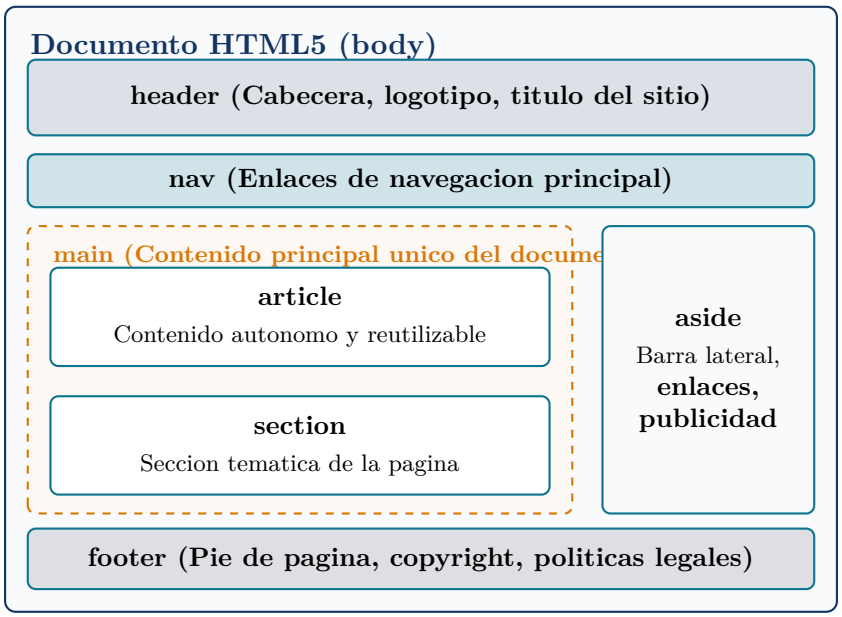


Figura 2.1: Esquema visual de la arquitectura semántica en un documento HTML5.

- **`<footer>`:** Pie de página que alberga información de autoría, derechos de copia (copyright), enlaces a políticas de privacidad y datos de contacto.

2.1.11 Formularios Web Interactivos (`<form>`)

Los formularios son el principal mecanismo nativo para que el usuario introduzca datos y preferencias que serán procesados por el servidor web o por JavaScript en el cliente.

```
<form action="/api/v1/registro" method="POST" enctype="application/x-www-form-urlencoded">
    <!-- Campo de texto con label asociado mediante id -->
    <div>
        <label for="nombreUsuario">Nombre de Usuario:</label>
        <input type="text" id="nombreUsuario" name="username" required minlength="4" maxlength="20" placeholder="Ej. d_sticea">
    </div>

    <!-- Correo electronico con validacion nativa -->
    <div>
        <label for="correo">Correo Electronico:</label>
        <input type="email" id="correo" name="email" required placeholder="usuario@dominio.com">
    </div>

    <!-- Contrasenia segura -->
    <div>
        <label for="clave">Contrasenia:</label>
        <input type="password" id="clave" name="password" required minlength="8">
    </div>

    <!-- Selector desplegable -->
    <div>
        <label for="rol">Rol Profesional:</label>
        <select id="rol" name="role">
            <option value="dev">Desarrollador de Software</option>
            <option value="qa">Ingeniero de QA / Testing</option>
```

```
            <option value="devops">Especialista DevOps</option>
        </select>
    </div>

    <!-- Botones de radio (exclusivos entre si por compartir el atributo name) -->
    <fieldset>
        <legend>Tipo de Suscripcion:</legend>
        <label><input type="radio" name="plan" value="free" checked> Gratuita</label>
        <label><input type="radio" name="plan" value="premium"> Premium</label>
    </fieldset>

    <!-- Casilla de verificacion (Checkbox) -->
    <div>
        <label>
            <input type="checkbox" name="terminos" required>
            Acepto los terminos y condiciones legales
        </label>
    </div>

    <!-- Botones de accion -->
    <button type="submit">Registrar Cuenta</button>
    <button type="reset">Limpiar Formulario</button>
</form>
```

Listing 2.6: Formulario accesible y validado en HTML5.

Conceptos Críticos de Formularios en Ingeniería

1. **Atributos `action` y `method`:** `action` especifica la URL del endpoint del servidor que recibirá los datos. `method` determina el verbo HTTP:
   - `GET`: Concatena los datos del formulario directamente en la URL (*query string*). Es visible en el historial y barra de navegación; **jamás debe usarse para contraseñas o datos sensibles**. Adecuado para motores de búsqueda o filtros.
   - `POST`: Envía los datos empaquetados en el cuerpo (*body*) de la petición HTTP. Seguro y sin límites prácticos de longitud.
2. **El Atributo `enctype`:** Por defecto es `application/x-www-form-urlencoded`. Si el formulario incluye campos para subir archivos (`<input type="file»`), es **estrictamente obligatorio** cambiarlo a `multipart/form-data`.
3. **La Asociación Semántica con `<label for="...»`:** La etiqueta `<label>` debe vincularse mediante su atributo `for` al atributo `id` del `<input>`. Esto garantiza dos cosas:
   - *Accesibilidad:* Los lectores de pantalla anuncian la etiqueta cuando el cursor entra en el campo.
   - *Usabilidad en Pantallas Táctiles:* Al pulsar con el dedo sobre el texto de la etiqueta, el campo de entrada recibe el foco automáticamente.
4. **El Atributo `name`:** Es el identificador clave que el servidor recibirá en el diccionario de datos. Un campo sin atributo `name` es completamente ignorado al enviar el formulario.
5. **Validación en Cliente vs. Servidor:** HTML5 incluye validación nativa automática (`required`, `pattern`, `minlength`, `maxlength`, `type=.ᵉᵐail|number"`).

▲ ¡Atención / Ingeniería!: Máxima de Seguridad en Software

La validación en cliente de HTML5 es una herramienta de usabilidad (*UX*) para orientar al usuario rápidamente, pero **nunca constituye una medida de seguridad**. Un atacante puede desactivar JavaScript, modificar el HTML en las DevTools o emitir una petición HTTP manipulada mediante herramientas como `curl` o Postman. **La validación estricta y desinfección de todos los campos en el servidor backend es obligatoria e ineludible.**

2.1.12 Elementos Genéricos: `<div>` y `<span>`

Cuando se requiere agrupar elementos únicamente para aplicar estilos CSS o comportamientos de JavaScript y ningún elemento semántico se ajusta a la naturaleza del bloque, se recurre a los elementos genéricos:

- **`<div>` (Division):** Elemento genérico de nivel de bloque (*block-level*). Ocupa por defecto el 100 % del ancho disponible y genera un salto de línea antes y después.
- **`<span>`:** Elemento genérico en línea (*inline*). Solo ocupa el ancho estricto de su contenido y no altera el flujo horizontal del texto.

2.2 CSS3: Cascada, Box Model, Flexbox y Grid Layout

2.2.1 Definición y Principios de CSS

► Concepto Clave: CSS (Cascading Style Sheets)

CSS es el lenguaje de hojas de estilo utilizado para describir la presentación visual, la maquetación espacial, los colores, tipografías y el comportamiento adaptativo de los documentos estructurados en HTML.

El principio fundacional de CSS es la **separación radical de responsabilidades**: HTML define *qué* información existe en la página, mientras que CSS define *cómo* se visualiza dicha información en cualquier pantalla (móvil, tableta, monitor ultra-panorámico o impresora).

2.2.2 Métodos de Inclusión de CSS

Existen tres formas de vincular código CSS a un documento HTML:

1. **Estilos en Línea (Inline Styles):** Mediante el atributo `style` en cualquier etiqueta:

```
<p style="color: red; font-size: 16px;">Texto en rojo</p>

```

   Veredicto de Ingeniería: Pésima práctica. Viola el principio de separación, ensucia el HTML, no es reutilizable y posee una especificidad destructiva que dificulta el mantenimiento.

2. **Hojas de Estilo Internas:** Mediante la etiqueta `<style>` dentro del `<head>`:

```
<head>
    <style>
        p { color: #143462; }
    </style>
</head>
```

```
6
```

Veredicto de Ingeniería: Aceptable únicamente para prototipos rápidos, pruebas de concepto o plantillas de correo electrónico HTML (que requieren estilos incrustados).

3. **Hojas de Estilo Externas (El Estándar Profesional):** En un archivo independiente con extensión `.css` vinculado desde el `<head>`:

```
1    <head>
2        <link rel="stylesheet" href="css/estilos.css">
3    </head>
4
```

Veredicto de Ingeniería: **La práctica recomendada por antonomasia**. Permite que múltiples páginas compartan las mismas reglas de estilo, facilita la división del trabajo y aprovecha los mecanismos de caché del navegador: una vez descargado el archivo CSS, no vuelve a pedirse a la red, acelerando enormemente la navegación.

2.2.3 Sintaxis de una Regla CSS

Toda regla CSS consta de dos partes: un **selector** que determina a qué elementos del DOM se aplicará la regla, y un **bloque de declaraciones** delimitado por llaves que contiene pares de propiedad y valor finalizados por punto y coma:

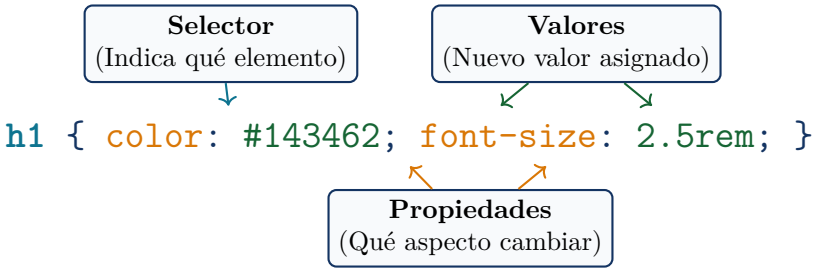


Figura 2.2: Anatomía formal de una regla CSS.

2.2.4 El Algoritmo de la Cascada, Especificidad y Herencia

¿Qué ocurre si a un mismo elemento HTML le aplican múltiples reglas CSS contradictorias? El motor del navegador resuelve el conflicto ejecutando el algoritmo formal de la **Cascada**, sustentado en tres pilares:

1. Origen e Importancia de la Regla

El navegador ordena las reglas según su origen en orden ascendente: estilos del navegador por defecto → estilos del usuario → estilos del autor (nuestro archivo CSS). Si se añade la directiva `!important` a una declaración, esta salta por encima de todas las demás reglas ordinarias.

▲ ¡Atención / Ingeniería!: Antipatrón de Ingeniería: El Abuso de !important

El uso indiscriminado de `!important` destruye el flujo natural de la cascada y desencadena guerras de especificidad donde cada regla nueva necesita otro `!important` para sobrescribir la anterior. En código profesional, su uso está restringido a casos de fuerza mayor (como sobreescrituras en utilidades aisladas de accesibilidad).

2. La Fórmula de la Especificidad

Si dos reglas tienen el mismo origen, el conflicto se desempata mediante el cálculo matemático de la **especificidad**, representada como una tupla de cuatro columnas (a, b, c, d):

Tabla 2.1: Cálculo de la especificidad en CSS.

Columna	Tipo de Selector	Ejemplos	Puntuación
a	Estilos en línea	`style="..."`	$(1, 0, 0, 0)$
b	Selectores de ID	`#menu-principal`, `#cabecera`	$(0, 1, 0, 0)$
c	Clases, atributos y pseudoclases	`.btn`, `[type="text"]`, `:hover`	$(0, 0, 1, 0)$
d	Elementos y pseudoelementos	`h1`, `div`, `p`, `::before`	$(0, 0, 0, 1)$

La comparación se realiza estrictamente de izquierda a derecha: una regla con un ID $(0, 1, 0, 0)$ vence a una regla con mil clases juntas $(0, 0, 1000, 0)$.

3. Orden de Aparición en el Código

Si dos reglas tienen exactamente la misma especificidad, gana **la última regla declarada en el código fuente** (el navegador lee secuencialmente de arriba abajo).

2.2.5 Colores y Variables CSS (Custom Properties)

CSS admite múltiples formas de especificar valores cromáticos:

- **Nombres de Color:** 140 nombres predefinidos (ej. `blue`, `red`, `steelblue`). Adecuados solo para pruebas rápidas.
- **Hexadecimal:** Formato `#RRGGBB` o compacto `#RGB` (donde cada canal varía de `00` a `FF`). Admite un cuarto canal de transparencia opcional: `#RRGGBBAA`.
- **Notación Funcional RGB / RGBA:** `rgb(255, 0, 0)` o `rgba(20, 52, 98, 0.85)` (donano el valor alfa varía de 0 totalmente transparente a 1 opaco).
- **Notación HSL / HSLA:** `hsl(hue, saturation, lightness)`. Tono (0 a 360 grados en el círculo cromático), Saturación (0 % gris a 100 % color puro) y Luminosidad (0 % negro a 100 % blanco). Es el formato predilecto de los diseñadores porque permite crear paletas armónicas variando solo la luminosidad.

```
/* Declaracion en el nodo raiz para alcance global */
:root {
    --color-primary: #143462;
    --color-accent: #d97706;
    --spacing-base: 1rem;
    --font-heading: 'Inter', sans-serif;
}

/* Uso dinamico en cualquier selector */
.btn-primary {
    background-color: var(--color-primary);
    color: white;
    padding: var(--spacing-base);
    font-family: var(--font-heading);
    transition: background-color 0.2s ease;
}

```

```
.btn-primary:hover {
    background-color: var(--color-accent);
}
```

Listing 2.7: Declaración y uso de variables nativas en CSS3.

2.2.6 Tipografía, Fuentes y Unidades de Medida

Pilas Tipográficas (*Font Stacks*) y Google Fonts

La propiedad `font-family` define la lista de fuentes a utilizar. Como no todas las máquinas tienen las mismas fuentes instaladas, se define una lista de reserva que culmina en una familia genérica del sistema:

```
body {
    /* Intenta cargar Inter; si no esta, busca Helvetica; si no, la sans-serif del sistema */
    font-family: 'Inter', 'Helvetica Neue', Arial, sans-serif;
}
```

Para utilizar fuentes web modernas de alta calidad, se utiliza **Google Fonts**, descargando la tipografía en la cabecera HTML mediante:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap" rel="stylesheet">
```

Unidades Absolutas vs. Relativas

- **Unidades Absolutas (`px`, `pt`):** `px` representa píxeles lógicos CSS. No varían en función de la configuración del usuario.
- **Unidades Relativas a Fuentes (`em` vs. `rem`):**
  - `em`: Es relativo al `font-size` del elemento padre inmediato. Es propenso a efectos multiplicativos indeseados en estructuras anidadas.
  - **`rem` (Root EM):** Es relativo únicamente al tamaño de fuente definido en el elemento raíz (`<html>`, por defecto 16px en todos los navegadores). Por tanto, `1rem = 16px`, `1.5rem = 24px`, `2rem = 32px`.

> **▲ ¡Atención / Ingeniería!: Por qué rem es el Estándar de Accesibilidad**
>
> Si un usuario con problemas de visión configura el tamaño de fuente predeterminado de su navegador en 24px en lugar de 16px, una página maquetada con `rem` escalará armónicamente todos los textos, márgenes y botones de manera automática. Una página maquetada rígidamente en `px` ignorará la preferencia del usuario, rompiendo la accesibilidad.

- **Unidades de Ventana Gráfica (Viewport Units):**
  - `1vw`: El 1 % del ancho total de la ventana visible del navegador.
  - `1vh`: El 1 % de la altura total de la ventana visible.

2.2.7 El Modelo de Caja (*Box Model*)

En CSS, todo elemento del DOM es concebido por el motor de renderizado como una caja rectangular compuesta por cuatro capas concéntricas:

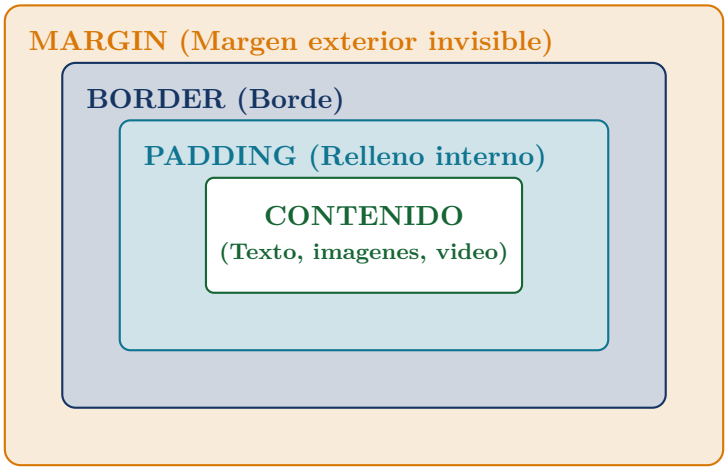


Figura 2.3: Capas concéntricas del Modelo de Caja en CSS.

1. **Content (Contenido):** El área real donde se dibuja el texto o las imágenes.
2. **Padding (Relleno):** Espacio transparente interior entre el contenido y el borde. Hereda el color de fondo del elemento.
3. **Border (Borde):** Línea que delimita el elemento. Posee grosor, estilo (`solid`, `dashed`) y color.
4. **Margin (Margen):** Espacio exterior transparente que separa el elemento de sus vecinos.

▲ ¡Atención / Ingeniería!: El Reseteo Universal Obligatorio: content-box vs. border-box

Por defecto histórico, el modelo de caja de CSS es `box-sizing: content-box`. En este modo, si definimos `width: 300px; padding: 20px; border: 5px solid black;`, el ancho total real que ocupará la caja en la pantalla será:

$$\text{Ancho Total} = 300 + (2 \times 20) + (2 \times 5) = 350\text{px}$$

Esto causaba que añadir padding rompiera continuamente las maquetaciones calculadas en columnas porcentuales.
Para solucionar esto de raíz, la industria del software utiliza universalmente el siguiente **reseteo en la primera línea de todo archivo CSS**:

```
*, *::before, *::after {
    box-sizing: border-box;
}
```

Con `border-box`, el ancho declarado (`width: 300px`) pasa a ser la dimensión exterior total fija; el padding y el borde se descuentan automáticamente hacia adentro, facilitando enormemente el cálculo mental y espacial.

2.2.8 Catálogo Avanzado de Selectores CSS

- **Selector Universal (*):** Selecciona todos y cada uno de los elementos de la página.

- **Selectores Básicos:** Por etiqueta (`p`), por clase (`.destacado`) y por ID único (`#cabecera`).
- **Combinadores Jerárquicos:**
  - *Descendiente (`A B`):* Selecciona cualquier elemento `B` que esté dentro de `A`, sin importar la profundidad del anidamiento.
  - *Hijo Directo (`A > B`):* Selecciona elementos `B` que sean hijos directos e inmediatos de `A`.
  - *Hermano Adyacente (`A + B`):* Selecciona el elemento `B` que esté inmediatamente a continuación de `A` al mismo nivel jerárquico.
  - *Hermano General (`A ~ B`):* Selecciona todos los hermanos `B` que sigan a `A` compartiendo el mismo elemento padre.
- **Selectores de Atributo:**
  - `[target]`: Elementos que posean dicho atributo.
  - `[type="button"]`: Coincidencia exacta de valor.
  - `[href^="https"]`: Cuyo valor comience por dicha cadena.
  - `[src$=".png"]`: Cuyo valor termine por dicha extensión.
  - `[class*="btn-"]`: Cuyo valor contenga la subcadena indicada.
- **Pseudoclases Esenciales:**
  - *De Interacción y Estado:* `:hover` (ratón encima), `:active` (momento exacto del clic presionado), `:focus` (elemento con foco activo del teclado o cursor), `:visited` (enlace ya visitado previamente).
  - *Estructurales:* `:first-child`, `:last-child`, `:nth-child(even)` (filas pares), `:nth-child(2n+1)` (filas impares), `:not(.excluido)`.
- **Pseudoelementos:** Permiten dar estilo a fragmentos abstractos del elemento o insertar contenido decorativo sin alterar el HTML: `::before` y `::after` (obligan a definir la propiedad `content: `), `::selection` (color de selección de texto del usuario).

2.2.9 Comportamiento de Visualización: La Propiedad `display`

La propiedad `display` determina cómo interactúa la caja en el flujo normal del documento:

- `display: block`: Ocupa todo el ancho horizontal disponible forzando saltos de línea. Respeta `width`, `height`, `margin` y `padding` en los cuatro lados (ej. `<div>`, `<p>`, `<h1>`).
- `display: inline`: Fluye de forma natural en la misma línea que el texto. Ignora las propiedades `width` y `height`; los márgenes verticales no desplazan a los elementos adyacentes (ej. `<span>`, `<a>`, `<strong>`).
- `display: inline-block`: El híbrido perfecto. Se alinea horizontalmente en la misma línea que otros elementos, pero **respeta escrupulosamente las dimensiones declaradas de anchura, altura y márgenes**. Ideal para botones o tarjetas en línea.
- `display: none`: Elimina completamente el elemento del árbol de renderizado. No ocupa ningún espacio visual en pantalla.

2.2.10 Posicionamiento Tradicional (`position`)

- `position: static`: El valor por defecto. El elemento sigue el flujo normal del documento. Ignora las propiedades de coordenadas (`top`, `left`, `right`, `bottom`, `z-index`).
- `position: relative`: Se desplaza respecto a la posición natural que le correspondía originalmente, **pero preserva el hueco vacío original en el flujo**, sin mover a los elementos vecinos.
- `position: absolute`: Se desvincula por completo del flujo normal y se posiciona de forma exacta respecto a las coordenadas del **ancestro posicionado más cercano** (cualquier contenedor previo que tenga una posición distinta a `static`). Si no existe tal ancestro, se posiciona respecto al área de visualización inicial (`<html>`).
- `position: fixed`: Queda fijado rígidamente a la ventana gráfica del navegador (*viewport*). No se desplaza aunque el usuario haga scroll vertical en la página (ideal para barras de navegación superiores permanentes o botones flotantes de chat).
- `position: sticky`: Actúa como `relative` durante el scroll ordinario hasta que cruza un umbral específico de coordenadas (ej. `top: 0`), instante en el cual se «adhiere» permanentemente a la pantalla comportándose como `fixed`.
- **Índice de Profundidad (`z-index`):** Entero que especifica el orden de apilamiento tridimensional hacia el usuario de elementos posicionados. Requiere una posición distinta de `static`.

2.2.11 Maquetación Moderna 1: Flexbox (Flexible Box Layout)

▶ Concepto Clave: Flexbox

Flexbox es un modelo de maquetación unidimensional diseñado para distribuir eficientemente el espacio libre entre elementos y alinearlos de forma flexible a lo largo de un único eje rector (el **eje principal**), adaptándose a cualquier tamaño de pantalla.

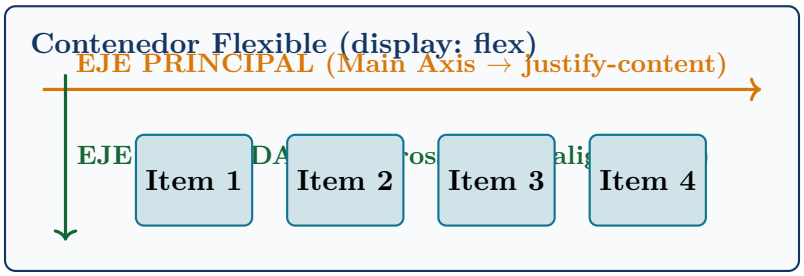


Figura 2.4: Diagrama de ejes y alineación en Flexbox.

Propiedades del Contenedor (`display: flex`)

- **`flex-direction`:** Determina la orientación del eje principal:
  - `row` (por defecto): Horizontal de izquierda a derecha.
  - `row-reverse`: Horizontal de derecha a izquierda.
  - `column`: Vertical de arriba a abajo.
  - `column-reverse`: Vertical de abajo a arriba.

- **`flex-wrap`:** Controla si los elementos hijos deben mantenerse en una sola línea forzosa o pueden desbordar en múltiples filas:
  - `nowrap` (por defecto): Todos los items se comprimen en una sola línea.
  - `wrap`: Los elementos que no caben saltan a la siguiente línea.
  - `wrap-reverse`: Salto de línea hacia arriba.
- **`justify-content`:** Alinea y distribuye el espacio sobrante en el **eje principal**:
  - `flex-start`: Alineados al inicio del eje.
  - `flex-end`: Alineados al final del eje.
  - `center`: Centrados en el eje principal.
  - `space-between`: El primer item pegado al inicio, el último al final, y espacio equidistante entre ellos.
  - `space-around`: Espacio igual a ambos lados de cada elemento (los extremos tienen la mitad de espacio que el hueco interior).
  - `space-evenly`: Espaciado matemáticamente idéntico entre todos los elementos y los bordes exteriores.
- **`align-items`:** Alinea los elementos en el **eje secundario (cruzado)**:
  - `stretch` (por defecto): Se estiran para ocupar todo el alto del contenedor.
  - `flex-start` / `flex-end`: Alineados al inicio o final del eje cruzado.
  - `center`: Centrado vertical perfecto en el eje secundario.
  - `baseline`: Alineados según la línea base de sus textos.
- **`gap`:** La forma moderna y elegante de definir separación limpia entre elementos (`gap: 1.5rem;`), eliminando la necesidad de márgenes laterales complejos.

Propiedades de los Elementos Hijos (Flex Items)

- **`flex-grow`:** Factor de crecimiento proporcional para absorber el espacio disponible residual del contenedor (por defecto 0).
- **`flex-shrink`:** Factor de contracción proporcional cuando el espacio es insuficiente (por defecto 1).
- **`flex-basis`:** El tamaño base inicial del elemento antes de aplicar factores de crecimiento o contracción (ej. `flex-basis: 250px;`).
- **Shorthand Canónico:** `flex: <grow> <shrink> <basis>;` (ejemplo habitual: `flex: 1 1 auto;`).
- **`order`:** Modifica el orden visual de representación sin alterar el orden original del DOM HTML.

2.2.12 Maquetación Moderna 2: CSS Grid Layout

▶ Concepto Clave: CSS Grid Layout

CSS Grid es el sistema de maquetación **bidimensional** más potente de CSS. A diferencia de Flexbox (que opera fundamentalmente en filas o columnas aisladas), Grid permite estructurar el espacio simultáneamente en filas y columnas ortogonales, asemejándose a una mesa de dibujo arquitectónico.

```
.galeria-grid {
    display: grid;
    /* Crea columnas de al menos 250px que se expanden a partes iguales */
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    grid-gap: 1.5rem;
}

.item-destacado {
    /* Ocupa desde la columna 1 hasta la 3 */
    grid-column: 1 / 3;
    /* Ocupa 2 filas consecutivas */
    grid-row: span 2;
}
```

Listing 2.8: Configuración de una rejilla moderna con CSS Grid.

Conceptos Fundamentales de Grid

- **Pistas de Cuadrícula (Grid Tracks):** Las filas o columnas individuales definidas con `grid-template-columns` y `grid-template-rows`.
- **La Unidad de Fracción (`fr`):** Representa una fracción matemática del espacio libre restante dentro del contenedor de cuadrícula. Por ejemplo: `grid-template-columns: 1fr 2fr 1fr;` divide el ancho en 4 partes iguales, asignando la mitad del espacio total a la columna central.
- **Funciones Auxiliares Potentes:**
  - `repeat(3, 1fr)`: Evita repetir valores idénticos (`1fr 1fr 1fr`).
  - `minmax(min, max)`: Garantiza que una pista nunca baje de un mínimo ni supere un máximo.
  - `auto-fit` con `minmax`: Permite crear cuadrículas responsivas que se reorganizan solas según el tamaño de la pantalla **sin necesidad de escribir una sola media query**.
- **Maquetación Visual por Áreas (`grid-template-areas`):** Permite maquetar el sitio entero «dibujando» literalmente su estructura en código ASCII:

```
.layout-pagina {
    display: grid;
    grid-template-areas:
        "header  header"
        "sidebar main"
        "footer  footer";
    grid-template-columns: 250px 1fr;
}
.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
```

```
    .footer  { grid-area: footer; }

```

Tabla 2.2: Criterios de decisión en ingeniería: Flexbox vs. CSS Grid.

Criterio de Diseño	Flexbox (Flexible Box Layout)	CSS Grid Layout
Dimensionalidad	Unidimensional (Fila **o** Columna).	Bidimensional (Filas **y** Columnas simultáneas).
Enfoque de Diseño	Basado en el **contenido**: los elementos definen su propio tamaño y se adaptan fluidamente.	Basado en la **estructura**: se define la cuadrícula primero y los elementos se ubican en ella.
Casos de Uso Típicos	Barras de navegación, grupos de botones, alineación vertical de un icono con un texto, tarjetas en carrusel.	Maquetación global de páginas completas, dashboards de analítica, galerías de fotos tipo mosaico irregular.

2.3 Bootstrap 5: Framework Front-End para Diseño Rápido

2.3.1 Definición, Filosofía y Evolución

▶ Concepto Clave: Bootstrap

Bootstrap es el framework de código abierto (bajo licencia permisiva MIT) más extendido a nivel global para el diseño rápido, estructurado y adaptable a dispositivos móviles de interfaces web front-end. Fue creado originalmente en Twitter por Mark Otto y Jacob Thornton.

Más de dos millones de sitios web a nivel internacional se construyen con Bootstrap. La versión actual, **Bootstrap 5**, introdujo una profunda revolución técnica respecto a versiones históricas:

- **Eliminación Total de jQuery:** Bootstrap 5 reescribió todos sus componentes interactivos (modales, tooltips, acordeones) en **JavaScript puro (Vanilla JS)**, reduciendo drásticamente el peso de la página y mejorando el rendimiento.
- **Integración Nativa de Popper.js:** Motor robusto para el posicionamiento matemático de menús desplegables y tooltips.
- **Adopción Masiva de Variables CSS:** Permite cambiar dinámicamente colores temáticos en tiempo de ejecución.
- **Soporte Nativo de Modo Oscuro (*Dark Mode*):** Implementado fácilmente mediante el atributo HTML `data-bs-theme="dark"`.

2.3.2 Inclusión de Bootstrap en Proyectos

La manera más rápida y eficiente de integrar Bootstrap 5 en producción es mediante su red de distribución de contenidos (**CDN**):

```
<!DOCTYPE html>
<html lang="es">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sitio Profesional con Bootstrap 5</title>
    <!-- 1. CSS de Bootstrap -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <!-- 2. Iconos Oficiales de Bootstrap -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
</head>
<body>
    <div class="container py-4">
        <h1 class="display-4 text-primary">¡Hola con Bootstrap 5!</h1>
    </div>

    <!-- 3. Bundle JS de Bootstrap (incluye Popper.js de forma integrada) -->
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Listing 2.9: Plantilla canónica inicial con Bootstrap 5.3 CDN.

2.3.3 El Sistema de Maquetación: Contenedores y Grid de 12 Columnas

El corazón arquitectónico de Bootstrap es su sistema de maquetación responsivo basado en **Flexbox**:

1. Contenedores (`.container` vs `.container-fluid`)

El contenedor es el elemento padre obligatorio que envuelve y centra la cuadrícula:

- `.container`: Ancho responsivo fijo que salta a un ancho determinado al alcanzar cada punto de interrupción (*breakpoint*), manteniendo márgenes automáticos centrados en pantalla.
- `.container-fluid`: Ocupa el 100 % del ancho de la pantalla de forma continua y fluida en cualquier resolución.

2. Los Breakpoints Oficiales de Bootstrap 5

Bootstrap define seis puntos de corte estándar basados en el ancho mínimo en píxeles:

- **Extra Pequeño (`xs`):** $<$ 576px (móviles verticales, sin infijo de clase: `.col-12`).
- **Pequeño (`sm`):** $\geq$ 576px (móviles apaisados / pantallas pequeñas).
- **Medio (`md`):** $\geq$ 768px (tabletas verticales).
- **Grande (`lg`):** $\geq$ 992px (portátiles y ordenadores de escritorio estándar).
- **Extra Grande (`xl`):** $\geq$ 1200px (monitores de alta resolución).
- **Extra Extra Grande (`xxl`):** $\geq$ 1400px (pantallas ultrapanorámicas de gran formato).

3. La Jerarquía Sagrada: Container → Row → Col

▲ ¡Atención / Ingeniería!: Regla Estructural Inviolable en Bootstrap

En Bootstrap, **las filas (`.row`) deben ser hijas directas de un contenedor (`.container`)**, y **las columnas (`.col-*`) deben ser hijas directas e inmediatas de una fila (`.row`)**.
Una fila aplica márgenes negativos para compensar el padding horizontal de las columnas (*gutters*). Colocar contenido directamente dentro de un `.row` sin una columna que lo envuelva destruye la alineación matemática de la rejilla.

El sistema divide la pantalla horizontalmente en **12 columnas virtuales**. Los elementos declaran cuántas columnas ocupan mediante clases del tipo `.col-{breakpoint}-{número}`:

```
<div class="container">
    <div class="row g-4">
        <!-- En movil ocupa 12 columnas (100%), en tablet 6 (50%) y en escritorio 4 (33.3%) -->
        <div class="col-12 col-md-6 col-lg-4">
            <div class="p-3 bg-light border rounded">Columna 1</div>
        </div>
        <div class="col-12 col-md-6 col-lg-4">
            <div class="p-3 bg-light border rounded">Columna 2</div>
        </div>
        <div class="col-12 col-md-6 col-lg-4">
            <div class="p-3 bg-light border rounded">Columna 3</div>
        </div>
    </div>
</div>
```

Listing 2.10: Ejemplo del sistema Grid adaptable a múltiples pantallas.

2.3.4 Paleta Semántica de Colores y Tipografía

Bootstrap proporciona un sistema consistente de nombres semánticos que se aplica a textos (`text-*`) y fondos (`bg-*`):

- `primary` (azul corporativo), `secondary` (gris sobrio), `success` (verde éxito o confirmación), `danger` (rojo error o alerta crítica), `warning` (amarillo advertencia), `info` (cian informativo), `light` (gris muy claro) y `dark` (antracita oscuro).
- **Tipografía:** Clases de titulares display (`.display-1` a `.display-6`), textos destacados con `.lead`, textos secundarios con `.text-muted` o `.text-body-secondary`.

2.3.5 Catálogo de Componentes UI Esenciales

- **Botones (`.btn`):** Se combinan clases base y modificadores: `.btn .btn-primary`, versiones de contorno hueco `.btn-outline-danger`, tamaños `.btn-lg` o `.btn-sm`, y grupos cohesionados con `.btn-group`.
- **Iconografía (Bootstrap Icons):** Más de 1.800 iconos vectoriales libres integrables fácilmente mediante clases CSS: `<i class="bi bi-person-check-fill"></i>`.
- **Badges / Insignias (`.badge`):** Pequeñas etiquetas numéricas o de estado: `<span class="badge bg-success rounded-pill">Activo</span>`.
- **Barras de Progreso (`.progress` y `.progress-bar`):**

```
<div class="progress" role="progressbar" aria-valuenow="75" aria-valuemin="0" aria-valuemax="100">
    <div class="progress-bar progress-bar-striped progress-bar-animated bg-info" style="width: 75%">75%</div>
</div>

```

- **Paginación (`.pagination`):** Estructurada mediante listas `<ul class="pagination»` con elementos `.page-item` y enlaces `.page-link`, admitiendo estados `.active` y `.disabled`.
- **Grupos de Listas (`.list-group`):** Componente flexible para presentar colecciones de elementos interactivos: `.list-group-item` y `.list-group-item-action`.
- **Tablas Estilizadas:** Simplemente agregando clases nativas al elemento estándar `<table>`:
  - `.table-striped`: Aplica un elegante sombreado estilo cebra a las filas alternas.
  - `.table-hover`: Efecto visual de realce al pasar el ratón por encima de cada fila.
  - `.table-bordered`: Cuadrícula completa con bordes limpios.
  - Envoltura `<div class="table-responsive»`: Garantiza que en pantallas móviles la tabla genere una barra de desplazamiento horizontal suave sin romper el diseño de la página.
- **Formularios Responsivos en Bootstrap:**
  - Controles de entrada: `.form-control` para campos de texto y contraseñas; `.form-select` para listas desplegables; `.form-check-input` para casillas y radios.
  - Grupos anexos (`.input-group`): Permite unir iconos o botones directamente pegados al campo de texto.
  - Validación visual instantánea: Las clases `.is-valid` e `.is-invalid` colorean automáticamente el borde en verde o rojo, mostrando los mensajes asociados en `.valid-feedback` o `.invalid-feedback`.
- **Barra de Navegación Profesional (`.navbar`):** El componente insignia de Bootstrap. Es completamente responsivo: en ordenadores de escritorio muestra los enlaces horizontales completos, mientras que en pantallas móviles colapsa automáticamente todos los enlaces dentro de un menú hamburguesa desplegable mediante el botón `.navbar-toggler`.

2.4 Ejercicios Prácticos Resueltos del Tema 2

A continuación se resuelven de forma íntegra y exhaustiva los ejercicios propuestos en el programa oficial de la asignatura.

2.4.1 Ejercicio 1 de HTML: Creación de Página Semántica sin Estilos

★ Ejercicio Resuelto: Ejercicio 1 (HTML): Portal Institucional Semántico sin Estilos

Enunciado Oficial: *Crea una página HTML (sin incluir estilos CSS). El contenido debe incluir, al menos: varias secciones y subsecciones, fotografías/imágenes con atributo alt, listas de elementos, tablas de datos e hiperenlaces (externos y anclas a la misma página).*
Solución de Ingeniería: Se implementa una página web corporativa estructurada para una empresa de software (*CloudPulse Technologies*), respetando la semántica de HTML5, accesibilidad universal y las mejores prácticas de la industria.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Portal corporativo de CloudPulse Technologies - Soluciones de Software">
    <title>CloudPulse Technologies | Soluciones Cloud y Desarrollo Web</title>
</head>
<body>

    <!-- Encabezado y Navegacion Principal -->
    <header>
        <h1>CloudPulse Technologies S.L.</h1>
        <p>Innovacion, Arquitectura de Software y Sistemas Distribuidos</p>
        <nav>
            <ul>
                <li><a href="#servicios">Nuestros Servicios</a></li>
                <li><a href="#productos">Planes y Precios</a></li>
                <li><a href="#contacto">Formulario de Contacto</a></li>
                <li><a href="https://www.urjc.es" target="_blank" rel="noopener noreferrer">Convenio Academico URJC</a></li>
            </ul>
        </nav>
    </header>

    <!-- Contenido Principal Unico -->
    <main>
        <!-- Seccion 1: Presentacion y Servicios -->
        <section id="servicios">
            <h2>Servicios de Ingenieria del Software</h2>
            <p>Diseniamos e implementamos plataformas digitales de alta escalabilidad con tecnologias modernas.</p>

            <figure>
                <img src="https://images.unsplash.com/photo-1518770660439-4636190af475?w=600"
                     alt="Circuito integrado representando procesamiento y arquitectura de sistemas distribuidos"
                     width="600" height="300">
                <figcaption>Infraestructura global optimizada para baja latencia.</figcaption>
            </figure>

            <h3>Areas de Especializacion Tecnica</h3>
            <ul>
                <li><strong>Desarrollo Front-End:</strong> Aplicaciones SPA con React, Angular y maquetacion semantica con HTML5/CSS3.</li>
```

```
42                 <li><strong>Arquitectura Back-End:</strong>
       Microservicios en Java Spring Boot, Node.js y APIs RESTful.</li>
43                 <li><strong>Bases de Datos Distribuidas:</strong>
       Persistencia hibrida relacional (PostgreSQL) y NoSQL (Redis,
       MongoDB).</li>
44             </ul>
45         </section>
46
47         <!-- Seccion 2: Tabla de Productos y Precios -->
48         <section id="productos">
49             <h2>Planes y Modelos de Servicio</h2>
50             <table>
51                 <caption>Matriz comparativa de planes de suscripcion
       CloudPulse 2026</caption>
52                 <thead>
53                     <tr>
54                         <th scope="col">Plan</th>
55                         <th scope="col">Capacidad Compute</th>
56                         <th scope="col">Almacenamiento</th>
57                         <th scope="col">SLA Garantizado</th>
58                         <th scope="col">Precio Mensual</th>
59                     </tr>
60                 </thead>
61                 <tbody>
62                     <tr>
63                         <th scope="row">Developer</th>
64                         <td>2 vCPUs / 4 GB RAM</td>
65                         <td>50 GB NVMe</td>
66                         <td>99.5%</td>
67                         <td>29.00 EUR</td>
68                     </tr>
69                     <tr>
70                         <th scope="row">Enterprise Pro</th>
71                         <td>8 vCPUs / 32 GB RAM</td>
72                         <td>500 GB NVMe</td>
73                         <td>99.99%</td>
74                         <td>199.00 EUR</td>
75                     </tr>
76                 </tbody>
77                 <tfoot>
78                     <tr>
79                         <th scope="row" colspan="4">Descuento Anual por
        Contratacion Anticipada</th>
80                         <td>-20%</td>
81                     </tr>
82                 </tfoot>
83             </table>
84         </section>
85
86         <!-- Seccion 3: Formulario de Contacto -->
87         <section id="contacto">
88             <h2>Solicitud de Informacion y Presupuesto</h2>
89             <form action="/api/solicitudes" method="POST">
90                 <div>
91                     <label for="clienteNombre">Nombre y Apellidos:</
       label>
92                     <input type="text" id="clienteNombre" name="nombre"
        required minlength="3">
93                 </div>
94                 <div>
```

```
                    <label for="clienteEmail">Correo Corporativo:</label>
                    <input type="email" id="clienteEmail" name="email" required>
                </div>
                <div>
                    <label for="tipoProyecto">Tipo de Proyecto:</label>
                    <select id="tipoProyecto" name="proyecto">
                        <option value="web">Desarrollo Web a Medida</option>
                        <option value="cloud">Migracion Cloud y DevOps</option>
                        <option value="auditoria">Auditoria de Rendimiento y Seguridad</option>
                    </select>
                </div>
                <div>
                    <label for="mensaje">Detalles del Proyecto:</label><br>
                    <textarea id="mensaje" name="descripcion" rows="4" cols="50" placeholder="Describa brevemente los requisitos tecnicos ..."></textarea>
                </div>
                <button type="submit">Enviar Solicitud</button>
            </form>
        </section>
    </main>

    <!-- Pie de Pagina -->
    <footer>
        <p>&copy; 2026 CloudPulse Technologies S.L. - Todos los derechos reservados.</p>
        <p><a href="#servicios">Volver al inicio de la pagina</a></p>
    </footer>

</body>
</html>
```

Listing 2.11: index_semantico.html: Solución completa al Ejercicio 1 de HTML.

2.4.2 Ejercicios Resueltos de CSS

★ Ejercicio Resuelto: Ejercicio 1 (CSS): Creación de un Botón Interactivo con Bordes Redondeados

Enunciado Oficial: *Crea una página web con HTML y CSS que implemente un botón estilizado con bordes redondeados y efectos interactivos.*
Solución de Ingeniería: Se modela una estructura limpia utilizando la etiqueta semántica `<a>` con apariencia de botón interactivo mediante clases CSS, transiciones suaves y estados pseudo-clase `:hover` y `:active`.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Boton Interactivo con CSS3</title>
    <link rel="stylesheet" href="button.css">
</head>
<body>
    <div class="contenedor-demo">
        <a href="https://www.urjc.es" class="btn-custom">Descubrir Grado</a>
    </div>
</body>
</html>
```

Listing 2.12: Estructura HTML del botón.

```
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    background-color: #f8fafc;
}

.btn-custom {
    display: inline-block;
    padding: 14px 32px;
    font-size: 16px;
    font-weight: 600;
    text-decoration: none;
    text-align: center;
    color: #ffffff;
    background-color: #143462; /* Azul corporativo */
    border: 2px solid #143462;
    border-radius: 30px;        /* Bordes completamente redondeados */
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
    transition: all 0.25s ease-in-out;
    cursor: pointer;
}

/* Efecto al pasar el cursor */
```

```
.btn-custom:hover {
    background-color: #d97706; /* Transicion a ambar */
    border-color: #d97706;
    box-shadow: 0 10px 15px -3px rgba(217, 119, 6, 0.3);
    transform: translateY(-2px); /* Ligero desplazamiento hacia arriba
    */
}

/* Efecto al hacer clic */
.btn-custom:active {
    transform: translateY(1px);
    box-shadow: 0 2px 4px -1px rgba(0, 0, 0, 0.1);
}
```

Listing 2.13: button.css: Estilos CSS del botón con bordes redondeados.

★ Ejercicio Resuelto: Ejercicio 2 (CSS): Replicación de Figura con Flexbox

Enunciado Oficial: *Trata de replicar la siguiente figura utilizando Flexbox con un contenedor que aloja 10 items numerados.*
Solución Canónica Oficial: El ejercicio evalúa la capacidad de comprender el flujo invertido de líneas con `flex-wrap: wrap-reverse`, la distribución en el eje principal con `space-around` y el centrado en el eje secundario con `align-items: center`.

```
body {
    padding-top: 1em;
}

.container {
    display: flex;
    align-items: center;
    justify-content: space-around;
    flex-wrap: wrap-reverse;
    background: steelblue;
    width: 100px;
    height: 100px;
}

.item {
    background: grey;
    min-height: 20px;
    min-width: 20px;
    border: 1px solid black;
    text-align: center;
    color: white;
    font-size: 11px;
    line-height: 20px;
}
```

Listing 2.14: Solucion canonica al Ejercicio 2 de Flexbox.

★ Ejercicio Resuelto: Ejercicio 3 (CSS): Replicación de Distribución con CSS Grid

Enunciado Oficial: *Trata de replicar la figura de 4 elementos coloreados utilizando CSS Grid.*
Solución Canónica Oficial: Se crea una rejilla donde el elemento `.a` se expande horizontalmente abarcando las dos primeras columnas (`grid-column-start: 1; grid-column-end: 3;`), y los elementos restantes se ubican secuencialmente en las filas inferiores.

```
.grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    grid-gap: 5px;
    max-width: 300px;
}

.item {
    color: white;
    padding: 1em;
    text-align: center;
```

```
    font-weight: bold;
}

/* Posicionamiento explicito en pistas de cuadricula */
.a {
    background: blue;
    grid-column-start: 1;
    grid-column-end: 3; /* Ocupa las dos columnas superiores */
}

.b {
    background: red;
    grid-row-start: 2;
}

.c {
    background: green;
    grid-row-start: 2;
}

.d {
    background: orange;
    grid-column-start: 1;
    grid-column-end: 3; /* Ocupa las dos columnas inferiores */
}
```

Listing 2.15: Solucion canonica al Ejercicio 3 de Grid.

2.4.3 Ejercicio 14 de Bootstrap Resuelto: Réplica de Portal Web

★ Ejercicio Resuelto: Ejercicio 14 (Bootstrap): Portal Académico CodeURJC Responsivo

Enunciado Oficial: *Replica la página web del ejercicio utilizando componentes Bootstrap 5, integrando la URL del logotipo oficial (`https://www.codeurjc.es/img/codeurjc_logo.jpg`) y personalizando los estilos de Bootstrap.*

Solución de Ingeniería: Se implementa una página completa y funcional utilizando el sistema de navegación colapsable (`navbar`), rejilla responsiva de 12 columnas con tarjetas (`cards`), tabla estilizada y formulario de inscripción, demostrando la integración limpia de estilos personalizados sobre las clases de Bootstrap.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CodeURJC - Centro de Desarrollo y Formacion Web</title>
    <!-- Bootstrap 5.3.3 CSS -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <!-- Bootstrap Icons -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">

    <!-- Estilos de Personalizacion sobre Bootstrap -->
    <style>
        :root {
            --bs-primary: #143462;
            --bs-primary-rgb: 20, 52, 98;
        }
        .navbar-brand img {
            height: 48px;
            object-fit: contain;
        }
        .hero-banner {
            background: linear-gradient(135deg, #143462 0%, #0e7490 100%);
            color: white;
            border-radius: 12px;
        }
        .card-custom {
            transition: transform 0.2s ease, box-shadow 0.2s ease;
            border: 1px solid rgba(0, 0, 0, 0.08);
        }
        .card-custom:hover {
            transform: translateY(-5px);
            box-shadow: 0 10px 20px rgba(0,0,0,0.1);
        }
    </style>
</head>
<body class="bg-light">

    <!-- 1. Barra de Navegacion Responsiva -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark sticky-top shadow-sm">
        <div class="container">
```

```
42              <a class="navbar-brand d-flex align-items-center gap-2"
    href="#">
43                  <img src="https://www.codeurjc.es/img/codeurjc_logo.jpg
    " alt="Logotipo Oficial CodeURJC" class="rounded bg-white p-1">
44                  <span class="fw-bold tracking-wide">CodeURJC</span>
45              </a>
46              <button class="navbar-toggler" type="button" data-bs-toggle
    ="collapse" data-bs-target="#navMenu" aria-controls="navMenu" aria-
    expanded="false" aria-label="Abrir navegacion">
47                  <span class="navbar-toggler-icon"></span>
48              </button>
49              <div class="collapse navbar-collapse" id="navMenu">
50                  <ul class="navbar-nav ms-auto mb-2 mb-lg-0">
51                      <li class="nav-item"><a class="nav-link active"
    href="#"><i class="bi bi-house-door me-1"></i>Inicio</a></li>
52                      <li class="nav-item"><a class="nav-link" href="#
    cursos"><i class="bi bi-journal-code me-1"></i>Cursos</a></li>
53                      <li class="nav-item"><a class="nav-link" href="#
    talleres"><i class="bi bi-calendar-event me-1"></i>Talleres</a></li
    >
54                      <li class="nav-item"><a class="nav-link btn btn-
    primary text-white ms-lg-3 px-3" href="#registro">Inscribirse</a></
    li>
55                  </ul>
56              </div>
57          </div>
58      </nav>
59
60      <!-- 2. Hero Banner Introductorio -->
61      <header class="container my-4">
62          <div class="hero-banner p-5 text-center shadow">
63              <h1 class="display-5 fw-bold">Aprende Ingenieria Web en
    CodeURJC</h1>
64              <p class="lead col-lg-8 mx-auto mt-3">
65                  Formacion de vanguardia en tecnologías de cliente y
    servidor: HTML5 semántico, CSS3 avanzado, Bootstrap 5 y
    arquitecturas web de alto rendimiento.
66              </p>
67              <div class="mt-4">
68                  <a href="#cursos" class="btn btn-warning btn-lg fw-
    semibold px-4 me-2">Ver Catalogo</a>
69                  <a href="#registro" class="btn btn-outline-light btn-lg
     px-4">Crear Cuenta</a>
70              </div>
71          </div>
72      </header>
73
74      <!-- 3. Seccion de Tarjetas con Sistema Grid de 12 Columnas -->
75      <section class="container my-5" id="cursos">
76          <div class="text-center mb-4">
77              <h2 class="fw-bold text-dark">Modulos Formativos Destacados
    </h2>
78              <p class="text-muted">Desarrolla competencias reales de la
    industria del software.</p>
79          </div>
80
81          <div class="row g-4">
82              <!-- Tarjeta 1: HTML5 y Accesibilidad -->
83              <div class="col-12 col-md-6 col-lg-4">
84                  <div class="card h-100 card-custom shadow-sm">
85                      <div class="card-body">
```

```
86                          <div class="d-flex align-items-center mb-3">
87                              <span class="badge bg-primary p-2 me-2"><i
    class="bi bi-filetype-html fs-4"></i></span>
88                              <h5 class="card-title fw-bold mb-0">HTML5
    Semantico</h5>
89                          </div>
90                          <p class="card-text text-secondary">Aprende a
    estructurar documentos limpios, accesibles para lectores de
    pantalla y optimizados para SEO.</p>
91                          <div class="progress mb-3" style="height: 8px;"
    >
92                              <div class="progress-bar bg-success" style=
    "width: 100%;"></div>
93                          </div>
94                          <a href="#" class="btn btn-sm btn-outline-
    primary w-100">Acceder al Temario</a>
95                      </div>
96                  </div>
97              </div>
98
99              <!-- Tarjeta 2: CSS3 y Maquetacion Avanzada -->
100             <div class="col-12 col-md-6 col-lg-4">
101                 <div class="card h-100 card-custom shadow-sm">
102                     <div class="card-body">
103                         <div class="d-flex align-items-center mb-3">
104                             <span class="badge bg-info text-white p-2
    me-2"><i class="bi bi-filetype-css fs-4"></i></span>
105                             <h5 class="card-title fw-bold mb-0">CSS3,
    Flexbox & Grid</h5>
106                         </div>
107                         <p class="card-text text-secondary">Domina el
    modelo de caja, cascada, especificidad y las herramientas de
    maquetacion moderna responsiva.</p>
108                         <div class="progress mb-3" style="height: 8px;"
    >
109                             <div class="progress-bar bg-info" style="
    width: 85%;"></div>
110                         </div>
111                         <a href="#" class="btn btn-sm btn-outline-info
    w-100">Acceder al Temario</a>
112                     </div>
113                 </div>
114             </div>
115
116             <!-- Tarjeta 3: Bootstrap 5 -->
117             <div class="col-12 col-md-6 col-lg-4">
118                 <div class="card h-100 card-custom shadow-sm">
119                     <div class="card-body">
120                         <div class="d-flex align-items-center mb-3">
121                             <span class="badge bg-warning text-dark p-2
     me-2"><i class="bi bi-bootstrap-fill fs-4"></i></span>
122                             <h5 class="card-title fw-bold mb-0">
    Bootstrap 5.3</h5>
123                         </div>
124                         <p class="card-text text-secondary">Construye
    interfaces agiles y adaptables con la cuadricula de 12 columnas y
    su catalogo de componentes UI.</p>
125                         <div class="progress mb-3" style="height: 8px;"
    >
126                             <div class="progress-bar bg-warning" style=
    "width: 70%;"></div>
```

```
                        </div>
                        <a href="#" class="btn btn-sm btn-outline-
    warning w-100">Acceder al Temario</a>
                    </div>
                </div>
            </div>
        </div>
    </section>

    <!-- 4. Seccion de Tabla Responsiva de Horarios -->
    <section class="container my-5" id="talleres">
        <div class="card shadow-sm border-0">
            <div class="card-header bg-dark text-white py-3">
                <h4 class="mb-0 fw-bold"><i class="bi bi-table me-2"></
    i>Calendario de Talleres Practicos</h4>
            </div>
            <div class="card-body p-0">
                <div class="table-responsive">
                    <table class="table table-striped table-hover mb-0
    align-middle">
                        <thead class="table-secondary">
                            <tr>
                                <th>Codigo</th>
                                <th>Taller</th>
                                <th>Laboratorio</th>
                                <th>Fecha</th>
                                <th>Estado</th>
                            </tr>
                        </thead>
                        <tbody>
                            <tr>
                                <td><strong>TL-101</strong></td>
                                <td>Maquetacion de Landing Pages con
    Grid</td>
                                <td>Lab III - Aula 204 (Mostoles)</td>
                                <td>Viernes, 16:00</td>
                                <td><span class="badge bg-success">
    Plazas Disponibles</span></td>
                            </tr>
                            <tr>
                                <td><strong>TL-102</strong></td>
                                <td>Disenio de Componentes con
    Bootstrap 5</td>
                                <td>Lab II - Aula 102 (Fuenlabrada)</td
    >
                                <td>Lunes, 18:30</td>
                                <td><span class="badge bg-warning text-
    dark">Ultimas Plazas</span></td>
                            </tr>
                            <tr>
                                <td><strong>TL-103</strong></td>
                                <td>Integracion de APIs REST y JSON</td
    >
                                <td>Online (Teams)</td>
                                <td>Miercoles, 17:00</td>
                                <td><span class="badge bg-secondary">
    Completo</span></td>
                            </tr>
                        </tbody>
                    </table>
                </div>
```

```
178             </div>
179         </div>
180     </section>
181 
182     <!-- 5. Formulario de Inscripcion Estilizado -->
183     <section class="container my-5" id="registro">
184         <div class="row justify-content-center">
185             <div class="col-12 col-md-8 col-lg-6">
186                 <div class="card shadow border-0 p-4">
187                     <h3 class="fw-bold text-center mb-3">Inscripcion a
    CodeURJC</h3>
188                     <p class="text-muted text-center mb-4">Completa el
    formulario para reservar tu plaza en los talleres.</p>
189 
190                     <form>
191                         <div class="mb-3">
192                             <label for="nombreCompleto" class="form-
    label fw-semibold">Nombre y Apellidos:</label>
193                             <div class="input-group">
194                                 <span class="input-group-text"><i class
    ="bi bi-person"></i></span>
195                                 <input type="text" class="form-control"
     id="nombreCompleto" placeholder="David Sebastian" required>
196                             </div>
197                         </div>
198 
199                         <div class="mb-3">
200                             <label for="emailAlu" class="form-label fw-
    semibold">Correo Institucional URJC:</label>
201                             <div class="input-group">
202                                 <span class="input-group-text"><i class
    ="bi bi-envelope"></i></span>
203                                 <input type="email" class="form-control
    " id="emailAlu" placeholder="d.sticea@alumnos.urjc.es" required>
204                             </div>
205                         </div>
206 
207                         <div class="mb-3">
208                             <label for="tallerSel" class="form-label fw
    -semibold">Seleccionar Taller:</label>
209                             <select class="form-select" id="tallerSel"
    required>
210                                 <option value="" selected disabled>
    Elige una opcion...</option>
211                                 <option value="1">TL-101: Maquetacion
    con Grid</option>
212                                 <option value="2">TL-102: Bootstrap 5
    Practico</option>
213                             </select>
214                         </div>
215 
216                         <div class="form-check mb-4">
217                             <input class="form-check-input" type="
    checkbox" id="aceptoTerminos" required>
218                             <label class="form-check-label text-
    secondary" for="aceptoTerminos">
219                                 Acepto la politica de tratamiento de
    datos personales de la URJC.
220                             </label>
221                         </div>
222 
```

```
223                         <button type="submit" class="btn btn-primary w
    -100 py-2 fw-semibold">Confirmar Matricula</button>
224                     </form>
225                 </div>
226             </div>
227         </div>
228     </section>
229
230     <!-- 6. Pie de Pagina Corporativo -->
231     <footer class="bg-dark text-white py-4 mt-5">
232         <div class="container text-center">
233             <p class="mb-1">&copy; 2026 CodeURJC - Escuela Tecnica
    Superior de Ingenieria Informatica.</p>
234             <p class="text-secondary small mb-0">Desarrollado para la
    materia Fundamentos de la Web.</p>
235         </div>
236     </footer>
237
238     <!-- Bootstrap 5 Bundle JS -->
239     <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/
    bootstrap.bundle.min.js"></script>
240 </body>
241 </html>
```

Listing 2.16: codeurjc_portal.html: Solución completa al Ejercicio 14 de Bootstrap.

CAPÍTULO 3

Tecnologías del Servidor Web: JavaScript, Node.js, Express y MongoDB

En este capítulo se aborda de manera exhaustiva el desarrollo del lado del servidor (backend) en la web moderna. Partiendo de los fundamentos del lenguaje JavaScript, sus runtimes (con especial énfasis en Node.js y el motor V8) y sus mecanismos avanzados de estructuras de datos y modularidad (ESM y CommonJS), se transita hacia la creación de aplicaciones web estructuradas mediante el framework Express. Se profundiza en la gestión del ciclo de petición-respuesta HTTP, el renderizado de vistas con el motor de plantillas Mustache, la subida segura de archivos multipartes con Multer, y el manejo de persistencia mediante bases de datos documentales con MongoDB y su driver oficial para Node.js. El contenido integra la totalidad de las diapositivas de la asignatura enriquecidas con las justificaciones arquitectónicas, modelos de memoria y consideraciones de seguridad imprescindibles para un ingeniero de software.

3.1 Introducción a JavaScript y Runtimes (3.1.1)

3.1.1 Naturaleza y Evolución de JavaScript

JavaScript nació en 1995 de la mano de Brendan Eich en Netscape Communications, bajo el nombre original de **Mocha**, posteriormente rebautizado como **LiveScript** y finalmente denominado **JavaScript**. Dicha denominación comercial respondió a una estrategia de marketing deliberada: en aquel momento Java experimentaba un auge extraordinario en la industria, por lo que se aprovechó la notoriedad del nombre pese a tratarse de dos tecnologías con filosofías, paradigmas y modelos de ejecución completamente independientes.

▲ ¡Atención / Ingeniería!: JavaScript no es Java

A pesar de compartir similitudes superficiales en la sintaxis de control de flujo heredada del lenguaje C (como las llaves `{}`, bucles `for` y condicionales `if`), Java y JavaScript son diametralmente opuestos:

- **Java:** Lenguaje de tipado estático y fuerte, fuertemente orientado a clases basadas en clases, compilado a bytecode intermedio que se ejecuta sobre la Máquina Virtual de Java (JVM).

- **JavaScript:** Lenguaje de tipado dinámico y débil, multiparadigma, orientado a objetos basado originalmente en prototipos, ejecutado por runtimes que emplean compilación Just-In-Time (JIT) sobre el código fuente original.

El estándar formal que rige la sintaxis y evolución de JavaScript es **ECMAScript**, administrado por la organización internacional **ECMA** a través del comité técnico **TC39** (*Technical Committee 39*), a diferencia de los estándares de marcado y estilo (HTML y CSS) que son regulados por el **W3C**. En los primeros años del desarrollo web, los distintos fabricantes de navegadores implementaban dialectos con divergencias notables (como JScript en Internet Explorer), lo que causaba graves problemas de interoperabilidad. En la actualidad, la adopción rigurosa del estándar ECMAScript (a partir de la trascendental especificación ECMAScript 2015 o ES6) garantiza una compatibilidad homogénea entre todos los motores modernos.

Las propiedades esenciales que definen a JavaScript como lenguaje son:

1. **Lenguaje de Scripting (Interpretado/JIT):** No requiere de un paso de compilación manual previo para generar binarios ejecutables. El runtime analiza el código fuente, genera un árbol sintáctico abstracto (AST) y lo compila dinámicamente en tiempo de ejecución.
2. **Tipado Dinámico:** Los tipos están asociados a los valores en memoria y no a las variables. Una misma variable puede albergar valores de distinta naturaleza a lo largo de su ciclo de vida.
3. **Funciones de Primer Orden:** Las funciones son tratadas como cualquier otro valor: pueden asignarse a variables, pasarse como argumentos a otras rutinas y devolverse como resultado desde otras funciones, posibilitando el paradigma funcional.
4. **Orientación a Objetos Prototípica y Basada en Clases:** Permite la creación y mutación dinámica de objetos en tiempo de ejecución, extendida con la sintaxis de clases formales introducida en ES6.

3.1.2 JavaScript en el Navegador vs Fuera del Navegador

Durante más de una década, JavaScript existió de forma casi exclusiva confinado dentro del entorno del navegador web (*browser environment*), donde su cometido principal consistía en aportar interactividad mediante la manipulación del Document Object Model (DOM), responder a eventos de la interfaz de usuario y orquestar comunicaciones asíncronas con el servidor sin recargar la página (AJAX / Fetch API).

▶ Concepto Clave: Runtime de JavaScript

Un **runtime** es el entorno de ejecución completo que engloba un motor de JavaScript (encargado de parsear y ejecutar el código) junto con un conjunto de interfaces de programación de aplicaciones (APIs) provistas por el host y un bucle de eventos (*Event Loop*) que gestiona las tareas asíncronas y la concurrencia.

En 2009, Ryan Dahl presentó **Node.js**, un proyecto que extrajo el motor **V8** (desarrollado por Google para el navegador Chrome) y lo combinó con una biblioteca en C llamada **libuv** para dotar a JavaScript de acceso directo a los recursos del sistema operativo: sistema de archivos (*filesystem*), conexiones de red TCP/UDP, procesos del sistema y memoria binaria nativa (*buffers*).

Figura 3.1: Divergencia arquitectónica entre el runtime de navegador y el runtime de servidor Node.js.

Aunque Node.js es el runtime fuera del navegador más extendido en la industria, existen alternativas modernas notables:

- **Deno:** Creado también por Ryan Dahl con un enfoque reforzado en seguridad (acceso denegado por defecto a red y disco), soporte nativo de TypeScript y adherencia estricta a los estándares Web API.
- **Bun:** Diseñado desde cero en lenguaje Zig, enfocado en maximizar el rendimiento de arranque y ejecución, integrando empaquetador, transpilador y gestor de paquetes ultrarápido.

3.1.3 Arquitectura, Rendimiento y Concurrencia en Node.js

Desde la perspectiva de la ingeniería de software, la ventaja competitiva de Node.js no radica en la velocidad de cálculo matemático puro de JavaScript (donde lenguajes compilados con tipado estático como C, Rust o Go resultan superiores), sino en su **modelo de entrada/salida no bloqueante y dirigido por eventos** (*Non-blocking Event-Driven I/O*).

- **Modelo tradicional basado en hilos (Thread-per-request):** Servidores como Apache HTTP Server o contenedores tradicionales de servlets en Java asignan un hilo del sistema operativo a cada cliente conectado. Mientras el hilo espera una consulta a la base de datos o una lectura en disco, permanece inactivo (bloqueado), consumiendo recursos de memoria de pila (alrededor de 1–2 MB por hilo) y sobrecargando al planificador del sistema operativo con continuos cambios de contexto (*context switches*).
- **Modelo de Node.js (Single Thread con Event Loop):** Un único hilo principal de ejecución atiende todas las peticiones entrantes. Cuando se requiere una operación de E/S costosa (acceso a disco, consulta de red), Node delega la tarea a los hilos de soporte de bajo nivel de `libuv` o al propio núcleo del sistema operativo. El hilo principal queda libre de inmediato para procesar otras peticiones. Una vez finalizada la operación de E/S, se encola una función de retorno (*callback* o resolución de promesa) que el **Event Loop** ejecuta cuando la pila de llamadas queda vacía.

► Concepto Clave: Ventajas en Arquitecturas Modernas

1. **Alta concurrencia en E/S:** Capacidad de mantener decenas de miles de conexiones concurrentes abiertas con un consumo de memoria mínimo.
2. **Tiempo de arranque casi instantáneo:** Característica crítica en arquitecturas *Serverless* (como AWS Lambda o Google Cloud Run), donde las funciones deben

instanciarse bajo demanda sin incurrir en latencias excesivas de inicio en frío (*cold starts*).

3.1.4 Versiones de Node.js y Entorno de Desarrollo

El proyecto Node.js sigue un calendario de versiones semestral:

- **Versiones Current (Actuales):** Incorporan las funcionalidades de vanguardia de ECMAScript y APIs experimentales. Tienen un ciclo de vida corto y no están recomendadas para despliegues empresariales en producción.
- **Versiones LTS (Long Term Support):** Cada versión par (v18, v20, v22) entra en fase LTS activa y recibe soporte de seguridad y correcciones de errores críticas durante 30 meses. Es el estándar indiscutible para entornos corporativos y de producción.

Para la construcción y depuración de proyectos utilizaremos el editor **Visual Studio Code (VS Code)**, el cual incluye integración nativa con el depurador de Node.js, terminal integrado y gestión de puntos de ruptura (*breakpoints*) interactivos para inspeccionar el estado de la pila y variables locales en tiempo de ejecución.

★ Ejercicio Resuelto: 1

Hola Mundo en Navegador y Node.js **Enunciado:** Construir una solución mínima que demuestre la ejecución de código JavaScript en ambos entornos:

1. En el navegador mediante un documento HTML `index.html` que enlace un script externo `app.js`.
2. En el servidor ejecutando directamente el script mediante el comando de Node.js.

Solución y Análisis:
Para el entorno del navegador, creamos el archivo de script:

```
console.log("Hello World!");
```

Listing 3.1: app.js para navegador y servidor

Y el documento HTML que lo referencia:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejercicio 1 - Hello World</title>
    <!-- El navegador descarga y ejecuta el script al analizar la
    cabecera -->
    <script src="app.js"></script>
</head>
<body>
    <h1>Abra la Consola de Desarrollador (F12)</h1>
</body>
</html>
```

Listing 3.2: index.html para ejecución en navegador

Al abrir `index.html` en Google Chrome y desplegar las herramientas de desarrollador (*DevTools* → pestaña *Console*), observamos la salida `'Hello World!'`.
Para el entorno Node.js, no es necesario ningún documento HTML ni entorno gráfico. Basta con abrir una terminal en el directorio del archivo y ejecutar:

```
$ node app.js
```

```
2 Hello World!
```

Listing 3.3: Ejecución directa en consola con Node.js

En este segundo caso, el proceso arranca, inicializa el motor V8, compila `app.js`, imprime la cadena en la salida estándar (`stdout`) del sistema y finaliza el proceso de forma limpia.

3.2 El Lenguaje JavaScript (3.1.2)

3.2.1 Comparativa de Paradigmas: Java vs JavaScript

Para un ingeniero de software habituado a Java, la transición hacia JavaScript requiere interiorizar diferencias fundamentales en el sistema de tipos y el modelo de ejecución:

Dimensión	Java	JavaScript
Sistema de Tipos	Estático y Fuerte. Los tipos se declaran en código y se validan en tiempo de compilación.	Dinámico y Débil. Los tipos se infieren en tiempo de ejecución y se permiten conversiones automáticas implícitas.
Compilación	Código fuente → Bytecode (`.class`) → Ejecutado por JVM.	Código fuente analizado directamente por el runtime; compilación JIT transparente a máquina.
Tipos Primitivos vs Objetos	Coexisten tipos primitivos (`int`, `double`) con objetos envoltura (`Integer`).	Todos los valores se comportan conceptualmente como objetos con propiedades y métodos.
Modelo Orientado a Objetos	Basado estrictamente en clases cerradas. Las clases definen la estructura fija de los objetos.	Basado en prototipos. En tiempo de ejecución se pueden añadir, modificar o borrar propiedades de cualquier objeto.
Funciones	Siempre vinculadas a una clase (métodos de instancia o estáticos). Lambdas son interfaces funcionales.	Ciudadanos de primer orden independientes (*first-class citizens*). Pueden existir aisladas.
Manejo de Memoria	Recolector de basura (Garbage Collector) sobre la memoria Heap de la JVM.	Recolector de basura automático sobre el Heap del motor V8 (marcado y barrido generacional).

Tabla 3.1: Comparativa técnica de fundamentos entre Java y JavaScript.

3.2.2 Sintaxis Básica, Declaración de Variables y Tipos

En JavaScript, las instrucciones pueden terminar opcionalmente en punto y coma (`;`). El intérprete dispone de un mecanismo denominado *Automatic Semicolon Insertion* (ASI) que inserta puntos y coma automáticamente en los saltos de línea; sin embargo, en ingeniería de software profesional se recomienda explícitamente escribir los puntos y coma para prevenir comportamientos anómalos en retornos de función multilínea.

Declaración: `let`, `const` y la problemática de `var`

En el estándar original de JavaScript se utilizaba la palabra reservada `var`. En el desarrollo moderno, **`var` está formalmente desaconsejada**:

- `var` tiene ámbito de función (*function scope*) e ignora los bloques delimitados por llaves (como `if` o `for`). Además, sufre de *hoisting* (elevación implícita de la declaración a la

cabecera del ámbito con valor `undefined`).

- `let` introduce **ámbito de bloque** (*block scope*), garantizando que la variable solo sea accesible dentro del bloque `{ }` donde fue declarada, exactamente igual que en Java o C++.
- `const` declara una referencia inmutable de bloque. La variable debe inicializarse obligatoriamente en su declaración y no puede reasignarse.

▲ ¡Atención / Ingeniería!: Mutabilidad de objetos con `const`

Declarar un array u objeto con `const` impide reasignar la variable a otro objeto en memoria, pero **no hace inmutable su contenido interno**. Las propiedades del objeto o elementos del array pueden ser modificados libremente:

```
1 const usuario = { nombre: "Ana", edad: 25 };
2 usuario.edad = 26; // TOTALMENTE VALIDO (mutacion interna)
3 // usuario = { nombre: "Carlos" }; // ERROR: Assignment to constant
      variable.
```

Valores no definidos y nulidad

JavaScript distingue claramente entre dos conceptos que en otros lenguajes suelen colapsar:

- `undefined`: Representa la ausencia de valor por defecto. Es el valor que adquiere cualquier variable declarada que aún no ha sido inicializada, o el retorno de funciones sin sentencia `return` explícita.
- `null`: Representa la ausencia intencionada y explícita de cualquier objeto o referencia. Se asigna de manera deliberada por el programador.

Si se intenta acceder a una variable que nunca fue declarada, el motor lanza una excepción `ReferenceError`. No obstante, si se asigna un valor a una variable no declarada sin usar el modo estricto (*strict mode*), JavaScript creará accidentalmente una variable global, lo que constituye una fuente habitual de errores críticos.

Tipos de datos básicos y Template Strings

Los tipos de datos primitivos en JavaScript son:

- **Number:** Representa tanto números enteros como números reales de coma flotante utilizando el formato IEEE 754 de 64 bits (doble precisión). La división entre números enteros que no sea exacta resulta siempre en un número real (`5 / 2 === 2.5`). Dispone de valores especiales como `Infinity`, `-Infinity` y `NaN` (*Not a Number*, generado por operaciones aritméticas inválidas como `0 / 0` o parseos fallidos).
- **Boolean:** Valores booleanos estrictos: `true` o `false`.
- **String:** Secuencia inmutable de caracteres UTF-16. Se puede delimitar mediante comillas simples (`'...'`) o dobles (`"..."`). No existe un tipo de dato `char` dedicado; un carácter individual es simplemente un string de longitud 1.
- **Template Literals (Template Strings):** Cadenas delimitadas por comillas invertidas (`` `...` ``). Ofrecen soporte nativo para interpolación de expresiones mediante la sintaxis `${expresion}` y permiten cadenas multilínea limpias sin requerir concatenación con `+` o caracteres de escape `\n`:

```
let usuario = "Maria";
let intentos = 3;
console.log(`El usuario ${usuario} ha consumido ${intentos} intentos.`);
```

3.2.3 Operadores, Coerción de Tipos e Igualdad Estricta

Uno de los aspectos más críticos en ingeniería de software con JavaScript es comprender la diferencia entre la **igualdad abstracta o débil** (==) y la **igualdad estricta** (===).

▶ Concepto Clave: Igualdad Débil vs Igualdad Estricta

- **Operador == (Igualdad Débil con Coerción):** Si los dos operandos son de distinto tipo, JavaScript intenta convertir implícitamente uno de ellos a un tipo común antes de comparar. Esto conduce a anomalías antiintuitivas:
  - `"" == 0` evalúa a `true` (cadena vacía se coacciona a 0).
  - `0 == false` evalúa a `true`.
  - `'0' == false` evalúa a `true`.
  - `null == undefined` evalúa a `true`.
- **Operador === (Igualdad Estricta sin Coerción):** Devuelve `true` única y exclusivamente si ambos operandos son del mismo tipo y poseen exactamente el mismo valor.

▲ ¡Atención / Ingeniería!: Regla de oro de ingeniería

En código de producción profesional, se debe utilizar **siempre el operador de igualdad estricta (===) y de desigualdad estricta (!==)**, eliminando cualquier riesgo de bugs silenciosos debidos a la coerción implícita de tipos.

En cuanto a la comparación de referencias:

- Dos cadenas de caracteres (`String`) son comparadas por su **contenido lexicográfico**: si poseen los mismos caracteres en el mismo orden, === evalúa a `true` (análogo al método `.equals()` de Java).
- Dos objetos o arrays son comparados por su **identidad de referencia**: === solo devuelve `true` si ambas variables apuntan a la misma dirección física de memoria en el heap (análogo al operador == en Java para referencias).

3.2.4 Arrays en JavaScript

Un array en JavaScript es un objeto especializado que mantiene una colección indexada con base 0. A diferencia de los arrays estáticos de Java (de tamaño inmutable y tipo homogéneo), los arrays en JavaScript:

1. Son completamente dinámicos: crecen y decrecen en tiempo de ejecución sin necesidad de dimensionamiento previo.
2. Son heterogéneos: pueden almacenar valores de diferentes tipos de datos mezclados dentro de la misma instancia (`['texto', 42, true, { id: 1 }]`).
3. Poseen la propiedad `length`: dicha propiedad no es de solo lectura; modificar manualmente `array.length` trunca o expande el array.

Acceso fuera de rango y arrays dispersos

Si se intenta leer una posición más allá del rango actual del array (`array[99]`), JavaScript no lanza ninguna excepción de desbordamiento de índice (como ocurriría con `ArrayIndexOutOfBoundsException` en Java), sino que devuelve limpiamente el valor `undefined`.

Por el contrario, si se asigna un valor a una posición más allá de los límites actuales:

```
let numeros = [10, 20];
numeros[4] = 50;
// El array pasa a ser: [10, 20, undefined, undefined, 50]
// Su propiedad length se actualiza automaticamente a 5
```

Esto genera un **array disperso** (*sparse array*), donde las posiciones intermedias no inicializadas contienen huecos (*empty slots*) que devuelven `undefined` al ser leídas.

Operaciones de mutación sobre Arrays

- `push(elem1, ...)`: Añade uno o más elementos al final del array e incrementa su longitud, devolviendo la nueva longitud. Complejidad $O(1)$ amortizada.
- `pop()`: Extrae y devuelve el último elemento del array, decrementando `length` en 1. Complejidad $O(1)$.
- `shift()`: Extrae y devuelve el primer elemento (índice 0), desplazando todos los elementos restantes a la izquierda. Complejidad $O(n)$.
- `unshift(elem1, ...)`: Inserta elementos al comienzo del array desplazando los existentes a la derecha. Complejidad $O(n)$.
- Operador `delete array[i]`: Elimina el valor del índice especificado pero **no redimensiona el array**; deja un hueco con valor `undefined`.
- Método `splice(start, deleteCount, item1, item2, ...)`: Permite modificar el contenido de un array eliminando elementos existentes y/o agregando nuevos elementos en cualquier posición intermedia sin dejar huecos:

```
let frutas = ['manzana', 'pera', 'platano', 'uva'];
// Borrar 1 elemento a partir del indice 1 ('pera')
frutas.splice(1, 1); // frutas: ['manzana', 'platano', 'uva']

// Insertar 'naranja' en el indice 1 sin borrar ninguno
frutas.splice(1, 0, 'naranja'); // frutas: ['manzana', 'naranja', '
    platano', 'uva']
```

3.2.5 Sentencias de Control de Flujo y Evaluación Booleana

Las estructuras condicionales (`if`, `else if`, `else`) y de bucle (`for`, `while`, `do...while`, `switch`) comparten la sintaxis familiar de los lenguajes derivados de C. Sin embargo, en JavaScript la condición evaluada en un `if` no requiere ser estrictamente un valor booleano: se evalúa la veracidad intrínseca (*truthiness*) del valor.

► Concepto Clave: Valores Falsy en JavaScript

Existen exactamente **seis valores falsy** que se evalúan automáticamente a `false` en contextos booleanos:

1. `false`

2. `0` (y `-0`, `0n`)
3. `""` (cadena de texto vacía)
4. `null`
5. `undefined`
6. `NaN`

Cualquier otro valor en el lenguaje (incluyendo cadenas no vacías como `'0'`, arrays vacíos `[]` y objetos vacíos `{}`) se evalúa como **truthy** (`true`).

Esta propiedad permite escribir expresiones idiomáticas concisas para validar la presencia de valores o cadenas:

```
let nombreUsuario = req.body.nombre;
if (!nombreUsuario) {
    // Se ejecuta si nombreUsuario es undefined, null o cadena vacia ""
    nombreUsuario = "Anonimo";
}
```

3.2.6 Funciones, Ámbitos, Clausuras y Funciones Flecha

Las funciones constituyen el pilar fundamental sobre el que se articula la arquitectura de JavaScript.

Declaración de Funciones y Funciones Anónimas

Una función puede definirse mediante una declaración tradicional con nombre o asignando una función anónima a una variable:

```
// Declaracion formal nombrada (sufre hoisting)
function multiplicar(a, b) {
    return a * b;
}

// Expresion de funcion anonima asignada a constante
const sumar = function(a, b) {
    return a + b;
};
```

Al ser ciudadanos de primer orden, las funciones pueden pasarse como argumentos a otras funciones (patrón **Callback**) o retornarse desde ellas:

```
function procesarNumeros(lista, criterio) {
    for (let i = 0; i < lista.length; i++) {
        if (criterio(lista[i])) {
            console.log(lista[i]);
        }
    }
}

function esPar(n) { return n % 2 === 0; }
procesarNumeros([1, 2, 3, 4, 5, 6], esPar); // Imprime 2, 4, 6
```

Parámetros Opcionales, por Defecto y Rest

En JavaScript, el número de argumentos reales pasados en la invocación no necesita coincidir con el número de parámetros formales declarados en la cabecera:

- Si se pasan **menos argumentos**, los parámetros omitidos toman automáticamente el valor `undefined` (sin provocar error de compilación ni de ejecución).
- Si se pasan **más argumentos**, los argumentos sobrantes se ignoran de forma transparente.

A partir de ES6 se pueden definir valores por defecto y agrupar parámetros variables mediante el operador Rest (`...`):

```
// Parametro con valor por defecto
function formatearTexto(texto, separador = ",") {
    return texto.split(" ").join(separador);
}

// Parametro Rest: agrupa los argumentos restantes en un Array real
function calcularSumaTotal(multiplicador, ...valores) {
    let suma = 0;
    for (let v of valores) { suma += v; }
    return suma * multiplicador;
}
```

Ámbito Léxico y Clausuras (Closures)

El ámbito de una función en JavaScript se determina estáticamente en el momento en que dicha función se escribe en el código fuente (**ámbito léxico**), y no en el lugar donde se invoca.

> **▶ Concepto Clave: Clausura (Closure)**
>
> Una **clausura** es la combinación de una función junto con el entorno léxico en el cual dicha función fue declarada. Esto permite a la función recordar y mantener acceso a las variables externas de su función contenedora, incluso después de que la función contenedora haya finalizado su ejecución y retornado.

A diferencia de las expresiones lambda en Java (donde las variables capturadas del ámbito externo deben ser obligatoriamente `final` o efectivamente finales), en JavaScript la función retiene una referencia viva a la propia variable. Si la variable externa muta su valor, la función observará el nuevo valor actualizado:

```
let mensaje = "Hola";
function saludar() {
    console.log(mensaje);
}
saludar(); // Imprime "Hola"
mensaje = "Adios";
saludar(); // Imprime "Adios" (acceso vivo por referencia)
```

Funciones Flecha (Arrow Functions)

Introducidas en ES6, las funciones flecha proporcionan una notación extraordinariamente concisa para definir funciones anónimas:

```
// Sintaxis estandar con cuerpo de bloque
const duplicar = (x) => { return x * 2; };

// Retorno implicito (cuerpo de expresion sin llaves)
const duplicarConciso = x => x * 2;

// Invocacion directa en callbacks
let impares = [1, 2, 3, 4, 5].filter(num => num % 2 !== 0);
```

▲ ¡Atención / Ingeniería!: Diferencias críticas de las Funciones Flecha

Las funciones flecha no son un mero sustituto cosmético de las funciones regulares:

1. **No poseen su propio enlace a `this`:** Capturan el valor de `this` léxicamente del contexto circundante donde fueron creadas, resolviendo el clásico problema de pérdida de contexto en callbacks de métodos.
2. **No pueden utilizarse como constructores:** Invocar una arrow function con el operador `new` genera un `TypeError`.
3. **No disponen del objeto especial `arguments`.**

3.2.7 Programación Orientada a Objetos: Literales y Clases ES6

Objetos Literales Dinámicos

Un objeto en JavaScript es esencialmente una colección asociativa de pares clave-valor (propiedades) donde las claves son cadenas y los valores pueden ser datos o funciones (métodos). Se crean de manera directa mediante la sintaxis de llaves `{}`:

```
let empleado = {
    nombre: "Pepe",
    salario: 1800,
    // Metodo de instancia
    obtenerFicha: function() {
        // En JavaScript el uso de 'this' es OBLIGATORIO para acceder a propiedades
        return `Empleado: ${this.nombre}, Salario: ${this.salario} EUR`;
    }
};

// Acceso mediante notacion de punto
console.log(empleado.obtenerFicha());

// Mutabilidad dinamica: adicion y modificacion de propiedades en tiempo de ejecucion
empleado.telefono = "600123456";
empleado.calcularBono = function() {
    return this.salario * 0.10;
};
```

Esta enorme flexibilidad permite la construcción rápida de objetos individuales (*singletons*) y estructuras de datos para transferir información (*Data Transfer Objects* o DTOs).

Clases Formales en ES6

Para estructurar aplicaciones robustas con patrones de diseño orientados a objetos, ES6 introdujo la sintaxis formal de clases, la cual actúa como una capa de azúcar sintáctico sobre el sistema de prototipos subyacente de JavaScript:

```
class Figura {
    constructor(id, x, y) {
        this.id = id;
        this.x = x;
        this.y = y;
    }

    desplazar(dx, dy) {
        this.x += dx;
        this.y += dy;
```

```
    }

    toString() {
        return `Figura[${this.id}] en (${this.x}, ${this.y})`;
    }
}

// Herencia de clases mediante 'extends' y llamada al constructor padre con 'super()'
class Rectangulo extends Figura {
    constructor(id, x, y, ancho, alto) {
        super(id, x, y); // Invocacion obligatoria al constructor de Figura
        this.ancho = ancho;
        this.alto = alto;
    }

    calcularArea() {
        return this.ancho * this.alto;
    }

    // Sobrescritura de metodos (override)
    toString() {
        return `Rectangulo > ${super.toString()}, Area: ${this.calcularArea()}`;
    }

    // Metodos estaticos de factoria o utilidad
    static crearPorDefecto() {
        return new Rectangulo("default", 0, 0, 10, 10);
    }
}
```

A partir de ECMAScript 2022 se incorporó el soporte para campos y métodos privados reales mediante el prefijo de almohadilla `#`:

```
class CuentaBancaria {
    #saldo; // Campo privado real inaccesible desde el exterior

    constructor(saldoInicial) {
        this.#saldo = saldoInicial;
    }

    ingresar(cantidad) {
        if (cantidad > 0) this.#saldo += cantidad;
    }

    obtenerSaldo() {
        return this.#saldo;
    }
}
```

En JavaScript no existen interfaces explícitas (`interface`) como en Java. Dado que el lenguaje es de tipado dinámico, la compatibilidad se fundamenta en el principio de **Duck Typing**: si un objeto dispone de los métodos y propiedades que el cliente espera consumir, el código opera correctamente con independencia de su jerarquía de herencia.

3.2.8 Manejo de Excepciones y Clases de Error Personalizadas

El control de errores en JavaScript sigue el paradigma de bloques estructurados `try...catch...finally`. La sentencia `throw` permite elevar excepciones en cualquier instante:

```
try {
    if (divisor === 0) {
        throw new Error("Division por cero no permitida");
    }
    let resultado = dividendo / divisor;
} catch (error) {
    console.error("Error capturado:", error.message);
} finally {
    // Bloque de limpieza que se ejecuta SIEMPRE (se lance o no excepcion)
    cerrarRecursos();
}
```

Aunque técnicamente es posible lanzar cualquier valor con `throw` (incluso números o cadenas de texto como `throw 'Error fatal'`), en ingeniería de software se exige elevar siempre instancias de la clase nativa `Error` o derivadas de ella, ya que contienen la traza de la pila de llamadas (*stack trace*) indispensable para la depuración en producción.

Diseño de jerarquías de errores empresariales

En aplicaciones de servidor con Node.js y Express resulta imprescindible modelar errores de negocio enriquecidos con códigos de estado HTTP y banderas de operatividad:

```
// Clase base de error de aplicacion
export class AppError extends Error {
    constructor(mensaje, statusCode, esOperacional = true) {
        super(mensaje);
        this.name = this.constructor.name;
        this.statusCode = statusCode;
        this.esOperacional = esOperacional;
        // Captura limpia de la traza de pila en entornos V8/Node.js
        Error.captureStackTrace(this, this.constructor);
    }
}

// Error de recurso no encontrado (HTTP 404)
export class NotFoundError extends AppError {
    constructor(recurso = "Recurso") {
        super(`${recurso} no encontrado en el sistema`, 404);
    }
}

// Error de validacion de datos de entrada (HTTP 400)
export class ValidationError extends AppError {
    constructor(mensaje) {
        super(mensaje, 400);
    }
}
```

3.2.9 Ejercicios Resueltos del Bloque 3.1.2

★ Ejercicio Resuelto: 2

Clasificación de Pares e Impares en Array Bidimensional **Enunciado:** Implementar una función `separararParesImpares(numeros)` que reciba como parámetro un array unidimensional de enteros y retorne un array bidimensional de dos elementos:

- El primer elemento (índice 0) apuntará a un array que contendrá todos los números pares.

- El segundo elemento (índice 1) apuntará a un array que contendrá todos los números impares.

Demostrar su funcionamiento mediante varias llamadas con arrays de prueba.

Solución y Análisis:

```javascript
function separarParesImpares(numeros) {
    const pares = [];
    const impares = [];

    for (let i = 0; i < numeros.length; i++) {
        const num = numeros[i];
        if (num % 2 === 0) {
            pares.push(num);
        } else {
            impares.push(num);
        }
    }

    return [pares, impares];
}

// Pruebas de verificacion
const muestra1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const resultado1 = separarParesImpares(muestra1);
console.log("Pares:", resultado1[0]);     // [2, 4, 6, 8, 10]
console.log("Impares:", resultado1[1]);   // [1, 3, 5, 7, 9]

const muestra2 = [11, 13, 17];
console.log(separarParesImpares(muestra2)); // [[], [11, 13, 17]]
```

Listing 3.4: Solución del Ejercicio 2

★ Ejercicio Resuelto: 3

Eliminación de Ceros y Ordenación por Longitud de Filas **Enunciado:** Implementar una función `limpiarCerosYOrdenar(matriz)` que reciba un array bidimensional (matriz), elimine de cada fila todos los ceros presentes y ordene las filas resultantes de menor a mayor en función de su longitud, utilizando el algoritmo de ordenación de la burbuja (*Bubble Sort*).

Solución y Análisis:

```javascript
function limpiarCerosYOrdenar(matriz) {
    // 1. Crear una copia profunda de las filas eliminando ceros
    const matrizFiltrada = [];
    for (let i = 0; i < matriz.length; i++) {
        const filaLimpia = [];
        for (let j = 0; j < matriz[i].length; j++) {
            if (matriz[i][j] !== 0) {
                filaLimpia.push(matriz[i][j]);
            }
        }
        matrizFiltrada.push(filaLimpia);
    }

    // 2. Ordenacion por tamano de fila mediante Bubble Sort adaptado
    const n = matrizFiltrada.length;
    for (let i = 0; i < n - 1; i++) {
        for (let j = 0; j < n - i - 1; j++) {
```

```
            if (matrizFiltrada[j].length > matrizFiltrada[j + 1].length
    ) {
                // Intercambio de referencias de filas
                const temp = matrizFiltrada[j];
                matrizFiltrada[j] = matrizFiltrada[j + 1];
                matrizFiltrada[j + 1] = temp;
            }
        }
    }

    return matrizFiltrada;
}

// Demostracion
const datos = [
    [0, 5, 0, 8, 0, 2], // 3 elementos tras filtrar
    [1, 0, 0],          // 1 elemento tras filtrar
    [0, 0, 0, 0],       // 0 elementos tras filtrar
    [4, 7, 9, 3]        // 4 elementos tras filtrar
];

const resultado = limpiarCerosYOrdenar(datos);
console.log("Matriz procesada:", resultado);
// Salida esperada ordenada por longitud:
// [ [], [1], [5, 8, 2], [4, 7, 9, 3] ]
```

Listing 3.5: Solución del Ejercicio 3

★ Ejercicio Resuelto: 4

Filtrado Configurable con Callbacks y Arrow Functions **Enunciado:** Refactorizar el Ejercicio 3 para que el criterio de descarte de números no sea exclusivamente el cero, sino que sea configurable por el llamador mediante una función de predicado pasada como parámetro (`debeEliminar`). Demostrar el uso de *arrow functions* en las llamadas de prueba.

Solución y Análisis:

```
function filtrarYOrdenar(matriz, debeEliminar) {
    const filtrada = [];
    for (let fila of matriz) {
        const nuevaFila = [];
        for (let val of fila) {
            // Si el predicado no se cumple, conservamos el valor
            if (!debeEliminar(val)) {
                nuevaFila.push(val);
            }
        }
        filtrada.push(nuevaFila);
    }

    // Ordenacion utilizando el metodo nativo .sort con comparator
    filtrada.sort((a, b) => a.length - b.length);
    return filtrada;
}

const matrizPrueba = [
    [-5, 10, -2, 8],
    [15, -1, 30],
    [-8, -9, -10]
```

```
];

// Prueba A: Eliminar todos los numeros negativos
const sinNegativos = filtrarYOrdenar(matrizPrueba, n => n < 0);
console.log("Sin negativos:", sinNegativos);

// Prueba B: Eliminar numeros impares
const sinImpares = filtrarYOrdenar(matrizPrueba, n => n % 2 !== 0);
console.log("Sin impares:", sinImpares);
```

Listing 3.6: Solución del Ejercicio 4

★ Ejercicio Resuelto: 5

Modelado de Rectángulos y Análisis de Colecciones (5, 5b, 6, 7 y 8) **Enunciado:**

1. Diseñar una jerarquía de clases con una clase base abstracta simulada `Shape` y clases derivadas `Rectangle` (ancho, alto) y `Circle` (radio). Cada figura debe implementar `area()` y `perimeter()`.
2. Implementar una clase `ShapeAnalyzer` capaz de recibir una colección de figuras y calcular: suma total de áreas, suma de perímetros, área media, perímetro medio, y la media del ratio área/perímetro de todas las figuras.
3. Incorporar manejo robusto de excepciones (Ejercicio 8): impedir la instanciación directa de `Shape`, impedir dimensiones negativas o no numéricas (`isNaN`), y elevar errores personalizados.

Solución y Análisis:

```
// Error de dominio personalizado
class InvalidDimensionError extends Error {
    constructor(mensaje) {
        super(mensaje);
        this.name = "InvalidDimensionError";
    }
}

// Clase base abstracta
class Shape {
    constructor(color = "negro") {
        if (new.target === Shape) {
            throw new Error("No se puede instanciar directamente la
    clase abstracta Shape.");
        }
        this.color = color;
    }

    area() {
        throw new Error("El metodo area() debe ser implementado por la
    subclase.");
    }

    perimeter() {
        throw new Error("El metodo perimeter() debe ser implementado
    por la subclase.");
    }

    areaPerimeterRatio() {
        const perim = this.perimeter();
        if (perim === 0) return 0;
        return this.area() / perim;
```

```
    }
}

class Rectangle extends Shape {
    constructor(width, height, color = "azul") {
        super(color);
        this.#validarDimension(width, "ancho");
        this.#validarDimension(height, "alto");
        this.width = width;
        this.height = height;
    }

    #validarDimension(val, nombre) {
        if (typeof val !== "number" || isNaN(val) || val <= 0) {
            throw new InvalidDimensionError(`El valor de ${nombre} debe ser un numero positivo (> 0). Recibido: ${val}`);
        }
    }

    area() { return this.width * this.height; }
    perimeter() { return 2 * (this.width + this.height); }
}

class Circle extends Shape {
    constructor(radius, color = "rojo") {
        super(color);
        if (typeof radius !== "number" || isNaN(radius) || radius <= 0) {
            throw new InvalidDimensionError(`El radio debe ser un numero positivo. Recibido: ${radius}`);
        }
        this.radius = radius;
    }

    area() { return Math.PI * this.radius * this.radius; }
    perimeter() { return 2 * Math.PI * this.radius; }
}

class ShapeAnalyzer {
    constructor(shapes = []) {
        this.shapes = shapes;
    }

    addShape(shape) {
        if (!(shape instanceof Shape)) {
            throw new TypeError("Solo se pueden agregar instancias derivadas de Shape.");
        }
        this.shapes.push(shape);
    }

    totalArea() {
        return this.shapes.reduce((acc, s) => acc + s.area(), 0);
    }

    totalPerimeter() {
        return this.shapes.reduce((acc, s) => acc + s.perimeter(), 0);
    }

    averageArea() {
        return this.shapes.length === 0 ? 0 : this.totalArea() / this.
```

```
        shapes.length;
87      }
88
89      averagePerimeter() {
90          return this.shapes.length === 0 ? 0 : this.totalPerimeter() /
        this.shapes.length;
91      }
92
93      averageRatio() {
94          if (this.shapes.length === 0) return 0;
95          const sumRatios = this.shapes.reduce((acc, s) => acc + s.
        areaPerimeterRatio(), 0);
96          return sumRatios / this.shapes.length;
97      }
98  }
99
100 // Verificacion y captura de excepciones
101 try {
102     const analyzer = new ShapeAnalyzer();
103     analyzer.addShape(new Rectangle(10, 5));
104     analyzer.addShape(new Rectangle(20, 10));
105     analyzer.addShape(new Circle(7));
106
107     console.log(`Area Total: ${analyzer.totalArea().toFixed(2)}`);
108     console.log(`Perimetro Total: ${analyzer.totalPerimeter().toFixed
        (2)}`);
109     console.log(`Area Media: ${analyzer.averageArea().toFixed(2)}`);
110     console.log(`Ratio Medio A/P: ${analyzer.averageRatio().toFixed(2)}
        `);
111
112     // Prueba de validacion
113     new Rectangle(-5, 10); // Provoca excepcion
114 } catch (error) {
115     console.error("Excepcion capturada con exito:", error.name, "-",
        error.message);
116 }
```

Listing 3.7: Jerarquía completa de Figuras con Manejo de Errores y Analizador

3.3 Estructuras de Datos en JavaScript (3.1.3)

3.3.1 Almacenamiento en Memoria y Complejidad Algorítmica

A partir de la especificación ECMAScript 2015 (ES6), la API estándar de JavaScript incorpora tres estructuras de datos fundamentales en memoria: **Array**, **Set** y **Map**. La elección juiciosa de la estructura adecuada determina la escalabilidad y el rendimiento computacional de las aplicaciones web en el servidor:

3.3.2 Conjuntos (`Set`)

Un `Set` es una colección de valores donde cada elemento puede aparecer una única vez.

- **Criterio de Igualdad:** Para determinar si un elemento es duplicado, el `Set` utiliza internamente una variante del operador de igualdad estricta (`===`). En consecuencia:
    - Las cadenas de texto con los mismos caracteres son consideradas idénticas (`'Madrid' === 'Madrid'`).

Propiedad	Array	Set	Map
Elementos	Secuencia indexada. Permite duplicados.	Colección de valores únicos. Sin duplicados.	Pares clave-valor. Claves únicas de cualquier tipo.
Acceso por Posición	Eficiente $O(1)$ mediante `arr[i]`.	No soportado.	No soportado.
Inserción	Final: $O(1)$ amortizado (`push`). Inicio/Centro: $O(n)$ (`unshift`, `splice`).	Eficiente $O(1)$ promedio (`add`).	Eficiente $O(1)$ promedio (`set`).
Eliminación	Final: $O(1)$ (`pop`). Centro: $O(n)$ (`splice`).	Eficiente $O(1)$ promedio (`delete`).	Eficiente $O(1)$ promedio (`delete`).
Búsqueda	Ineficiente $O(n)$ (`indexOf`, `find`).	Ultra-eficiente $O(1)$ promedio (`has`).	Ultra-eficiente $O(1)$ promedio (`get`, `has`).
Propiedad de Tamaño	`.length` (mutable).	`.size` (solo lectura).	`.size` (solo lectura).

Tabla 3.2: Comparativa de complejidad temporal y características de estructuras de datos en JavaScript.

  - Dos objetos con las mismas propiedades pero creados de forma independiente son referencias de memoria distintas y **no se consideran duplicados** (`{ id: 1 } !== { id: 1 }`).

- **Operaciones Básicas:**

```
const coleccion = new Set();
coleccion.add(10).add(20).add(10); // add es encadenable. El segundo 10 se ignora
console.log(coleccion.size); // 2
console.log(coleccion.has(20)); // true (busqueda O(1))
coleccion.delete(20); // Borra el elemento y devuelve true
```

- **Interoperabilidad con Arrays:**

```
// Eliminacion instantanea de duplicados en un Array:
const repetidos = ['Madrid', 'Sevilla', 'Madrid', 'Bilbao'];
const unicos = [...new Set(repetidos)]; // ['Madrid', 'Sevilla', 'Bilbao']
```

3.3.3 Mapas (`Map`)

Un `Map` asocia claves arbitrarias con valores. A diferencia de los objetos literales comunes `{}`, donde las claves se convierten forzosamente a cadenas de texto o símbolos:

1. En un `Map`, las claves pueden ser de cualquier tipo de dato, incluyendo objetos, funciones o tipos primitivos.
2. Respeta estrictamente el orden de inserción de las entradas al iterar.
3. Dispone de la propiedad directa `.size`.
4. Presenta un rendimiento notablemente superior en escenarios de inserción y borrado intensivo frecuente de pares clave-valor.

```
const coches = new Map();
coches.set('1234-ABC', { marca: 'Seat', modelo: 'Leon' });
```

```
coches.set('5678-XYZ', { marca: 'Toyota', modelo: 'Corolla' });

// Recuperacion por clave O(1)
const c = coches.get('1234-ABC');
console.log(coches.has('9999-ZZZ')); // false
coches.delete('1234-ABC');
```

3.3.4 Patrones de Recorrido e Iteración

Para recorrer colecciones en JavaScript disponemos de múltiples mecanismos:

- **Bucle `for...of`:** El estándar preferido para iterar sobre cualquier objeto iterable (Arrays, Sets, Maps, strings):

```
// Recorrido de Set
for (const ciudad of setCiudades) { console.log(ciudad); }

// Recorrido de Map mediante desestructuracion [clave, valor]
for (const [matricula, vehiculo] of coches) {
    console.log(matricula, "->", vehiculo.modelo);
}
```

- **Métodos Funcionales de Iteración:** `forEach(callback)` permite ejecutar una función por cada elemento.
- **Métodos de Transformación de Arrays:**
  - `.map(fn)`: Genera un nuevo array transformando cada elemento.
  - `.filter(predicado)`: Genera un nuevo array con los elementos que satisfacen la condición booleana.
  - `.reduce(acumuladorFn, valorInicial)`: Reduce todos los elementos a un único valor acumulado (sumatorios, medias, agrupaciones).

3.3.5 Ordenación y Búsqueda

En JavaScript, únicamente los **Arrays** son susceptibles de ser ordenados mediante el método nativo `.sort()`.

▲ ¡Atención / Ingeniería!: La trampa de `.sort()` sin comparador

Por defecto, `array.sort()` convierte todos los elementos a cadenas de texto y los ordena lexicográficamente según su valor Unicode. Esto produce errores críticos al ordenar números:

```
const numeros = [25, 8, 7, 100];
numeros.sort();
console.log(numeros); // [100, 25, 7, 8]  (¡ERROR! '100' va antes que
    '25' alfabeticamente)
```

Para ordenar números o comparar propiedades de objetos, es **estrictamente obligatorio suministrar una función de comparación** `(a, b) => valor`:

- Si devuelve un valor negativo (< 0), `a` se sitúa antes que `b`.
- Si devuelve cero (0), su orden relativo se mantiene.
- Si devuelve un valor positivo (> 0), `b` se sitúa antes que `a`.

```
// Ordenacion numerica ascendente correcta:
numeros.sort((a, b) => a - b); // [7, 8, 25, 100]
```

```
3
4 // Ordenacion de objetos por longitud de cadena:
5 const nombres = ['Alejandro', 'Ana', 'Constantino', 'Eva'];
6 nombres.sort((s1, s2) => s1.length - s2.length);
```

3.3.6 Ejercicios Resueltos del Bloque 3.1.3

★ Ejercicio Resuelto: 1

Función `countDistinct` con `Set` **Enunciado:** Implementar una función `countDistinct(provincias)` que reciba un array de cadenas con nombres de provincias (posiblemente repetidas) y devuelva el número exacto de provincias únicas sin duplicados. Probar la función con un array de ejemplo.

Solución y Análisis:

```
1  function countDistinct(provincias) {
2      // Al instanciar un Set con un Array, se eliminan todos los
       duplicados en O(n)
3      const conjuntoUnicos = new Set(provincias);
4      return conjuntoUnicos.size;
5  }
6
7  const lista = [
8      "Madrid", "Barcelona", "Madrid", "Valencia",
9      "Sevilla", "Barcelona", "Zaragoza", "Madrid"
10 ];
11
12 console.log("Total provincias en lista:", lista.length); // 8
13 console.log("Provincias sin repetir:", countDistinct(lista)); // 5
```

Listing 3.8: Solución del Ejercicio 1 de Estructuras de Datos

★ Ejercicio Resuelto: 2

Gestión Eficiente de Aeropuertos con `AirportManager` **Enunciado:** Implementar una clase `AirportManager` para almacenar aeropuertos (cada uno con nombre y ciudad). Se exige poder consultar la información completa de cualquier aeropuerto por su nombre con la máxima eficiencia temporal posible. Seleccionar la estructura de datos óptima y validar el funcionamiento.

Solución y Análisis: Para garantizar un tiempo de búsqueda $O(1)$ amortizado frente al nombre del aeropuerto, la estructura de datos obligatoria es un **`Map`**, donde la clave es el nombre del aeropuerto y el valor es el objeto completo:

```
1  class Airport {
2      constructor(name, city) {
3          this.name = name;
4          this.city = city;
5      }
6  }
7
8  class AirportManager {
9      constructor() {
10         // Mapa asociativo: Clave = name (String), Valor = Airport
11         this.airports = new Map();
12     }
```

```
13
14      addAirport(airport) {
15          this.airports.set(airport.name, airport);
16      }
17
18      getAirport(name) {
19          // Acceso directo O(1)
20          return this.airports.get(name);
21      }
22  }
23
24  // Pruebas
25  const manager = new AirportManager();
26  manager.addAirport(new Airport("Barajas", "Madrid"));
27  manager.addAirport(new Airport("El Prat", "Barcelona"));
28  manager.addAirport(new Airport("Manises", "Valencia"));
29
30  console.log(manager.getAirport("Barajas")); // Airport { name: 'Barajas
        ', city: 'Madrid' }
31  console.log(manager.getAirport("Inexistente")); // undefined
```

Listing 3.9: Solución del Ejercicio 2 de Estructuras de Datos

★ Ejercicio Resuelto: 3

Gestor de Vuelos `TripManager` con Índices Multi-Estructura **Enunciado:** Diseñar una clase `TripManager` para gestionar en memoria un conjunto de viajes de una aerolínea (origen, destino y duración). El gestor debe proveer las siguientes consultas en el mínimo tiempo de ejecución posible:

1. Devolver todos los viajes que parten de una ciudad origen dada.
2. Devolver todos los viajes que llegan a una ciudad destino dada.
3. Devolver la lista completa de todos los viajes.
4. Devolver la lista sin duplicados de todas las ciudades conectadas por algún viaje.

Solución y Análisis: Para lograr consultas en tiempo $O(1)$ sin realizar recorridos secuenciales $O(n)$, empleamos una técnica clásica de bases de datos: **indexación mediante múltiples estructuras en memoria**:

```
1   class Trip {
2       constructor(origin, destination, duration) {
3           this.origin = origin;
4           this.destination = destination;
5           this.duration = duration;
6       }
7   }
8
9   class TripManager {
10      constructor() {
11          this.allTrips = []; // Lista completa
12          this.tripsByOrigin = new Map(); // Indice: origen -> Array<Trip
        >
13          this.tripsByDestination = new Map(); // Indice: destino ->
        Array<Trip>
14          this.cities = new Set(); // Conjunto de ciudades unicas
15      }
16
17      addTrip(trip) {
18          this.allTrips.push(trip);
19
```

```
        // Actualizar indice de origenes
        if (!this.tripsByOrigin.has(trip.origin)) {
            this.tripsByOrigin.set(trip.origin, []);
        }
        this.tripsByOrigin.get(trip.origin).push(trip);

        // Actualizar indice de destinos
        if (!this.tripsByDestination.has(trip.destination)) {
            this.tripsByDestination.set(trip.destination, []);
        }
        this.tripsByDestination.get(trip.destination).push(trip);

        // Registrar ciudades en el conjunto
        this.cities.add(trip.origin);
        this.cities.add(trip.destination);
    }

    getTripsByOrigin(origin) {
        // Retorno inmediato en O(1)
        return this.tripsByOrigin.get(origin) || [];
    }

    getTripsByDestination(destination) {
        // Retorno inmediato en O(1)
        return this.tripsByDestination.get(destination) || [];
    }

    getAllTrips() {
        return this.allTrips;
    }

    getAllCities() {
        // Conversion de Set a Array
        return [...this.cities];
    }
}
```

Listing 3.10: Solución del Ejercicio 3 de Estructuras de Datos

3.4 Modularidad en JavaScript: ESM y Node.js (3.1.4 y 3.1.5)

3.4.1 Principios de Modularidad y ECMAScript Modules (ESM)

El desarrollo de sistemas de software a escala empresarial exige descomponer las aplicaciones en piezas cohesivas e independientes denominadas **módulos**. La modularización ofrece tres beneficios directos:

1. **Encapsulación y Ocultamiento de Información:** Cada archivo tiene su propio ámbito privado. Solo los identificadores declarados con `export` son visibles fuera del archivo.
2. **Mantenibilidad y Prevención de Colisiones de Nombres:** Se erradica por completo la contaminación del objeto global.
3. **Reutilización de Código:** Funcionalidades transversales pueden compartirse entre distintos subsistemas.

El estándar oficial de la industria se denomina **ECMAScript Modules (ESM)**.

Mecanismos de Exportación

- **Exportación por Defecto (`export default`):** Un módulo puede designar un único valor o función como exportación principal. Al importar, el receptor puede nombrarlo libremente sin llaves:

```
// En logger.js
export default function log(msg) { console.log(`[INFO] ${msg}`); }

// En app.js
import miLogger from './logger.js';
miLogger("Servidor iniciado");
```

- **Exportaciones Nombradas (`export const/function`):** Permite exportar múltiples entidades individuales desde el mismo archivo:

```
// En mathUtils.js
export const PI = 3.14159;
export function sumar(a, b) { return a + b; }
export class Calculadora { ... }

// Importacion selectiva desestructurada mediante llaves:
import { PI, sumar } from './mathUtils.js';

// Importacion de espacio de nombres completo (Namespace Import):
import * as MathUtils from './mathUtils.js';
console.log(MathUtils.sumar(2, 3));
```

ESM en el Navegador y en Node.js

- **En el Navegador:** Se debe indicar explícitamente el atributo `type='module'` en la etiqueta `<script>`:

```
<script type="module" src="app.js"></script>
```

> **▲ ¡Atención / Ingeniería!: Restricción de Seguridad en el Navegador**
>
> Los módulos en el navegador se rigen por la política de seguridad CORS. Cargar un módulo mediante el protocolo local de archivos (`file:///`) falla por seguridad. Es **estrictamente necesario servir el archivo mediante un servidor HTTP local** (`http://localhost:...`).

- **En Node.js:** Para activar el sistema ESM en un proyecto Node, se debe configurar el archivo `package.json` incorporando la clave `'type': 'module'`:

```
{
  "name": "mi-proyecto-web",
  "version": "1.0.0",
  "type": "module"
}
```

3.4.2 Módulos Nativos de Node.js y el Prefijo `node:`

Node.js proporciona de fábrica una rica biblioteca de módulos del sistema. A partir de versiones recientes, las mejores prácticas de ingeniería exigen anteponer el prefijo **`node:`** al importar módulos del núcleo, lo que evita colisiones con paquetes instalados desde npm y acelera la resolución interna del runtime:

Módulo del Núcleo	Funcionalidad y Responsabilidad en el Sistema
`node:fs` / `node:fs/promises`	Operaciones sobre el sistema de archivos: lectura, escritura, creación y borrado de directorios de forma asíncrona mediante promesas.
`node:path`	Construcción, normalización y resolución segura de rutas de archivos en sistemas operativos heterogéneos (manejo de separadores `/` en Linux/macOS vs `\` en Windows).
`node:http` / `node:https`	Creación de servidores y clientes HTTP/HTTPS de bajo nivel sobre sockets TCP.
`node:events`	Implementación del patrón Observer mediante la clase central `EventEmitter`.
`node:crypto`	Criptografía avanzada: funciones hash (SHA-256), cifrado simétrico/asimétrico, firmas digitales y generación de tokens criptográficos aleatorios seguros.
`node:process`	Control e información del proceso en ejecución: lectura de variables de entorno (`process.env`), gestión de señales POSIX (`SIGTERM`, `SIGINT`) y salida controlada (`process.exit`).

Tabla 3.3: Principales módulos nativos incluidos en el entorno de ejecución de Node.js.

Ejemplo de lectura asíncrona de archivos con soporte moderno de *Top-Level Await*:

```
import { readFile } from 'node:fs/promises';

try {
    const contenido = await readFile('./config.json', 'utf8');
    const configuracion = JSON.parse(contenido);
    console.log("Configuracion cargada:", configuracion);
} catch (error) {
    console.error("Error al leer el archivo:", error.message);
}
```

3.4.3 Gestión de Paquetes con npm y Coexistencia con CommonJS

El ecosistema de Node.js se articula en torno a **npm** (*Node Package Manager*), el registro público de software libre más grande del mundo.

- **`package.json`:** Archivo descriptor del proyecto que define metadatos, comandos ejecutables en la sección `'scripts'` y las dependencias de la aplicación.
- **Instalación de Dependencias:**
  - `npm install -save <paquete>`: Instala una biblioteca necesaria para la ejecución en producción (se registra en `dependencies`).
  - `npm install -save-dev <paquete>`: Instala herramientas utilizadas exclusivamente durante la fase de desarrollo, como `nodemon` para recarga en caliente o linters (se registra en `devDependencies`).
- **La carpeta `node_modules`:** Contiene el código fuente descargado de todas las dependencias directas e indirectas. **Nunca debe subirse al repositorio Git**; debe figurar obligatoriamente en el archivo `.gitignore`, ya que puede reproducirse en cualquier momento ejecutando `npm install` a partir de `package.json` y el archivo de bloqueo determinista `package-lock.json`.
- **Versionado Semántico (SemVer):** Los paquetes utilizan el estándar `MAJOR.MINOR.PATCH`. El prefijo `^1.2.3` permite actualizaciones de versiones menores y parches compatibles; el

prefijo `~1.2.3` permite únicamente parches de corrección.

Transición Histórica: CommonJS vs ESM

Con anterioridad a la estandarización de ESM, Node.js utilizaba su propio sistema modular propietario: **CommonJS** (basado en las funciones síncronas `require()` y `module.exports`). Ambos sistemas coexisten actualmente en el ecosistema:

Operación	CommonJS (Legado)	ESM (Estándar Moderno)
Importar módulo completo	`const fs = require('fs');`	`import fs from 'node:fs';`
Importar elemento con nombre	`const { readFile } = require('fs');`	`import { readFile } from 'node:fs';`
Exportar valor único por defecto	`module.exports = miFuncion;`	`export default miFuncion;`
Exportar múltiples elementos	`exports.suma = suma;`	`export const suma = ...;`

Tabla 3.4: Equivalencias sintácticas entre CommonJS y ES Modules.

3.4.4 Ejercicios Resueltos del Bloque 3.1.4 y 3.1.5

★ Ejercicio Resuelto: 1

Modularización de Funciones y Consumo con ESM **Enunciado:** Extraer la función de eliminar ceros desarrollada en ejercicios previos a un módulo independiente denominado `arrayCleaner.js`. Exportar dicha función como valor por defecto e importarla en `app.js` para procesar un array de prueba.
Solución y Análisis:
Creamos el módulo `arrayCleaner.js`:

```
export default function eliminarCeros(array) {
    return array.filter(elemento => elemento !== 0);
}
```

Listing 3.11: arrayCleaner.js

Y el punto de entrada de la aplicación `app.js`:

```
import eliminarCeros from './arrayCleaner.js';

const datosBrutos = [0, 10, 0, 25, 0, 0, 42, 99];
const datosLimpios = eliminarCeros(datosBrutos);

console.log("Original:", datosBrutos);
console.log("Filtrado:", datosLimpios); // [10, 25, 42, 99]
```

Listing 3.12: app.js con importacion ESM

★ Ejercicio Resuelto: 2

Uso de Bibliotecas Externas con npm: `lodash` **Enunciado:** Instalar la biblioteca de utilidades `lodash` en el proyecto e implementar un script que emplee el método `_.without()` para eliminar ceros y valores no deseados de un array.
Solución y Análisis:
Paso 1: Inicialización del proyecto e instalación de dependencias en consola:

```
$ npm init -y
$ npm install --save lodash
```

Paso 2: Código del script `app.js`:

```
import _ from 'lodash';

const numeros = [0, 4, 0, 9, 12, 0, 5, 0];

// _.without crea un array nuevo excluyendo todos los valores proporcionados
const sinCeros = _.without(numeros, 0);
console.log("Array sin ceros mediante lodash:", sinCeros); // [4, 9, 12, 5]
```

Listing 3.13: app.js utilizando lodash

3.5 Aplicaciones Web con Node.js y Express (3.2.1)

3.5.1 Arquitectura de Express y Pipeline de Middlewares

Para la construcción de servicios web robustos y mantenibles no se suele interactuar directamente con el módulo nativo `node:http`, sino que se recurre a frameworks de servidor. El estándar *de facto* indiscutible en el ecosistema Node.js es **Express**.

▶ Concepto Clave: Framework Express

Express es un framework web minimalista, rápido y flexible que proporciona un conjunto robusto de utilidades para:

- El enrutamiento de peticiones HTTP en base a verbos (`GET`, `POST`, etc.) y patrones de URL.
- La orquestación secuencial de funciones intermediarias (**Middlewares**).
- La integración transparente con motores de renderizado de plantillas HTML en servidor (SSR).

El núcleo arquitectónico de Express se fundamenta en la cadena de responsabilidad o **Pipeline de Middlewares**. Cada middleware es una función que tiene acceso al objeto de petición (`req`), al objeto de respuesta (`res`) y a la siguiente función middleware de la pila mediante la función `next()`:

3.5.2 Estructura Profesional de un Proyecto Express

En aplicaciones que siguen las buenas prácticas de ingeniería de software, la configuración general del servidor se desacopla estrictamente de la lógica de enrutamiento y los modelos de negocio. La estructura canónica del proyecto organiza los directorios de la siguiente forma:

```
proyecto-express/
|-- package.json          # Metadatos, dependencias y scripts de ejecucion
|-- public/               # Archivos estaticos servidos directamente por HTTP
|   |-- css/styles.css
|   |-- img/logo.png
|-- src/                  # Codigo fuente de la logica de backend
|   |-- app.js            # Configuracion e inicializacion de middlewares y servidor
|   |-- router.js         # Definicion de rutas y controladores HTTP
```

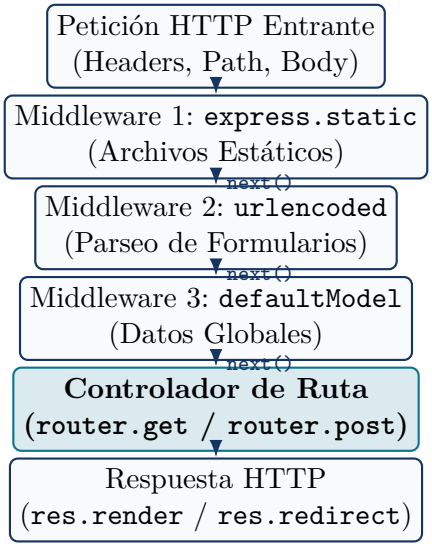


Figura 3.2: Flujo de procesamiento secuencial a través de la cadena de Middlewares en Express.

```
|   |-- board.js          # Logica del dominio y acceso a datos
|-- views/                # Plantillas dinamicas HTML (Mustache)
|   |-- index.html
|   |-- header.html
|   |-- footer.html
```

Inicialización y configuración en src/app.js

```
import express from 'express';
import mustacheExpress from 'mustache-express';
import router from './router.js';

const app = express();

// 1. Servicio de archivos estaticos desde el directorio /public
app.use(express.static('./public'));

// 2. Configuracion del motor de plantillas Mustache
app.set('view engine', 'html');
app.engine('html', mustacheExpress(), '.html');
app.set('views', './views');

// 3. Middleware para parsear cuerpos de peticion codificados como URL (
    Formularios POST)
app.use(express.urlencoded({ extended: true }));

// 4. Registro del router principal
app.use('/', router);

// 5. Apertura del puerto del servidor
const PORT = 3000;
app.listen(PORT, () => {
    console.log(`Servidor web escuchando en http://localhost:${PORT}/`);
});
```

Listing 3.14: src/app.js: Configuración integral de Express

3.5.3 Renderizado en Servidor (SSR) con Plantillas Mustache

En el desarrollo web del lado del servidor, concatenar cadenas de caracteres para construir código HTML dinámico resulta ineficiente, ilegible y altamente vulnerable a inyecciones de código malicioso. Por ello, se utilizan **motores de plantillas** (*template engines*). En este curso utilizaremos **Mustache** (mediante el paquete `mustache-express`), caracterizado por ser un motor libre de lógica arbitraria (*logic-less*).

Sintaxis Esencial de Mustache

- **Interpolación de Variables (`{{variable}}`):** Imprime el valor de la propiedad escapando automáticamente los caracteres HTML especiales para evitar ataques XSS (*Cross-Site Scripting*).
- **Secciones Condicionales e Iterativas (`{{#seccion}} ... {{/seccion}}`):**
  - Si la propiedad es un booleano con valor `true`, el bloque HTML se renderiza.
  - Si la propiedad es una lista o array, el bloque se itera por cada elemento. Dentro del bloque, el punto `{{.}}` representa el elemento actual en tipos primitivos, o se accede directamente a los nombres de los atributos si son objetos (`{{nombre}}`).
- **Secciones Invertidas (`{{^seccion}} ... {{/seccion}}`):** Se renderiza únicamente si la propiedad es `false`, `null`, `undefined` o un array vacío (actuando como el bloque `else` condicional).
- **Inclusión de Parciales (`{{>nombrePlantilla}}`):** Permite componer vistas reutilizando bloques compartidos como cabeceras (`header.html`) y pies de página (`footer.html`).

```
{{>header}}
<h1>Catalogo de Productos</h1>

{{#hayOfertas}}
    <p class="alerta">¡Hay descuentos especiales disponibles!</p>
{{/hayOfertas}}

<ul>
{{#productos}}
    <li><strong>{{nombre}}</strong> - Precio: {{precio}} EUR</li>
{{/productos}}
{{^productos}}
    <p>No se encontraron productos registrados en el sistema.</p>
{{/productos}}
</ul>

{{>footer}}
```

Listing 3.15: views/listado.html: Ejemplo de plantilla Mustache

3.5.4 Procesamiento de Formularios, Parámetros y el Patrón PRG

La comunicación de datos desde el navegador hacia el servidor Express se materializa mediante tres vías principales:

1. Formularios GET vs POST

- **Formularios GET:** Los datos de los campos de texto se codifican en la propia URL en forma de *Query String* (`/buscar?termino=laptop&pagina=2`). En Express, los valores

se leen inmediatamente desde el objeto **`req.query`** (`req.query.termino`). Está pensado para consultas seguras e idempotentes que no modifican el estado del servidor.

- **Formularios POST:** Los datos viajan encapsulados en el cuerpo (*body*) de la petición HTTP con la cabecera `Content-Type: application/x-www-form-urlencoded`. En Express, para que dichos campos sean procesados es imprescindible registrar el middleware `express.urlencoded({ extended: true })`, tras lo cual están accesibles en el objeto **`req.body`** (`req.body.nombre`).

2. Parámetros de Ruta (Path Parameters)

Cuando un dato forma parte integral de la identidad del recurso REST, se incluye como un segmento de la propia ruta precedido por dos puntos:

```
// Captura de parametros en la URL
router.get('/usuarios/:id/pedidos/:numPedido', (req, res) => {
    const idUsuario = req.params.id;
    const pedido = req.params.numPedido;
    res.send(`Consultando pedido ${pedido} del usuario ${idUsuario}`);
});
```

El Patrón Post-Redirect-Get (PRG)

Uno de los principios de diseño de mayor relevancia en ingeniería web es el patrón **Post-Redirect-Get**:

▲ ¡Atención / Ingeniería!: Por qué es fundamental el Patrón PRG

Cuando un usuario envía un formulario mediante el método `POST` (por ejemplo, para realizar un pago o insertar un registro) y el servidor responde renderizando directamente un HTML con código 200 OK, si el usuario pulsa la tecla F5 o el botón de recargar del navegador, **la petición POST completa volverá a enviarse**, provocando una inserción duplicada en la base de datos o un cobro repetido.
Para solucionar este defecto arquitectónico, el patrón PRG prescribe:

1. El cliente envía el formulario mediante `POST`.
2. El servidor procesa la mutación de datos.
3. En lugar de responder con HTML, el servidor emite una redirección HTTP 302 o 303 mediante `res.redirect('/ruta-destino')`.
4. El navegador recibe la orden de redirección y solicita automáticamente la nueva página mediante una petición inocua de tipo **`GET`**.

De este modo, si el usuario refresca la pantalla, solo refrescará la petición GET final sin duplicar la acción de inserción.

3.5.5 Ejercicios Resueltos del Bloque 3.2.1: Tablón de Anuncios

★ Ejercicio Resuelto: 4

Aplicación Web Completa del Tablón de Anuncios **Enunciado:** Desarrollar una aplicación web completa con Node.js y Express que gestione un Tablón de Anuncios en memoria con las siguientes especificaciones:

1. **Página Principal (/):** Lista todos los anuncios existentes (mostrando usuario y título) con un enlace para ver el detalle de cada uno y otro enlace para redactar un nuevo anuncio.

2. **Página de Formulario (`/new_post.html`):** Formulario que solicita usuario, título y cuerpo del mensaje.
3. **Controlador de Inserción (`POST /post/new`):** Recoge los datos del formulario, almacena el nuevo post asignándole un identificador único numérico incremental y muestra la vista de confirmación.
4. **Página de Detalle (`/post/:id`):** Muestra el usuario, título y texto completo del post, junto con un enlace para eliminarlo.
5. **Controlador de Eliminación (`/post/:id/delete`):** Elimina el post especificado y confirma la operación.

Solución y Análisis:

Comenzamos implementando la capa de gestión de datos en memoria en `src/board.js`, encapsulada mediante un `Map` para garantizar accesos $O(1)$ por identificador:

```
const posts = new Map();
let nextId = 1;

export function addPost(post) {
    const id = (nextId++).toString();
    post.id = id;
    posts.set(id, post);
    return post;
}

export function getPosts() {
    return Array.from(posts.values());
}

export function getPost(id) {
    return posts.get(id);
}

export function deletePost(id) {
    const post = posts.get(id);
    posts.delete(id);
    return post;
}
```

Listing 3.16: src/board.js: Gestión de anuncios en memoria

A continuación, implementamos el controlador de rutas en `src/router.js`:

```
import express from 'express';
import * as board from './board.js';

const router = express.Router();

// Listado de anuncios
router.get('/', (req, res) => {
    res.render('index', {
        posts: board.getPosts()
    });
});

// Guardar nuevo anuncio
router.post('/post/new', (req, res) => {
    const post = {
        user: req.body.user,
        title: req.body.title,
        text: req.body.text
```

```
    };
    board.addPost(post);
    res.render('saved_post', { id: post.id });
});

// Ver detalle de anuncio
router.get('/post/:id', (req, res) => {
    const post = board.getPost(req.params.id);
    if (!post) {
        return res.status(404).send("Anuncio no encontrado");
    }
    res.render('show_post', { post });
});

// Eliminar anuncio
router.get('/post/:id/delete', (req, res) => {
    board.deletePost(req.params.id);
    res.render('deleted_post');
});

export default router;
```

Listing 3.17: src/router.js: Enrutamiento del tablón de anuncios

Y las plantillas correspondientes en la carpeta `views/`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>Tablon de Anuncios</title></head>
<body>
    <h1>Tablon de Anuncios</h1>
    {{#posts}}
        <p><a href="/post/{{id}}">{{user}}: {{title}}</a></p>
    {{/posts}}
    {{^posts}}
        <p>No hay ningun anuncio publicado todavia.</p>
    {{/posts}}
    <hr>
    <a href="/new_post.html">Crear nuevo anuncio</a>
</body>
</html>
```

Listing 3.18: views/index.html: Listado de anuncios

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>Nuevo Anuncio</title></head>
<body>
    <h1>Publicar Anuncio</h1>
    <form action="/post/new" method="post">
        <p>Usuario: <input type="text" name="user" required></p>
        <p>Titulo: <input type="text" name="title" required></p>
        <p>Mensaje:<br><textarea name="text" rows="5" cols="40"
    required></textarea></p>
        <p><input type="submit" value="Guardar Anuncio"></p>
    </form>
</body>
</html>
```

Listing 3.19: public/new_post.html: Formulario HTML estático

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>{{post.title}}</title></head>
<body>
    <h1>{{post.title}}</h1>
    <p><strong>Autor:</strong> {{post.user}}</p>
    <p>{{post.text}}</p>
    <hr>
    <a href="/post/{{post.id}}/delete">Eliminar anuncio</a> |
    <a href="/">Volver al listado</a>
</body>
</html>
```

Listing 3.20: views/show_post.html: Detalle del anuncio

▲ ¡Atención / Ingeniería!: Advertencia de Seguridad: Vulnerabilidad CSRF

Tal como se señala en las diapositivas, eliminar un registro sensible mediante una petición `GET` en un enlace (`/post/:id/delete`) vulnera las buenas prácticas de seguridad y expone la aplicación a ataques de **Cross-Site Request Forgery (CSRF)**. Un sitio web malicioso podría incrustar una etiqueta `<img src='http://servidor-victima/post/1/delete'>` y provocar la eliminación involuntaria de anuncios si el usuario autenticado visita dicha página. En aplicaciones de producción, las acciones de borrado deben orquestarse siempre mediante formularios `POST` protegidos con tokens anti-CSRF.

3.6 Gestión y Subida de Archivos con Multer (3.2.2)

3.6.1 El Protocolo de Subida de Ficheros: `multipart/form-data`

El envío estándar de formularios mediante `application/x-www-form-urlencoded` no resulta adecuado para transmitir flujos binarios de información (como imágenes JPG o documentos PDF), ya que el coste de codificación sobrecarga sustancialmente el ancho de banda. Para subir archivos se utiliza el tipo de contenido **`multipart/form-data`**, donde el cuerpo de la petición se divide en secciones delimitadas por fronteras (*boundaries*), cada una con sus propias cabeceras y flujo binario.

Para habilitar este comportamiento en el cliente, el formulario HTML debe incorporar obligatoriamente el atributo `enctype`:

```
<form action="/upload_image" method="post" enctype="multipart/form-data">
    <input type="file" name="imagen" accept=".jpg, .jpeg, .png">
    <input type="submit" value="Subir Archivo">
</form>
```

3.6.2 Almacenamiento Seguro: ¿Por qué NUNCA en la carpeta `public/`?

Uno de los errores más graves cometidos por desarrolladores noveles consiste en almacenar los archivos subidos por los usuarios directamente dentro del directorio `public/` de archivos estáticos.

▲ ¡Atención / Ingeniería!: Riesgos de almacenar subidas en `public/`

1. **Mezcla indebida de código estático y dinámico:** Los recursos estáticos forman parte del código fuente de la aplicación (controlados por Git), mientras que las subidas de los usuarios son datos de producción volátiles.
2. **Permisos de solo lectura en contenedores:** En despliegues empresariales modernos con Docker o plataformas Cloud (AWS ECS, Kubernetes), el contenedor de la aplicación suele montarse con un sistema de archivos de solo lectura (*read-only root filesystem*), por lo que cualquier intento de escribir dentro de `public/` provocará un fallo de permisos catastrófico.
3. **Vulnerabilidad de Ejecución Remota de Código (RCE):** Si un usuario malintencionado logra subir un archivo ejecutable (como un script JavaScript o PHP) a la carpeta pública y el servidor lo sirve sin restricciones, el atacante podría solicitar la ejecución remota de dicho archivo en el host.

Por tanto, las subidas deben dirigirse **siempre a una carpeta externa dedicada (por ejemplo, `uploads/`)**, y el acceso a dichos archivos debe canalizarse a través de un controlador que verifique la autenticación y aplique medidas de seguridad.

3.6.3 Configuración de Multer y Servicio Seguro de Imágenes

Para gestionar peticiones `multipart/form-data` en Express se utiliza la biblioteca **`multer`**:

```
$ npm install --save multer
```

Multer intercepta la petición entrante, guarda el flujo binario en el disco asignándole un nombre de archivo pseudoaleatorio único (sin extensión, para prevenir sobreescrituras y ejecuciones no deseadas) y añade a la petición el objeto **`req.file`**:

```
import express from 'express';
import multer from 'multer';

const router = express.Router();
// Configuracion del directorio de destino para subidas
const upload = multer({ dest: 'uploads/' });

// Ruta de subida de imagen individual
router.post('/upload_image', upload.single('imagen'), (req, res) => {
    if (!req.file) {
        return res.status(400).send("No se ha seleccionado ningun archivo");
    }

    // req.file contiene metadatos criticos:
    // filename: nombre generado en el disco ('uploads/a83f98c...')
    // originalname: nombre original del archivo en el cliente
    // mimetype: tipo MIME ('image/jpeg')
    // size: tamano en bytes
    res.render('imagen_subida', { nombreArchivo: req.file.filename });
});

// Servicio seguro del archivo para descarga o visualizacion
router.get('/image', (req, res) => {
    const filename = req.query.filename;

    // PREVENCION CRITICA DE PATH TRAVERSAL:
    // Si el nombre contiene barras diagonales (\ o /), se rechaza la
    peticion
    if (/[\\/]/.test(filename)) {
        return res.status(400).send("Ataque Path Traversal detectado");
```

```
    }

    // Envio seguro del archivo como descarga o stream binario
    res.download('uploads/' + filename);
});
```

Listing 3.21: Integración y control seguro de subidas con Multer

3.6.4 Ejercicio Resuelto 5: Tablón de Anuncios con Imágenes

★ Ejercicio Resuelto: 5

Ampliación del Tablón con Soporte Multimedia y Carga de Demo **Enunciado:** Ampliar el Tablón de Anuncios (Ejercicio 4) incorporando las siguientes capacidades:

1. Permitir adjuntar opcionalmente una imagen al publicar un nuevo anuncio mediante Multer.
2. Servir la imagen en la vista de detalle mediante la ruta `/post/:id/image`.
3. Al eliminar un anuncio, borrar físicamente la imagen asociada del disco mediante el módulo `node:fs/promises`.
4. Implementar un script `load_data.js` que limpie el directorio `uploads/`, cargue anuncios de demostración desde un archivo JSON y copie las imágenes correspondientes desde `data/images/`.

Solución y Análisis:

Actualizamos el formulario `new_post.html` para admitir archivos:

```
<form action="/post/new" method="post" enctype="multipart/form-data">
    <p>Usuario: <input type="text" name="user" required></p>
    <p>Titulo: <input type="text" name="title" required></p>
    <p>Mensaje:<br><textarea name="text" rows="5" cols="40"></textarea>
    </p>
    <p>Imagen adjunta: <input type="file" name="image" accept=".jpg,.
    jpeg,.png"></p>
    <p><input type="submit" value="Publicar con Imagen"></p>
</form>
```

En el controlador `src/router.js`, adaptamos la ruta de inserción y de borrado:

```
import express from 'express';
import multer from 'multer';
import fs from 'node:fs/promises';
import * as board from './board.js';

const router = express.Router();
const UPLOADS_FOLDER = './uploads';
const upload = multer({ dest: UPLOADS_FOLDER });

// Insercion con Multer
router.post('/post/new', upload.single('image'), (req, res) => {
    const post = {
        user: req.body.user,
        title: req.body.title,
        text: req.body.text,
        // Si el usuario subio imagen, guardamos el nombre generado
        imageFilename: req.file ? req.file.filename : null
    };
    board.addPost(post);
    res.render('saved_post', { id: post.id });
});

```

```
// Servir la imagen asociada al post
router.get('/post/:id/image', (req, res) => {
    const post = board.getPost(req.params.id);
    if (!post || !post.imageFilename) {
        return res.status(404).send("Imagen no encontrada");
    }
    res.download(`${UPLOADS_FOLDER}/${post.imageFilename}`);
});

// Eliminacion con borrado fisico de la imagen en disco
router.get('/post/:id/delete', async (req, res) => {
    const post = board.deletePost(req.params.id);
    if (post && post.imageFilename) {
        try {
            await fs.rm(`${UPLOADS_FOLDER}/${post.imageFilename}`);
            console.log(`Archivo ${post.imageFilename} eliminado del disco`);
        } catch (err) {
            console.error("Error al borrar el archivo fisico:", err.message);
        }
    }
    res.render('deleted_post');
});
```

Listing 3.22: src/router.js con soporte de subida y borrado de imágenes

En la plantilla `views/show_post.html`, mostramos la imagen de forma condicional:

```
<h1>{{post.title}}</h1>
<p>Publicado por: {{post.user}}</p>
<p>{{post.text}}</p>

{{#post.imageFilename}}
    <div>
        <img src="/post/{{post.id}}/image" alt="Imagen del anuncio" style="max-width:400px; height:auto;">
    </div>
{{/post.imageFilename}}

<hr>
<a href="/post/{{post.id}}/delete">Eliminar anuncio</a> | <a href="/">Inicio</a>
```

Por último, el script de inicialización `load_data.js` orquesta la carga idempotente de datos:

```
import fs from 'node:fs/promises';
import * as board from './src/board.js';

const UPLOADS_FOLDER = './uploads';
const DATA_FOLDER = './data';

// 1. Leer y parsear el archivo JSON de anuncios
const dataRaw = await fs.readFile(`${DATA_FOLDER}/data.json`, 'utf8');
const anunciosDemo = JSON.parse(dataRaw);

for (const post of anunciosDemo) {
    board.addPost(post);
}

// 2. Limpiar la carpeta de subidas y recrearla
```

```
16 await fs.rm(UPLOADS_FOLDER, { recursive: true, force: true });
17 await fs.mkdir(UPLOADS_FOLDER, { recursive: true });
18
19 // 3. Copiar las imagenes de prueba a la carpeta de uploads activa
20 await fs.cp(`${DATA_FOLDER}/images`, UPLOADS_FOLDER, { recursive: true
       });
21
22 console.log("Datos de demostracion e imagenes cargados con exito.");
```

Listing 3.23: load_data.js: Inicialización de datos de demo e imágenes

3.7 Introducción a las Bases de Datos (3.3.1)

3.7.1 El Problema de la Persistencia en Memoria Volátil

En las secciones precedentes hemos gestionado la información del Tablón de Anuncios mediante estructuras de datos en memoria (`Map`). Aunque este enfoque resulta pedagógicamente directo, presenta limitaciones arquitectónicas insalvables en producción:

1. **Volatilidad:** Al detenerse el proceso de Node.js (por reinicios, despliegue de nuevas versiones o fallos imprevistos), toda la información almacenada en la memoria RAM se pierde irreversiblemente.
2. **Imposibilidad de Escalado Horizontal:** Si la carga de peticiones se incrementa y se despliegan múltiples instancias de la aplicación web tras un balanceador de carga, cada instancia mantendría su propia memoria local aislada, generando inconsistencias graves e inaceptables para los usuarios.
3. **Falta de Concurrencia Robusta:** La memoria local carece de mecanismos formales para garantizar transacciones, aislamiento y control de concurrencia ante miles de operaciones simultáneas.

Para resolver estos desafíos, la arquitectura de los sistemas software separa limpiamente la capa de aplicación de la **capa de persistencia**, materializada mediante un **Sistema Gestor de Bases de Datos (SGBD / DBMS)**.



Figura 3.3: Arquitectura típica de tres capas (Three-tier Architecture) en aplicaciones web.

3.7.2 Bases de Datos Relacionales (SQL) vs NoSQL

Históricamente, el panorama de la persistencia estuvo dominado por las **Bases de Datos Relacionales (RDBMS)**, fundamentadas en el modelo relacional formal de E.F. Codd:

- La información se organiza en **tablas** (relaciones) bidimensionales compuestas por filas (tuplas) y columnas tipadas (atributos).
- Las relaciones entre entidades se modelan mediante **claves primarias** (*Primary Keys*) y **claves foráneas** (*Foreign Keys*).
- Se consultan y manipulan mediante el lenguaje estandarizado **SQL** (*Structured Query Language*).

- Garantizan propiedades **ACID** estrictas (Atomicidad, Consistencia, Aislamiento y Durabilidad).

A partir de la década de 2000, la explosión de la Web 2.0 y el volumen masivo de datos impulsaron el movimiento **NoSQL** (*Not Only SQL*), motivado por tres causas primordiales:

▶ **Concepto Clave: Factores que impulsaron el auge de NoSQL**

1. **Escalabilidad Horizontal (*Sharding*):** Las bases de datos relacionales fueron concebidas para escalar verticalmente (adquiriendo servidores con más CPU y RAM). Distribuir tablas relacionales unidas por complejas operaciones `JOIN` a lo largo de un clúster de cientos de máquinas resulta extremadamente costoso y difícil. Las soluciones NoSQL fueron diseñadas desde su concepción para particionarse de forma nativa en clústeres horizontales.
2. **Desajuste de Impedancia (*Impedance Mismatch*):** Los desarrolladores modelan sus dominios en lenguajes orientados a objetos con jerarquías, arrays y objetos anidados en memoria. Mapear este grafo de objetos hacia tablas planas relacionales exige descomponer la información en múltiples tablas intermedias unidas por claves ajenas (requiriendo bibliotecas ORM como Hibernate o TypeORM que añaden complejidad).
3. **Flexibilidad de Esquema (*Schemaless*):** En una base de datos relacional, la estructura de las tablas debe definirse rígidamente antes de insertar el primer dato mediante sentencias DDL (`CREATE TABLE`). Modificar el esquema de una tabla con millones de filas en producción requiere costosas migraciones con bloqueos. Las bases de datos NoSQL permiten esquemas dinámicos donde cada registro puede adaptar su estructura de forma flexible.

3.8 MongoDB: Modelo de Documentos y Shell (3.3.2)

3.8.1 Modelo Documental, BSON y el Identificador `_id`

MongoDB es una base de datos orientada a documentos líder en la industria. La jerarquía de abstracción de MongoDB establece una correspondencia conceptual directa con el mundo relacional:

Concepto Relacional	Concepto MongoDB	Descripción Técnica
Base de Datos	Base de Datos (*Database*)	Contenedor lógico de nivel superior.
Tabla	Colección (*Collection*)	Conjunto agrupado de documentos sin esquema rígido obligatorio.
Fila / Tupla	Documento (*Document*)	Registro individual de información estructurada.
Columna	Campo / Atributo (*Field*)	Par clave-valor contenido dentro de un documento.
Clave Primaria (`PRIMARY KEY`)	Campo `_id` (`ObjectId`)	Identificador universal único e indexado de cada documento.

Tabla 3.5: Mapeo conceptual entre el modelo relacional y la base de datos documental MongoDB.

▶ **Concepto Clave: Documento BSON**

Internamente, MongoDB almacena y transmite los datos en formato **BSON** (*Binary JSON*). BSON es una representación binaria de documentos tipo JSON que proporciona:

- Alta velocidad de codificación y decodificación computacional.
- Capacidad de atravesar campos de forma rápida sin parsear el texto completo.
- Tipos de datos extendidos que no existen en el estándar JSON: enteros de 64 bits, números `Decimal128`, fechas nativas (`Date`), datos binarios y el tipo especial `ObjectId`.

Cada documento en MongoDB contiene obligatoriamente un campo denominado **`_id`** que actúa como clave primaria única. Si el usuario no suministra un valor explícito al insertar, el motor genera automáticamente un valor de tipo `ObjectId`:

- Es un valor binario de **12 bytes** (representado habitualmente como una cadena hexadecimal de 24 caracteres).
- Se compone internamente de: un timestamp de 4 bytes (momento exacto de creación en segundos desde epoch), un identificador de máquina/proceso de 5 bytes y un contador incremental de 3 bytes.
- Permite extraer de forma gratuita la fecha exacta de creación del documento sin requerir un campo `createdAt` adicional.

3.8.2 Ediciones y Despliegue con Docker

MongoDB se distribuye en tres modalidades principales:

- **Community Edition:** Versión gratuita y de código abierto para desarrollo y entornos locales.
- **Enterprise Edition:** Versión comercial que incorpora soporte de seguridad empresarial (Kerberos, LDAP, cifrado en reposo) y herramientas avanzadas de auditoría.
- **MongoDB Atlas:** Plataforma de base de datos totalmente gestionada en la nube (DBaaS) disponible sobre AWS, Google Cloud y Microsoft Azure.

Para desarrollo local, la mejor práctica de ingeniería consiste en instanciar MongoDB de forma aislada mediante un contenedor **Docker**:

```
$ docker run --rm -p 27017:27017 -d --name local-mongo mongo:8.2
```

El puerto predeterminado de comunicación de MongoDB es el **27017**.

Para inspeccionar y administrar la base de datos disponemos de:

- **MongoDB Compass:** Entorno gráfico de escritorio oficial (GUI).
- **mongosh (MongoDB Shell):** Interfaz interactiva de consola basada en JavaScript.

3.8.3 Operaciones CRUD en la Consola `mongosh`

Dentro de la consola interactiva `mongosh`, los comandos se ejecutan con sintaxis JavaScript:

1. Conexión y Selección de Base de Datos

```
// Conectar a la base de datos de pruebas (se crea automaticamente al
    insertar)
use tienda;
```

2. Inserción de Documentos

```
// Insercion de un documento individual
db.usuarios.insertOne({
    nombre: "Carlos Gomez",
    edad: 28,
    ciudad: "Madrid",
    preferencias: ["tecnologia", "fotografia"]
});
// Salida: { acknowledged: true, insertedId: ObjectId('66fa...') }
```

3. Consultas y Operadores de Filtro

```
// Consultar todos los documentos de la coleccion
db.usuarios.find();

// Consultar con filtro exacto
db.usuarios.findOne({ nombre: "Carlos Gomez" });

// Operadores de comparacion: $gt (>), $gte (>=), $lt (<), $lte (<=), $ne
    (!=)
db.usuarios.find({ edad: { $gte: 18 } });

// Operadores logicos: $and, $or
db.usuarios.find({
    $or: [
        { ciudad: "Madrid" },
        { edad: { $lt: 25 } }
    ]
});

// Proyecciones: 1 para incluir campo, 0 para excluir (el _id viene por
    defecto)
db.usuarios.find({ ciudad: "Madrid" }, { _id: 0, nombre: 1, edad: 1 });
```

4. Actualización y Borrado

```
// Actualizar el primer documento que cumpla la condicion mediante $set
db.usuarios.updateOne(
    { nombre: "Carlos Gomez" },
    { $set: { edad: 29 } }
);

// Borrar el primer documento coincidente
db.usuarios.deleteOne({ nombre: "Carlos Gomez" });

// Borrar todos los documentos de una coleccion
db.usuarios.deleteMany({ ciudad: "Madrid" });
```

3.9 MongoDB con Node.js (3.3.3)

3.9.1 El Driver Oficial `mongodb` y Conexión Asíncrona

Para conectar aplicaciones Node.js con MongoDB se emplea el driver oficial:

```
$ npm install --save mongodb
```

La comunicación con la base de datos involucra operaciones de E/S de red. En consecuencia, **todas las operaciones del cliente devuelven promesas (`Promise`)** y deben invocarse obligatoriamente mediante la sintaxis moderna **`async / await`**.

```
import { MongoClient, ObjectId } from 'mongodb';

const URI = 'mongodb://localhost:27017';
const client = new MongoClient(URI);

try {
    // 1. Establecer conexion con el cluster
    await client.connect();
    console.log("Conectado exitosamente a MongoDB");

    // 2. Obtener referencia a la base de datos y coleccion
    const db = client.db('cine');
    const peliculas = db.collection('peliculas');

    // 3. Insercion
    const resultado = await peliculas.insertOne({
        titulo: "Regreso al Futuro",
        director: "Robert Zemeckis",
        anio: 1985
    });
    console.log("Pelicula insertada con id:", resultado.insertedId);

} finally {
    // Cerrar cliente al finalizar el proceso
    await client.close();
}
```

Listing 3.24: Conexión y ciclo de vida básico con el driver de MongoDB

3.9.2 Operaciones de Inserción y Carga Masiva

- `collection.insertOne(doc)`: Inserta un documento único. Devuelve un objeto que contiene `insertedId`. Además, **el propio objeto pasado como argumento muta**, incorporando automáticamente la propiedad `_id` generada.
- `collection.insertMany([docs])`: Inserta un array de documentos en una única operación de red por lotes (*bulk insert*). Devuelve un mapa con todos los `insertedIds`.

Para inicializar bases de datos cargando ficheros JSON completos:

```
import { readFile } from 'node:fs/promises';

const datosStr = await readFile('./data/peliculas.json', 'utf8');
const listaPeliculas = JSON.parse(datosStr);

// Insercion masiva directa
const res = await peliculas.insertMany(listaPeliculas);
console.log(`${res.insertedCount} peliculas insertadas en la coleccion.`);
```

3.9.3 Consultas Avanzadas, Cursores, Paginación y Proyección

Procesamiento por Cursor vs `toArray()`

Al invocar `collection.find(filtro)`, MongoDB no transfiere de golpe todos los documentos a la memoria de la aplicación, sino que retorna un **Cursor**. Un cursor es un iterador que recupera los documentos en lotes bajo demanda a medida que se recorre:

```
// Enfoque 1: Carga total en un array en memoria (adecuado para colecciones pequenas)
const todos = await peliculas.find().toArray();

// Enfoque 2: Procesamiento secuencial mediante stream asincrono (adecuado para millones de filas)
const cursor = peliculas.find();
for await (const pelicula of cursor) {
    console.log("Procesando:", pelicula.titulo);
}
```

Filtros, Expresiones Regulares y Operadores Lógicos

```
// Consulta con operador $in (pertenencia a conjunto)
const cineClasico = await peliculas.find({
    anio: { $in: [1975, 1980, 1985] }
}).toArray();

// Consulta mediante expresiones regulares ($regex)
const peliculasAmor = await peliculas.find({
    titulo: { $regex: /love/i } // 'i' para busqueda insensible a mayusculas/minusculas
}).toArray();

// Consulta compuesta con $and y $or
const resultado = await peliculas.find({
    $and: [
        { anio: { $gte: 1980, $lte: 1989 } },
        { $or: [{ genero: "Ciencia Ficcion" }, { director: "Steven Spielberg" }] }
    ]
}).toArray();
```

Ordenación, Paginación y Proyección

En servicios web REST, devolver millones de registros satura la red y la memoria. Por ello, se implementa siempre paginación en el servidor combinando los métodos encadenables `.sort()`, `.skip()` y `.limit()`:

```
const pagina = 2;
const tamanoPagina = 10;

const resultadoPaginado = await peliculas.find()
    .sort({ anio: -1, titulo: 1 }) // Orden: anio descendente (-1), titulo ascendente (1)
    .skip((pagina - 1) * tamanoPagina) // Descartar los elementos de paginas previas
    .limit(tamanoPagina) // Limitar la respuesta al tamano de pagina
    .project({ _id: 1, titulo: 1, anio: 1 }) // Proyeccion: obtener unicamente titulo y anio
    .toArray();
```

3.9.4 Actualización y Eliminación de Documentos

- `updateOne(filtro, actualizacion)`: Modifica el primer documento que coincide con el filtro. Es imperativo utilizar operadores atómicos de actualización como **`$set`** (para modificar o añadir campos concretos) o **`$inc`** (para incrementar contadores numéricos). Si

se omite el operador y se pasa un objeto plano, las versiones modernas del driver lanzan error para prevenir la sustitución accidental de todo el documento.

```
const resUpdate = await peliculas.updateOne(
    { _id: new ObjectId(idCadena) },
    { $set: { recaudacion: 380000000, disponible: true } }
);
console.log(`Coincidentes: ${resUpdate.matchedCount}, Modificados: ${
    resUpdate.modifiedCount}`);
```

- `replaceOne(filtro, nuevoDocumento)`: Sustituye el documento por completo (conservando el mismo `_id`).
- `deleteOne(filtro)` y `deleteMany(filtro)`: Eliminan uno o múltiples documentos, devolviendo la propiedad `deletedCount`.

3.9.5 Gestión Avanzada de Arrays de Objetos Embebidos

Una de las capacidades más potentes de MongoDB consiste en modelar relaciones 1:N embebiendo arrays de subdocumentos dentro de un documento principal (por ejemplo, la lista de tareas dentro de un usuario o los comentarios dentro de un artículo).

Para manipular subdocumentos individuales dentro de un array sin sobrescribir el array entero, MongoDB dispone de operadores especializados:

```
// 1. Insercion de documento con subdocumentos que poseen su propio _id
const usuario = {
    nombre: "Juan",
    tareas: [
        { _id: new ObjectId(), titulo: "Aprender Node.js", hecha: true },
        { _id: new ObjectId(), titulo: "Aprender Express", hecha: false }
    ]
};
await usuarios.insertOne(usuario);

// 2. Agregar un nuevo subdocumento al array mediante $push
const nuevaTarea = { _id: new ObjectId(), titulo: "Aprender MongoDB", hecha:
    false };
await usuarios.updateOne(
    { _id: usuario._id },
    { $push: { tareas: nuevaTarea } }
);

// 3. Modificar un campo de un subdocumento especifico mediante el operador
    posicional $
// El filtro identifica la posicion coincidente dentro del array
await usuarios.updateOne(
    { _id: usuario._id, "tareas._id": nuevaTarea._id },
    { $set: { "tareas.$.hecha": true } }
);

// 4. Consultar y proyectar unicamente el subdocumento coincidente mediante
    $elemMatch
const docConSubdoc = await usuarios.findOne(
    { _id: usuario._id },
    { projection: { tareas: { $elemMatch: { _id: nuevaTarea._id } } } }
);
const tareaRecuperada = docConSubdoc.tareas[0];

// 5. Eliminar un subdocumento del array mediante $pull
await usuarios.updateOne(
```

```
    { _id: usuario._id },
    { $pull: { tareas: { _id: nuevaTarea._id } } }
);
```

Listing 3.25: Gestión de subdocumentos embebidos en arrays con MongoDB

3.9.6 Ejercicio Resuelto 1: Consultas Complejas sobre Catálogo de Películas

★ Ejercicio Resuelto: 1

Consultas sobre `movies.json` con el Driver de Node.js **Enunciado:** Cargar el conjunto de datos de películas provisto en `movies.json` en una colección de MongoDB y programar un script en Node.js que resuelva e imprima en consola las siguientes cuatro consultas de ingeniería:

1. El título y año de todas las películas producidas durante la década de los 80 (años 1980 a 1989 ambos inclusive).
2. La información íntegra de todas las películas que pertenezcan a la franquicia de *Star Wars*.
3. El título y año de las 5 películas más recientes registradas en la colección.
4. El título y género de las películas de género Romántico o que contengan la palabra `'love'` en su título (empleando expresiones regulares).

Solución y Análisis:

```
import { MongoClient } from 'mongodb';
import { readFile } from 'node:fs/promises';

const client = new MongoClient('mongodb://localhost:27017');

async function ejecutarConsultas() {
    try {
        await client.connect();
        const db = client.db('cine_db');
        const movies = db.collection('movies');

        // Carga inicial idempotente
        await movies.deleteMany({});
        const rawData = await readFile('./data/movies.json', 'utf8');
        await movies.insertMany(JSON.parse(rawData));
        console.log("Catalogo de peliculas cargado con exito.\n");

        // Consulta 1: Peliculas de los anos 80 proyectando titulo y anio
        console.log("=== 1. Peliculas de los anos 80 (1980-1989) ===");
        const decada80 = await movies.find({
            year: { $gte: 1980, $lte: 1989 }
        })
        .project({ _id: 0, title: 1, year: 1 })
        .toArray();
        console.log(decada80);

        // Consulta 2: Saga Star Wars (busqueda por patron en el titulo)
        console.log("\n=== 2. Peliculas de Star Wars ===");
        const starWars = await movies.find({
            title: { $regex: /Star Wars/i }
        }).toArray();
        console.log(starWars);

```

```
        // Consulta 3: Las 5 peliculas mas recientes
        console.log("\n=== 3. Las 5 peliculas mas recientes ===");
        const recientes = await movies.find()
            .sort({ year: -1 })
            .limit(5)
            .project({ _id: 0, title: 1, year: 1 })
            .toArray();
        console.log(recientes);

        // Consulta 4: Peliculas romanticas o con 'love' en el titulo
        console.log("\n=== 4. Peliculas Romanticas o con 'love' en el
    titulo ===");
        const romanticas = await movies.find({
            $or: [
                { genre: "Romance" },
                { genres: "Romance" },
                { title: { $regex: /love/i } }
            ]
        })
        .project({ _id: 0, title: 1, genre: 1, genres: 1 })
        .toArray();
        console.log(romanticas);

    } finally {
        await client.close();
    }
}

ejecutarConsultas().catch(console.error);
```

Listing 3.26: Script de consultas complejas sobre movies.json

3.10 Aplicaciones Web con MongoDB y Express (3.3.4)

3.10.1 Integración de la Capa de Persistencia en Express

El paso definitivo en el desarrollo del backend consiste en conectar la capa de controladores de Express con la base de datos documental MongoDB, sustituyendo por completo las colecciones efímeras en memoria (`Map`).

La integración exige tres adaptaciones sistemáticas:

1. **Gestión de Identificadores:** El identificador único pasa a ser el campo `_id` de tipo `ObjectId`. En los parámetros de ruta de Express, las URLs reciben identificadores como cadenas de texto (`req.params.id`), las cuales deben transformarse mediante `new ObjectId(req.params.id)` al consultar o modificar la base de datos.
2. **Controladores de Ruta Asíncronos:** Los manejadores de rutas en Express deben declararse obligatoriamente con la palabra reservada **`async (req, res) => { ... }`** para poder suspender la ejecución con `await` mientras MongoDB resuelve las operaciones de red.
3. **Reutilización de la Conexión:** En aplicaciones web en producción, **nunca se abre y cierra un `MongoClient` en cada petición HTTP**. El cliente mantiene internamente una agrupación de conexiones (*Connection Pool*). Se inicializa una única instancia de `MongoClient` al arrancar el servidor y se comparte entre todos los controladores.

3.10.2 Ejercicio Resuelto Completo: Tablón de Anuncios Persistente

★ Ejercicio Resuelto: 1

Tablón de Anuncios con Persistencia en MongoDB **Enunciado:** Refactorizar integralmente la aplicación web del Tablón de Anuncios (Ejercicios 4 y 5) para que todos los anuncios se persistan de forma duradera en una colección de MongoDB denominada `posts` dentro de la base de datos `board_db`. Mantener el soporte de subida y borrado de imágenes adjuntas.

Solución y Análisis:

Reescribimos la capa de datos en `src/board.js` para que delegue directamente en la colección de MongoDB:

```
import { MongoClient, ObjectId } from 'mongodb';

const URI = 'mongodb://localhost:27017';
const client = new MongoClient(URI);
await client.connect();

const db = client.db('board_db');
const postsCollection = db.collection('posts');

export async function addPost(post) {
    // MongoDB asigna y devuelve el _id automaticamente
    const result = await postsCollection.insertOne(post);
    post._id = result.insertedId;
    return post;
}

export async function getPosts() {
    return await postsCollection.find().toArray();
}

export async function getPost(id) {
    try {
        return await postsCollection.findOne({ _id: new ObjectId(id) })
    ;
    } catch (err) {
        // Captura de IDs con formato hexadecimal invalido
        return null;
    }
}

export async function deletePost(id) {
    try {
        const post = await getPost(id);
        if (post) {
            await postsCollection.deleteOne({ _id: new ObjectId(id) });
        }
        return post;
    } catch (err) {
        return null;
    }
}
```

Listing 3.27: src/board.js: Capa de persistencia con driver oficial de MongoDB

Actualizamos el enrutador `src/router.js` para utilizar funciones controladoras asíncronas:

```
import express from 'express';
import multer from 'multer';
import fs from 'node:fs/promises';
import * as board from './board.js';

const router = express.Router();
const UPLOADS_FOLDER = './uploads';
const upload = multer({ dest: UPLOADS_FOLDER });

// Listado de anuncios desde MongoDB
router.get('/', async (req, res) => {
    const posts = await board.getPosts();
    res.render('index', { posts });
});

// Insercion asincrona de nuevo anuncio
router.post('/post/new', upload.single('image'), async (req, res) => {
    const post = {
        user: req.body.user,
        title: req.body.title,
        text: req.body.text,
        imageFilename: req.file ? req.file.filename : null,
        fechaCreacion: new Date()
    };
    await board.addPost(post);
    // Redireccion PRG o renderizado con el _id generado por MongoDB
    res.render('saved_post', { id: post._id });
});

// Consulta de detalle de anuncio por su _id
router.get('/post/:id', async (req, res) => {
    const post = await board.getPost(req.params.id);
    if (!post) {
        return res.status(404).send("Anuncio no encontrado");
    }
    res.render('show_post', { post });
});

// Servicio de imagen
router.get('/post/:id/image', async (req, res) => {
    const post = await board.getPost(req.params.id);
    if (!post || !post.imageFilename) {
        return res.status(404).send("Imagen no disponible");
    }
    res.download(`${UPLOADS_FOLDER}/${post.imageFilename}`);
});

// Eliminacion en base de datos y borrado fisico de imagen
router.get('/post/:id/delete', async (req, res) => {
    const post = await board.deletePost(req.params.id);
    if (post && post.imageFilename) {
        try {
            await fs.rm(`${UPLOADS_FOLDER}/${post.imageFilename}`);
        } catch (err) {
            console.error("Error al borrar el archivo fisico:", err.
    message);
        }
    }
    res.render('deleted_post');
});

```

```
61 export default router;
```

Listing 3.28: src/router.js: Controladores asíncronos con MongoDB

En las plantillas Mustache (`views/index.html` y `views/saved_post.html`), la única adaptación requerida consiste en utilizar la propiedad **`{{_id}}`** generada por MongoDB en lugar del antiguo **`{{id}}`**:

```
1  <!DOCTYPE html>
2  <html>
3  <head><meta charset="UTF-8"><title>Tablon MongoDB</title></head>
4  <body>
5      <h1>Tablon de Anuncios (Persistencia MongoDB)</h1>
6      {{#posts}}
7          <p><a href="/post/{{_id}}">{{user}}: {{title}}</a></p>
8      {{/posts}}
9      {{^posts}}
10         <p>No hay anuncios en la base de datos.</p>
11     {{/posts}}
12     <hr>
13     <a href="/new_post.html">Crear nuevo anuncio</a>
14 </body>
15 </html>
```

Listing 3.29: views/index.html adaptado a MongoDB

Con esta arquitectura, el sistema garantiza persistencia duradera de la información, desacoplamiento estricto de responsabilidades, seguridad contra pérdidas de memoria y capacidad de escalabilidad horizontal.